

ORACLE
NetSuite

NetSuite CPQ Configurator Guide

2025.2

December 17, 2025



Copyright © 2005, 2025, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, and MySQL are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

If this document is in public or private pre-General Availability status:

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <https://www.oracle.com/corporate/accessibility>.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

Sample Code

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and "as available", for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at www.netsuite.com/tos.

Oracle may modify or remove sample code at any time without notice.

No Excessive Use of the Service

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer's use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle's other customers, Oracle may slow down or throttle Customer's excess use until such time that Customer's use stays within reasonable limits. If Customer's particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer's actual usage pattern.

Beta Features

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Table of Contents

NetSuite CPQ Configurator	1
Updated Labels for User Interface Elements	1
NetSuite CPQ Configurator Setup	23
Configuring Items from the Customer Center	26
NetSuite CPQ Configurator Products	27
Activating or Hiding Building Blocks with Rules	35
Working with the Product User Interface	39
Calculating the Final Pricing for the Configurable Item	78
Including Additional Items Related to the Configurable Item	80
Creating Work Orders for Configured Items	84
Creating Items Required by the Configuration	94
Creating Records Required by the Configuration	105
Setting Transaction Body and Line Fields Based on Configuration Data	111
Working with Product Configurations	114
Including Additional Conditions in Rules and Actions with Predefined Answers	115
Working with Configurable Subscriptions	120
Customizing Products with Scripts Using Action Records and Functions	134
Configuring Items From Transactions	155

NetSuite CPQ Configurator

Your account lets you create all kinds of items, such as inventory items, non-inventory items, and assemblies. To work with configurable items, you can use NetSuite CPQ Configurator.

NetSuite CPQ Configurator lets users configure complex items according to your business logic by selecting features and characteristics from a predefined set of options. After configuring the items, users can add them to transactions.

With NetSuite CPQ Configurator, you can:

- Define a visual space where users can configure complex items with thousands of possible combinations.
- Gather information about users' preferences with questions and answers on the product interface.
- Guide the configuration process by providing real-time feedback on users' choices with validation messages.
- Work with a complex price structure, which calculates and updates the final price for the main configurable item in real time based on users' choices.
- Display a dynamic 2D visual preview of the item while it is being configured.
- Automatically create items and records required by the configuration.
- Automatically set body and line field values for transactions with configured items.
- Save frequently used configurations and define a default configuration that automatically loads predefined options and features.
- Add accessories for the configured item to transactions.
- Use the interactive generative AI tool, NetSuite CPQ AI Assistant, to help users work with configurable items by suggesting valid options, clarifying configuration constraints, and prefilling answers based on user intent. Configure where and when the assistant appears, and ensure AI-generated content meets quality standards with tools like Text Enhance. For details, see the help topic [NetSuite CPQ AI Assistant](#).

If you install NetSuite CPQ Manufacturing, you can create work orders based on materials and routing steps from NetSuite CPQ Configurator. For more information, [Creating Work Orders for Configured Items](#).

After installing NetSuite CPQ Configurator, set up basic and additional options. For more information, see [NetSuite CPQ Configurator Setup](#).

After setting up NetSuite CPQ Configurator, you can create products. A product is the main working unit in this SuiteApp and represents the configurable item. Each product is made up of several building blocks activated by rules. For more information, see [NetSuite CPQ Configurator Products](#) and [Activating or Hiding Building Blocks with Rules](#).

When products are ready, users can configure the associated complex items from transactions. For more information, see [Configuring Items From Transactions](#).

Updated Labels for User Interface Elements

NetSuite CPQ Configurator includes updated labels for some elements you see on the user interface. Clearer and more informative labels provide a consistent user experience across the platform. Labels have been updated for the following elements:

- records

Note: Record labels no longer include hyphens.

- fields
- field options
- sections
- subtabs
- columns

Note: The changes only affect the labels you see on the user interface. The IDs of all records and fields remain unchanged. Scripts that reference those IDs will continue to run as expected.

The following table summarizes the updated labels for records.

Previous Record Label	New Record Label
Composite-Image	Image
Component-Field-Mapping	Line Field Mapping
Breakout-Item	Additional Item
Category	Rule Category
Progress	Progress Popup
RCR Sublist Component	RCR Sublist Line

Most records include a **Rule** or **Rules** section or subtab. Several fields and options in those sections or subtabs have been renamed. For more information, see [Updated Labels for Rule-Related Fields and Options](#).

Similarly, translation records and some of their fields have been renamed as well. For more information, see [Updated Labels for Translation Records and Fields](#).

Additionally, the **General** section has been renamed to **Primary Information** on all records. The **Notes** subtab has been renamed **System Notes** on all records.

Some fields on the product user interface have also been renamed. For more information, see [Updated Labels on the Product User Interface](#).

NetSuite CPQ Configurator creates fields on some external records. Some of those fields have been renamed. For more information, see [Updated Labels on External Records](#).

Choose a record to view the new labels, which are organized by record type:

Note: Options are listed under the updated label for the parent field (if applicable).

- [Updated Labels on the Options Record](#)
- [Updated Labels on the Product Record](#)
- [Updated Labels on the Question Record](#)
- [Updated Labels on the Answer Record](#)
- [Updated Labels on the Action Record](#)
- [Updated Labels on the Material Record](#)
- [Updated Labels on the Routing Step Record](#)

- Updated Labels on the Validation Record
- Updated Labels on the Pricing Record
- Updated Labels on the Table Record
- Updated Labels on the Image Record (Former Composite Image)
- Updated Labels on the Body Field Mapping Record
- Updated Labels on the Line Field Mapping Record (Former Component Field Mapping)
- Updated Labels on the Additional Item Record (Former Breakout Item)
- Updated Labels on the Rule Category Record (Former Category)
- Updated Labels on the Tab Record
- Updated Labels on the Group Record
- Updated Labels on the Progress Popup Record (Former Progress)
- Updated Labels on the Item Creation Record
- Updated Labels on the ITCR Component Record
- Updated Labels on Record Creation
- Updated Labels on the RCR Sublist Record
- Updated Labels on the RCR Sublist Line Record (Former RCR Sublist Component)
- Updated Labels on the Local Language Record
- Updated Labels on the Product Category Record
- Updated Labels on the Module Record
- Updated Labels on the Common Action Record
- Updated Labels on the Metrics Record

Updated Labels on the Options Record

The **General** section has been renamed to **Primary Information**. In this section, the following fields and options have been renamed:

Label	New Label	Type	Parent Field
Sequence	Sequence Number	Option label	Sort Products By
Submit to order page	Submit to transaction page	Field label	—
Enable translations	Show label translations	Field label	—
Default Configurator	Default Product	Field label	—
Clean Unused Configurations Older Than	Delete Unused Configurations Older Than	Field label	—
Disabled	Inactive	Option label	Delete Unused Configurations Older Than

The **Export/Import** section has been renamed to **Export/Import Products**. In this section, the following fields and options have been renamed:

Label	New Label	Type	Parent Field
NS links import options	Import Options for Record References	Field label	—

Label	New Label	Type	Parent Field
Don't import links	Do not import	Option label	Import Options for Record References
Import links using same IDs	Import by ID	Option label	Import Options for Record References
Import links using search by name	Import by name	Option label	Import Options for Record References

The **Show Configurator Buttons On Transactions** section has been renamed to **Add Configure Button to Transactions**. In this section, the following field has been renamed:

Field Label	New Field Label
Transactions	Transaction Types

The **Quote / Order Fields** subtab has been renamed to **Transaction Fields**. On this subtab, the following fields, subtabs, and columns have been renamed:

Label	New Label	Type
#	Line Number	Column label
Remove	Remove Line	Column label
Add	Add Field	Column label
Raw Data	JSON Data	Subtab label
Order Fields	Transaction Fields	Field label

On the **Appearance** subtab, the following fields and options have been renamed:

Label	New Label	Type	Parent Field
Embedded	Embedded in transaction	Option label	Configurator Start Type
Window Size	Popup Window Size	Field label	—
Full size	Full-Screen	Option label	Popup Window Size
90% size	90%-Screen	Option label	Popup Window Size

The **File Deletion System** subtab has been renamed to **Delete Temporary Files**. In this section, the following columns have been renamed:

Column Label	New Column Label
#	Line Number
Period	Number of Days
Remove	Remove Line
Add	Add Directory

On the **Submit** subtab, the following fields have been renamed:

Field Label	New Field Label
Serial Item Creation	Create items one by one
Serial Record Creation	Create records one by one
Update transaction entity	Add new resources to transaction fields

Updated Labels on the Product Record

The **General** section has been renamed to **Primary Information**. In this section, the following fields have been renamed:

Field Label	New Field Label
Sequence	Sequence Number
Disabled	Inactive
Disable cache	Deactivate cache
Check rules on every change	Update data on every change
Load tables for actions	Preload static and CSV tables for actions
Disallow saving of incomplete configs	Prevent submitting incomplete configurations
Disallow saving of invalid configs	Prevent submitting incompatible configurations
Separate all CPI in carousel	Enable image gallery
Fractional numbers on quantity	Allow decimal number in Quantity field
Consolidate items	Consolidate additional item lines
Merge data mappings for consolidated items	Merge mappings for consolidated additional item lines
Calculate rate for consolidated items	Calculate rate for consolidated additional item lines
Show Links container	Display link container
Links Container Sequence	Link Container Sequence Number
Category	Product Category
Product Special Field	Product Restriction Field
Default Tab	Start Tab
Enable external launch	Available on SuiteCommerce

The **Base Product** section has been renamed to **Base Item**. In this section, the following fields have been renamed:

Field Label	New Field Label
Base Generic Product (Created)	Base Item (Item Creation Record)
Base Generic Product	Base Item
Apply Column Mappings	Line Field Mapping Records

On the **Appearance** subtab, the following fields have been renamed:

Field Label	New Field Label
Default Image	Cover Image
Layout Style	Question Layout
CPI Height (PX)	Image Container Height (px)
Remove CPI borders	Remove image borders
Remove tabs line padding	Remove space around the tab line
Tab Border Width (PX)	Tab Spacing (px)
Tab Font Color	Tab Label Color
Tab Background Color	Tab Color
Tab Active Background Color	Active Tab Color
Group Header Background Color	Group Header Color
Group Header Font Color	Group Label Color
Question Header Background Color	Question Header Color
Question Header Font Color	Question Label Color
Custom HTML	Footer

The following subtabs have been renamed:

Subtab Label	New Subtab Label
Routing	Routing Steps
Composite Images	Images
Items	Additional Items
Categories	Rule Categories
Progress	Progress Popups

Updated Labels on the Question Record

The **General** section has been renamed to **Primary Information**. In this section, the following fields and options have been renamed:

Label	New Label	Type	Parent Field
Answer Type	Type	Field label	—
Break	Line Break	Option label	Type
Checkbox/Radio	Checkbox/Radio Button	Option label	Type
Dropdown	Dropdown List	Option label	Type

Label	New Label	Type	Parent Field
Inline HTML	Custom HTML	Option label	Type
Separator	Horizontal Line	Option label	Type
Table	QTable	Option label	Type
Tiles Dropdown	Tile Dropdown List	Option label	Type
Image gallery	Image	Option label	Attachment Type
Select Type	Answer Selection Type	Field label	—
None	Default	Option label	Answer Selection Type
Single	Single Choice	Option label	Answer Selection Type
Multiple	Multiple Choice	Option label	Answer Selection Type
Sort by Answer Sequence	Sequence Number	Option label	Sort Answers By
Sort by Name	Name	Option label	Sort Answers By
Question Spans	UI Columns	Field label	—
Label HTML	Label	Field label	—

On the **Appearance** subtab, the following fields have been renamed:

Field Label	New Field Label
Hide question label	Hide label
Hide line	Hide horizontal line
Hide title background	Hide label background color
Collapsible	Collapse tiles

On the **QTable** subtab, the following fields have been renamed:

Field Label	New Field Label
Table Rows Count (Initial)	Initial Rows
Table Height Limit (Rows)	Maximum Visible Rows
Dynamic Row Count	Allow row number editing
Manual Edit	Allow cell editing

Updated Labels on the Answer Record

The section **Product/Question** has been renamed to **Background Information**. In this section, the following field has been renamed:

Field Label	New Field Label
Question Answer Type	Question Type

The **General** section has been renamed to **Primary Information**. In this section, the following fields and options have been renamed:

Label	New Label	Type	Parent Field
Picklist	Popup Table	Option label	Input Box Answer Type
Picklist (dropdown)	Popup List	Option label	Input Box Answer Type
QTable Cell Type	QTable Column Type	Field label	—
Text input	Text Field	Option label	QTable Column Type
Picklist (dropdown)	Popup List	Option label	QTable Column Type
Dropdown	Dropdown List	Option label	QTable Column Type
Date/Time	Date & Time Field	Option label	QTable Column Type
Inline HTML	Custom HTML	Option label	QTable Column Type
hh:mm A	hh:mm AM/PM	Option label	Time Format
HH:mm	hh:mm (24 hours)	Option label	Time Format
hh-mm A	hh-mm AM/PM	Option label	Time Format
HH-mm	hh-mm (24 hours)	Option label	Time Format
Add text input	Add text field	Field label	—
Add text area	Add text box	Field label	—
Input Label Text	Input Group Label	Field label	—
Input label on the right	Show input group label on the right	Field label	—

On the **Appearance** subtab, the following fields have been renamed:

Field Label	New Field Label
Answer Image	Image
Hide answer label	Hide label
Show placeholder	Show label as placeholder
Tile Span	UI Columns
Show input when unchecked	Always show text field or box
Label HTML	Label
Tooltip HTML	Tooltip

The **Picklist** subtab has been renamed to **Popup Tables & Lists**. On this subtab, the following fields and sections have been renamed:

Label	New Label	Type
General	General Information	Section label
Table Column Display Configurator	Select Product Table Columns	Field label

Label	New Label	Type
Table Filter	Product Table Filter	Field label
Table Filter Bar Configurator	UI Popup Table Filter	Field label
Multi-select picklist	Allow multiple selections	Field label
Ignore read-only rule for pick button	View when read-only	Field label
Table Reference	Reference Table	Field label
Column in Table to Get Value	Reference Column	Field label
Dropbox options	Popup List	Section label
Custom Picklist Option	Popup List Options	Field label
Filter by value	Allow search before loading	Field label
Column(s) to Q/A Configuration	Mapping to Questions/Answers	Field label
Column(s) to QTable Configuration	Mapping to QTable	Field label
Q/A for Row(s) Mapping	Mapping JSON Rows to Question/Answer	Field label
QTable for Mapping	Target QTable	Field label
Clear QTable before mapping	Clear qTable cell before mapping	Field label
Clear mapped QAs on reset	Clear mapped questions/answers on reset	Field label

The **QTable Cell** subtab has been renamed to **QTable Cells**. On this subtab, the following fields, options, and sections have been renamed:

Field Label	New Field Label
QCell Dropdown Options	QTable Dropdown List Options
Inline HTML Content	QTable Custom HTML Content

Updated Labels on the Action Record

On the action record, the **General** section has been renamed to **Primary Information**. In this section, the following fields have been renamed:

Field Label	New Field Label
Category	Rule Category
Sequence	Sequence Number
Run only on first matching	Run only on first match

The section **Code Exec** has been renamed to **Code Editor**. In this section, the following field has been renamed:

Field Label	New Field Label
Add Action	Add Function

Updated Labels on the Material Record

On the material record, the **General** section has been renamed to **Primary Information**. In this section, the following fields have been renamed:

Field Label	New Field Label
Display Description	Description
Image	Item Display Image
Category	Rule Category
Routing	Routing Step
Effective Date	Effective Start Date
Cancel Date	Effective End Date
Phantom BOM	Phantom Material

The **Item** section has been renamed to **Material**. In this section, the following fields have been renamed:

Field Label	New Field Label
Item from Q/A	Resolve Item Label/Number
Item Creation	Item Creation Record

The **Quantities** section has been renamed to **Material Details**. In this section, the following fields have been renamed:

Field Label	New Field Label
Multiplier	Quantity Multiplier
UOM	Primary Sale Unit
Units	Primary Units Type
Apply Column Mappings	Line Field Mapping Records
Common Column Mappings	Common Line Field Mapping Records

Updated Labels on the Routing Step Record

On the routing step record, the **General** section has been renamed to **Primary Information**.

Field Label	New Field Label
Effective Date	Effective Start Date
Cancel Date	Effective End Date
Advanced manufacturing operation	Enable AM work bench operation fields

The **Operation Properties** section has been renamed to **Manufacturing Operation Task Details**. In this section, the following fields have been renamed:

Field Label	New Field Label
Setup Time	Setup Time (min)
Run Rate	Run Rate (min/unit)

The fields related to mapping records have been renamed as follows:

Field Label	New Field Label
Apply Column Mappings	Line Field Mapping Records
Common Column Mappings	Common Line Field Mapping Records

The **Advanced Manufacturing** section has been renamed to **Advanced Manufacturing Work Bench Operation**. In this section, the following fields have been renamed:

Field Label	New Field Label
AM Operation Name	Operation Name
AM Lag Type	Lag Type
AM Lag Value	Lag Value
AM Hold Time	Hold Time (min)
Incremental Quantity	Incremental QTY
AM Process Notes	Process Notes

Updated Labels on the Validation Record

On the validation record, the **General** section has been renamed to **Primary Information**. In this section, the following fields and options have been renamed:

Label	New Label	Type	Parent Field
Incomplete	Incomplete Configuration	Option label	Type
Invalid	Incompatible Configuration	Option label	Type
Effective Date	Effective Start Date	Field label	—
Cancel Date	Effective End Date	Field label	—
Category	Rule Category	Field label	—

Updated Labels on the Pricing Record

On the pricing record, the **General** section has been renamed to **Primary Information**. In this section, the following fields have been renamed:

Field Label	New Field Label
Category	Rule Category

Field Label	New Field Label
Effective Date	Effective Start Date
Cancel Date	Effective End Date
HTML Content	Label

In the **Pricing** section, the following fields have been renamed:

Field Label	New Field Label
Multiplier	Price Multiplier
Hide pricing	Hide price amount
Hide price (Total line)	Hide price line
Recalc subtotal	Calculate subtotal

Updated Labels on the Table Record

On the Table record, the **General** section has been renamed to **Primary Information**. In this section, the following fields and options have been renamed:

Label	New Label	Type	Parent Field
Table Type	Type	Field label	—
Static	Static Table	Option label	Type
Dynamic	Dynamic Table	Option label	Type
CE generated	Function-Generated Table	Option label	Type
CSV	CSV Table	Option label	Type
Document	CSV File	Field label	—
Use custom table data	Unlink static table from CSV file	Field label	—
Preload file data	Allow preloading CSV table	Field label	—

In the **CSV Table** section, the following fields and options have been renamed:

Label	New Label	Type	Parent Field
Delimiter	Column Separator	Field label	—
Double Quotes "	Double Quotation Marks	Option label	Escape Characters
Single Quotes '	Single Quotation Marks	Option label	Escape Characters

The **Raw Data** subtab has been renamed to **JSON Data**. On this subtab, the following field has been renamed:

Field Label	New Field Label
Rendered as JSON	Table

The **Table Columns** subtab has been renamed to **Table Editor**.

Updated Labels on the Image Record (Former Composite Image)

The composite image record has been renamed to image.

On the image record, the **General** section has been renamed to **Primary Information**. In this section, the following field has been renamed:

Field Label	New Field Label
Category	Rule Category

The section **Image Data** has been renamed to **Image Details**. In this section, the following fields, options, subtabs, and columns have been renamed:

Field Label	New Field Label
Carousel Group Code	View Code

Updated Labels on the Body Field Mapping Record

On the body field mapping record, the **General** section has been renamed to **Primary Information**. In this section, the following field has been renamed:

Field Label	New Field Label
Sequence	Sequence Number

The **Fields** subtab has been renamed to **Transaction Body Fields**. On the **Select Fields** subtab under the **Transaction Body Fields** subtab, the following column has been renamed:

Column Label	New Column Label
Emulated Value	Placeholder Value

Under the **Transaction Body Fields** subtab, the **Raw Data** subtab has been renamed to **JSON Data**. On this tab, the following field has been renamed:

Field Label	New Field Label
Fields	Transaction Body Fields

Updated Labels on the Line Field Mapping Record (Former Component Field Mapping)

The component field mapping record has been renamed to line field mapping.

On this record, the **General** section has been renamed to **Primary Information**. In this section, the following fields have been renamed:

Field Label	New Field Label
Sequence	Sequence Number
Apply to all items	Apply to all additional items
Apply to all routings	Apply to all routing steps

The **Fields** subtab has been renamed to **Transaction Line Fields**. Under the **Select Fields** subtab of the **Transaction Line Fields** subtab, the following column has been renamed:

Column Label	New Column Label
Emulated Value	Placeholder Value

Under the **Transaction Line Fields** subtab, the **Raw Data** subtab has been renamed to **JSON Data**. On this tab, the following field has been renamed:

Field Label	New Field Label
Fields	Transaction Line Fields

Updated Labels on the Additional Item Record (Former Breakout Item)

The breakout item record has been renamed to additional item.

On the additional item record, the **General** section has been renamed to **Primary Information**. In this section, the following fields have been renamed:

Field Label	New Field Label
Category	Rule Category
Category Sequence	Rule Category Sequence Number
Effective Date	Effective Start Date
Cancel Date	Effective End Date

The **Item section** has been renamed to **Additional Item**. In this section, the following fields have been renamed:

Field Label	New Field Label
Item from Q/A	Resolve Item Name/Number
Item Creation	Item Creation Record

In the **Pricing** section, the following field has been renamed:

Field Label	New Field Label
QTY	Quantity

The following fields related to mapping records have been renamed:

Field Label	New Field Label
Apply Column Mappings	Line Field Mapping Records
Common Column Mappings	Common Line Field Mapping Records

Updated Labels on the Rule Category Record (Former Category)

The category record has been renamed to rule category.

On this record, the **General** section has been renamed to **Primary Information**. In this section, the following fields have been renamed:

Field Label	New Field Label
Effective Date	Effective Start Date
Cancel Date	Effective End Date

Updated Labels on the Tab Record

On the tab record, the **General** section has been renamed to **Primary Information**. In this section, the following fields have been renamed:

Field Label	New Field Label
Is modal	Display groups as popup windows
Label HTML	Label
Picklist Reference	Add to Popup Table

The **Custom** subtab has been renamed to **Groups**.

Updated Labels on the Group Record

On the group record, the **General** section has been renamed to **Primary Information**. In this section, the following field has been renamed:

Field Label	New Field Label
Label HTML	Label

In the **Appearance** section, the following fields have been renamed:

Field Label	New Field Label
Collapsible	Allow collapsing

Field Label	New Field Label
Spans	UI Columns

Updated Labels on the Progress Popup Record (Former Progress)

The progress record has been renamed to progress popup.

On this record, the **General** section has been renamed to **Primary Information**. In this section, the following fields have been renamed:

On this record, the following fields have been renamed:

Field Label	New Field Label
Progress Code	Code
Progress Text	Message

Updated Labels on the Item Creation Record

On item creation the record, the **General** section has been renamed to **Primary Information**. In this section, the following options have been renamed:

Option Label	New Option Label	Parent Field
Don't create if exists	Do not create a new item	Existing Item Handling
Update existing item	Update the existing item	Existing Item Handling
Create item with new SKU	Create the item with a new name/number	Existing Item Handling

The **Item Data** section has been renamed to **New Item**. In this section, the following fields have been renamed:

Field Label	New Field Label
Type	Item Type
Item ID (SKU)	Item Name/Number

In the **Template Item** section, the following fields have been renamed:

Field Label	New Field Label
Template Item from QA	Resolve Template Item Name/Number
Template Item ID (SKU)	Template Item Name/Number
Item ID (SKU) Suffix	Item Name/Number Suffix

In the **Item Creation** section, the following field has been renamed:

Field Label	New Field Label
Write ID to QA	Answer for Item ID

The **Advanced** section has been renamed to **Manufacturing**. In this section, the following fields, options, subtabs, and columns have been renamed:

Field Label	New Field Label
Use all materials as components	Add all materials as components

The **Fields** subtab has been renamed to **Item Fields**. Under the **Select Fields** subtab of the **Item Fields** subtab, the following column has been renamed:

Column Label	New Column Label
Emulated Value	Placeholder Value

Under the **Item Fields** subtab, the **Raw Data** subtab has been renamed to **JSON Data**.

In the **Bill of Materials** section under the **BOM** subtab, the following fields have been renamed:

Field Label	New Field Label
Name	BOM Name
Memo	BOM Memo

Updated Labels on the ITCR Component Record

On the ITCR component record, the **General** section has been renamed to **Primary Information**. In this section, the following fields, options, subtabs, and columns have been renamed:

Field Label	New Field Label
Product Link	Product
Parent (ITCR)	Item Creation Record

In the **Item** section, the following fields have been renamed:

Field Label	New Field Label
From Item	Item
From CPQ Material	Material
From CPQ Created Item	Item Creation Record

On the **Select Column** subtab, the following column has been renamed:

Column Label	New Column Label
Emulate	Placeholder Value

The **Raw Data** subtab has been renamed to **JSON Data**.

Updated Labels on Record Creation

On the record named record creation, the **General** section has been renamed to **Primary Information**. In this section, the following fields, options have been renamed:

Option Label	New Label	Parent Field
Don't create if exists	Do not create a new record	Existing Record Handling
Update existing record	Update the existing record	Existing Record Handling
Create record with a different key field value	Create the record with a new key field value	Existing Record Handling

The **Record Data** section has been renamed to **New Record**. In this section, the following field has been renamed:

Field Label	New Field Label
Type	Record Type

In the **Template Record** section, the following field has been renamed:

Field Label	New Field Label
Template Record from QA	Resolve Template Record Key Field

In the **Record Creation** section, the following fields have been renamed:

Field Label	New Field Label
Write ID to QA	Answer for Record ID
Dynamic Mode	Enable dynamic mode

The **Fields** subtab has been renamed to **Record Fields**. Under the **Select Fields** subtab of the **Record Fields** subtab, the following column has been renamed:

Column Label	New Column Label
Emulated Value	Placeholder Value

Under **Record Fields** subtab, the **Raw Data** subtab has been renamed to **JSON Data**.

Updated Labels on the RCR Sublist Record

On the RCR sublist record, the **General** section has been renamed to **Primary Information**. In this section, the following fields and options have been renamed:

Label	New Label	Type	Parent Field
Clear	Delete & add new lines	Option label	Line Add Method
Append	Append new lines	Option label	Line Add Method
Overwrite	Overwrite existing lines	Option label	Line Add Method

The **Components** subtab has been renamed to **Sublist Lines**.

Updated Labels on the RCR Sublist Line Record (Former RCR Sublist Component)

The RCR sublist component record has been renamed to RCR sublist line.

On the RCR sublist line record, the **General** section has been renamed to **Primary Information**. In this section, the following field has been renamed:

Field Label	New Field Label
Sublist	RCR Sublist Record

The **Fields** subtab has been renamed to **Sublist Line Fields**. Under the **Select Fields** subtab of the **Sublist Line Fields** subtab, the following column has been renamed:

Column Label	New Column Label
Emulated Value	Placeholder Value

Under the **Sublist Line Fields** subtab, the **Raw Data** subtab has been renamed to **JSON Data**.

On the **JSON Data** subtab, the following field has been renamed:

Field Label	New Field Label
Columns	Sublist Line Fields

Updated Labels on the Local Language Record

On the local language record, the following field has been renamed:

Field Label	New Field Label
Code	Language Code

Updated Labels on the Product Category Record

On the product category record, the **General** section has been renamed to **Primary Information**. In this section, the following field has been renamed:

Field Label	New Field Label
Display SO	Display on transactions

Updated Labels on the Module Record

On the module the record, the **General** section has been renamed to **Primary Information**. In this section, the following field has been renamed:

Field Label	New Field Label
File (HTML)	HTML File

Updated Labels on the Common Action Record

On the common action record, the **General** section has been renamed to **Primary Information**. In this section, the following field has been renamed:

Field Label	New Field Label
Code Exec	Code

Updated Labels on the Metrics Record

In the **Metrics Query** section on the metrics records, the following fields and options have been renamed:

Label	New Label	Type	Parent Field
CPQC-Product	Product	Field label	—
Item Rule	Additional Item	Option label	—
Material Rule	Material	Option label	Search By
CPQC Item	Additional Item	Field label	—
CPQC Material	Material	Field label	—
CPQC Question	Question	Field label	—
Item ID	Item Name/Number	Option label	Search By
Display Name	Display Name/Code	Option label	Search By

In the **Results** section, the following options have been renamed:

Label	New Label	Type	Parent Field
Item ID	Item Name/Number	Option label	Search By
Display Name	Display Name/Code	Option label	Search By

Updated Labels for Rule-Related Fields and Options

The following fields and options in the **Rule** or **Rules** sections or subtabs have been renamed:

Label	New Label	Type	Parent Field
Show Rule	Rule	Field label	—
Reference Codes for Rule	Reference Codes	Field label	—

Label	New Label	Type	Parent Field
Operator Codes for Rule	Operators	Field label	—
() - Parenthetical Expression	() – Parentheses	Option label	Operators
[] - Interrogate Static Answer	[] – Square Brackets	Option label	Operators
X() - NOT Clause	X() – Negative Condition	Option label	Operators
%% - Question NOT Answered	%% – Unanswered Question	Option label	Operators
/ - Common to all (true)	*/* – Always True	Option label	Operators

Updated Labels for Translation Records and Fields

Translation records have been renamed as follows:

Record Label	New Record Label
Question-Trans	Question Translation
Answer-Trans	Answer Translation
Validation-Trans	Validation Translation
Pricing-Trans	Pricing Translation
Tab-Trans	Tab Translation
Group-Trans	Group Translation

The **Parent** field has been renamed using the parent record label. For example, if the parent record is a question, the new label is **Question**.

The **Label Text** field has been renamed to **Label Translation**.

Updated Labels on the Product User Interface

On the user interface top bar, the following field has been renamed:

Field Label	New Field Label
Unit	Subtotal

On the **Audit** menu, the following menu options, columns, and sections have been renamed:

Label	New Label	Type	Parent Menu Option
Extended Price	Subtotal	Column label	Pricing
Item ID (SKU)	Item Name/Number	Column label	Materials
Display name	Display Name/Code	Column label	Materials
UOM	Sale Unit	Column label	Materials

Label	New Label	Type	Parent Menu Option
Pur price	Purchase Price	Column label	Materials
Cost	Average Cost	Column label	Materials
Ext	Total Cost	Column label	Materials
Items	Additional Items	Menu option label	—
Item ID (SKU)	Item Name/Number	Column label	Additional Items
Display name	Display Name/Code	Column label	Additional Items
Cost	Average Cost	Column label	Additional Items
Ext	Total Cost	Column label	Additional Items
Type	Item Type	Column label	Item Creation
Item ID	Item Name/Number	Column label	Item Creation
Sequence	Sequence Number	Column label	Item Creation
CPI	Images	Menu option label	—
Item ID (SKU)	ID	Column label	Images
Sequence	Sequence Number	Column label	Images
Routings	Routing Steps	Menu option label	—
Oper seq	Operation Sequence	Column label	Routing Steps
Work center	Work Center	Column label	Routing Steps
Runtime	Run Rate	Column label	Routing Steps
Body Mapping Fields	Mapped Body Fields	Section label	Data Mapping
Sequence	Sequence Number	Column label	Data Mapping
Component Mapping Fields	Mapped Line Fields	Section label	Data Mapping
Column Id	Field ID	Column label	Data Mapping

Updated Labels on External Records

NetSuite CPQ Configurator creates fields on the following records outside the SuiteApp:

- Employee record
- Item record

On the employee record under the **Access** subtab, the following fields and options have been renamed:

Label	New Label	Type	Parent Field
NetSuite CPQ Config user	Allow CPQ Configurator use	Field label	—
Extended Role Privileges	CPQ Configurator Permissions	Field label	—
General	Basic	Option label	CPQ Configurator Permissions

On the item record under the **Custom** subtab, the following fields have been renamed:

Field Label	New Field Label
CPQC-Product Configurator	CPQ Configurator Product
CPQC-Product Code	CPQ Configurator Product Code

NetSuite CPQ Configurator Setup

After installing NetSuite CPQ Configurator, define basic settings on the options record at CPQ > Configurator > Settings. You can also configure additional settings to customize your experience with this SuiteApp.

To define basic settings, complete the tasks in the following help topics:

- [Selecting Eligible Transactions](#)
- [Sending Data to the Transaction Page](#)
- [Interacting with Configuration Line Items on Transactions](#)
- [Maintaining Enabled Specific Custom Transaction Body and Line Fields](#)

To customize your experience with NetSuite CPQ Configurator, you can complete the additional tasks in the following help topics:

- [Defining Product Sorting](#)
- [Defining How to Display the Product User Interface](#)
- [Deleting Unused Configuration Records](#)
- [Deleting Temporary Files](#)

When you're finished, save the options record. If you use the Customer Center, also see [Configuring Items from the Customer Center](#).

Selecting Eligible Transactions

Users can configure complex items by clicking the **Configure** button on eligible transactions. After installing NetSuite CPQ Configurator, choose the transactions that can include the **Configure** button. Go to the options record at CPQ > Configurator > Settings. Then, select the required transactions in the **Transaction Types** field.

The following transactions are available:

- sales orders
- invoices
- estimates/quotes
- opportunities
- credit memos
- purchase orders
- return authorizations

- purchase requisitions

Sending Data to the Transaction Page

When submitting the configuration, you can send data either to the transaction page in the browser or the transaction record in the database. Sending data to the transaction page provides more flexibility in applying changes because the transaction isn't automatically saved after submitting the configuration.

To send data to the transaction page, go to the options record at CPQ > Configurator > Settings. Then, check the **Submit to transaction page** box.

If you send data to the transaction page, you should also set an additional option. This option determines how configured items on the transactions are edited:

- Items can be updated.
- Items can be removed and added as new ones.

However, the default option of removing items is no longer used. On the **Submit** subtab of the options record at CPQ > Configurator > Settings, select **Update Items** in the **Item Submit Type** field. This option also maintains the original order of the items on the transaction.

Interacting with Configuration Line Items on Transactions

During the setup at CPQ > Configurator > Settings, you can decide how users interact with line items that belong to a configuration. If the configured item includes additional items (formerly called breakout items), you should only allow users to remove the entire configuration. Allowing users to edit line item values may affect the configuration requirements. If the configuration has strict requirements, you should prevent editing line item values.

When setting up NetSuite CPQ Configurator, choose your preferred option in the **Edit & Remove Options for CPQ Line Items** field:

- **Remove all and prevent editing** – Users can't remove specific configuration line items or edit line item values. They can only remove the entire configuration (main and additional items).
- **Remove all and allow editing** – Users can edit configuration line item values, but they can't delete specific configuration line items.
- **Remove single item and allow editing** – Users can edit line item values and remove specific line items without removing the entire configuration. The removed item can be a main or additional item. This is the default option.

Maintaining Enabled Specific Custom Transaction Body and Line Fields

NetSuite CPQ Configurator automatically creates and enables custom transaction body and line fields to work with eligible transactions. Make sure you keep these fields enabled to let users configure items from transactions. The following table lists the custom transaction body fields.

Body Field Name	Body Field ID
NetSuite CPQC iFrame	custbody_cpqc_iframe
Sole Description	custbody_cpqc_so_lineitem_description

Body Field Name	Body Field ID
Configurations	custbody_cpqc_configs
CPQC Message	custbody_cpqc_message
SO Changed Fields Data	custbody_cpqc_so_changed_fields_data
Sole SO-WO Converted	custbody_cpqc_trns_sowo_converted

The following table includes custom transaction line fields.

Line Field Name	Line Field ID
Configurator Data	custcol_cpqc_item_col_config_data
Inv Detail Data	custcol_cpqc_invdet
Configurator	custcol_cpqc_item_col_config
Configuration	custcol_cpqc_configuration
Config Status	custcol_cpqc_item_cfg_status
CPQC Message	custcol_cpqc_message
CPQC Line Number	custcol_cpqc_line_num

For more information about custom fields in NetSuite, see the help topic [Custom Fields](#).

Defining Product Sorting

Users view the available products on the Configurable Items page. They access this page by clicking the **Configure** button on eligible transactions. Users with an administrator role can also access a preview version of this page at CPQ > Configurator > Standalone.

You can sort the products on the Configurable Items page by name or sequence number. The default option is sorting products by name. To define the product sorting, go to the options record at CPQ > Configurator > Settings. Then, select your preferred option in the **Sort Products By** field. This sorting option doesn't affect the internal product list at CPQ > Configurator > Product Maintenance.

Defining How to Display the Product User Interface

The Configurable Items page and the product user interface are displayed to users in a new browser tab. Using a new browser tab, users can focus on configuring the complex item and switch back to the transaction page if needed.

Alternatively, you can display the configuration pages in a popup window or embedded in the transaction. To define how to display the configuration pages, go to the options record at CPQ > Configurator > Settings. On the **Appearance** subtab, select your preferred option in the **Configurator Start Type** field.

If you want to use a popup window, it can cover the entire transaction page or only 90% of it. Select an option in the **Popup Window Size** field.

If you embed the configuration pages in the transaction, the transaction page becomes dimmed and users cannot switch between the transaction and configuration page. Use this option if users need to configure items and view transaction details at the same time.

Deleting Unused Configuration Records

After submitting a configuration, you may delete the configured line item or the entire transaction. If you do this, the configuration record remains stored in the database without being associated with any transaction. Unused configuration records occupy storage space.

To free up space, you can automatically remove unused configuration records depending on their creation date. To delete unused configurations, go to the options record at CPQ > Configurator > Settings. Then, select an option in the **Delete Unused Configurations Older Than** field. You can delete configurations older than 3, 6, or 12 months.

 **Note:** After the installation, this option is inactive by default.

Deleting Temporary Files

When actions run, they create temporary files. If you're using actions, you should delete those temporary files after a certain number of days to free up storage space.

To delete temporary files, go to the options record at CPQ > Configurator > Settings. On the **Delete Temporary Files** subtab, click **Add Directory**. Then, select the directory that contains the temporary files you want to delete. Lastly, enter the number of days in the **Number of Days** column. Add as many directories as required.

Configuring Items from the Customer Center

If you use the Customer Center, ensure that users will be able to configure items from transactions when logged in to the Customer Center. To achieve this, assign all the required roles to the relevant NetSuite CPQ Configurator script deployments.

The following table sums up the required roles for each script deployment with their purpose.

 **Note:** If you use custom Customer Center roles, make sure you select the correct one. You can verify the role on the customer record by going to **System Information > Access** and review the **Role** field.

Script Name	Script Deployment ID	Script Type	Role	Description
CPQC-ST-Client	customdeploy_cpqc_st_client	Client	Customer Center	Configure items from transactions when users are logged in to the Customer Center.
CPQC-CL-SalesOrder	 Note: Add the role to the transaction deployments used in the Customer Center. - customdeploy_cpqc_cl_creditmemo - customdeploy_cpqc_cl_rtnauth - customdeploy_cpqc_cl_quote	Client	Customer Center	Configure items from transactions when users are logged in to the Customer Center.

Script Name	Script Deployment ID	Script Type	Role	Description
	- customdeploy_cpqc_cl_invoice - customdeploy_cpqc_cl_sales_order			
CPQC-UE-SalesOrder	 Note: Add the role to the transaction deployments used in the Customer Center. - customdeploy_cpqc_ue_creditmemo - customdeploy_cpqc_ue_rtnauth - customdeploy_cpqc_ue_quote - customdeploy_cpqc_ue_invoice - customdeploy_cpqc_ue_sales_order	User Event	Customer Center	- Configure items from transactions when users are logged in to the Customer Center. - Add newly created items to transactions when there are fewer than 300 items in the Customer Center.

To add the required roles to each script deployment:

1. Go to Customizations > Scripting > Script Deployments.
2. In the **Type** field in the Filters section, select the script deployment type.
3. Find and edit the required script deployment.
4. Go to the **Audience** subtab.
5. In the **External Roles** field, select the required roles.
6. Click **Save**.

NetSuite CPQ Configurator Products

NetSuite CPQ Configurator uses products to represent configurable items. Each product contains the business logic that defines how users can configure the item. Products also provide a user interface that guides users through the configuration process based on the business logic constraints. For example, a furniture company may offer a desk as a configurable item. This desk may be available in various shapes, sizes, colors, and so on. Customers can combine various options and features based on their preferences, but only within the combinations that are allowed.

Users with an administrator or a CPQ Product Builder role can create and maintain products. For more information about NetSuite CPQ custom roles, see the help topic [Assigning NetSuite CPQ Custom Roles](#).

Before making products available to users, you can preview the user interface during the setup to make sure everything works as expected. For more information, see [Previewing Products](#).

Each product is made up of several building blocks, such as questions, answers, and images. For more information, see [Working with Product Building Blocks](#).

To add configurable items to transactions, you must assign an item to the product during the setup. This base item (formerly called the base generic product) is associated with the configuration and added to the

transaction. If you only want to include additional items in the transaction, you don't need a base item. For more information about the base item, see [Assigning Base Items to Products](#).

You must also provide information about the product to open when users edit a configured item. For more information, see [Defining a Product to Edit Configured Items](#).

When the base item is added as the main configured item to transactions, you can set transaction line fields associated with it. To set transaction line fields for the main configured item, go to the product record and select mapping records in the **Line Field Mapping Records** field. For more information about mapping records, see [Setting Transaction Body and Line Fields Based on Configuration Data](#).

Products can be organized into categories, and you can export and import products as well. For more information, see [Creating Product Categories](#), [Exporting Products](#) and [Importing Products](#).

If you no longer need a product, you can make it inactive or delete it. To let users edit configured items associated with inactive or deleted products, replace these products with active products. For more information, see [Inactivating Products](#), [Deleting Products](#), and [Replacing Deleted or Inactive Products](#).

To create a product:

1. Go to CPQ > Configurator > Product Maintenance.
2. Click **New CPQC Product**.
3. Enter a descriptive name for the product.
Users view this name on the Configurable Items page and on the product user interface.
4. In the **Code** field, enter a unique identifier for this product.
The code must contain only uppercase letters, numbers, and underscore characters. You can use a shortened version of the name as the code.
5. (Optional) In the **Description** field, enter a brief explanation of this product.
The description isn't displayed to users.
6. In the **Sequence Number** field, enter a number to sort the product on the Configurable Items page.
You can enter a negative or positive number, for example, -2 or 5. Products are sorted from lowest to highest sequence number.
To display the products by sequence number, also select the corresponding option in the **Sort Products By** field at CPQ > Configurator > Settings.
7. To hide the product from the Configurable Items page until it's ready, check the **Inactive** box.
Clear this box to include the product on the Configurable Items page when you've completed the setup.
8. To preview the product faster during the setup, check the **Deactivate cache** box. By checking this box, you avoid the cache being frequently deleted and recreated every time you open the product interface.
Clear this box when the product is ready, and use the cache to load it faster.
9. (Optional) In the **Product Category** field, select a product category. For more information, see [Creating Product Categories](#).
10. Assign a base item (formerly called a base generic product) to the product. For more information, see [Assigning Base Items to Products](#).
If you only want to include additional items in the transaction, skip this step.
11. To set transaction line fields for the base item when it's added as the main configured item to transactions, select the mapping records in the **Line Field Mapping Records** field. For more information, see [Setting Transaction Body and Line Fields Based on Configuration Data](#).

12. On the **Appearance** subtab, select an image from the File Cabinet to represent the product in the **Cover Image** field. This image is displayed on the Configurable Items page with the product name.
13. On the **CPQ AI Assistance** subtab, define the AI description for use by the NetSuite CPQ AI Assistant. For more information, see the help topic [Setting Up CPQC Products for NetSuite CPQ AI Assistant](#).
14. Click **Save**.

Previewing Products

When setting up a product user interface, you can verify that everything works in the product preview. The preview lets you access and work with the product user interface without opening it from a transaction every time. To access the product preview, edit the product record and click the **Run Standalone** button.

You can also preview the list of configurable items displayed to users. To access the preview of the configurable item list, go to the Configurable Items page at CPQ > Configurator > Standalone. Users view the Configurable Items page by clicking the **Configure** button on a transaction.

Working with Product Building Blocks

Products are organized into building blocks, such as questions, answers, and images. Many building blocks are rule-based, and they're activated under certain conditions. For more information about rules, see [Activating or Hiding Building Blocks with Rules](#).

Create the user interface with the following building blocks:

- **Questions and answers** – Gather and display information about the configurable item. For more information, see [Gathering Information with Questions and Answers](#).
- **Validation messages** – Provide additional information or let users know about any inconsistencies or issues in the configuration. For more information, see [Displaying Validation Messages on the User Interface](#).
- **Images** – Help users visualize the final look and feel of the configurable item. For more information, see [Displaying Preview Images of the Configurable Item](#).
- **Tabs and groups** – Organize questions to guide users through the configuration process. [Organizing Questions into Tabs and Groups](#).

If you install NetSuite CPQ Manufacturing, you can use the following building blocks to create work orders:

- **Materials** – Make up the configurable item. For more information, see [Defining Materials for the Configurable Item](#).
- **Routing steps** – Represent the tasks required to build the configurable item. [Defining Routing Steps to Build the Configurable Item](#).

Other building blocks:

- **Prices** – Create the final price for the configurable item. For more information, see [Calculating the Final Pricing for the Configurable Item](#).
- **Additional items** (formerly called breakout items) – Complete the main configurable item. For more information, see [Including Additional Items Related to the Configurable Item](#).
- **Tables** – Provide structured data for answers. For more information, see [Storing Answer Options in Tables](#).

- **Item creation records** – Contain information to create items required by the configuration. For more information, see [Creating Items Required by the Configuration](#).
- **Creation records** – Contain information to create records required by the configuration. For more information, see [Creating Records Required by the Configuration](#).
- **Mapping records** – Contain information to set transaction body and line fields based on configuration data. For more information, see [Setting Transaction Body and Line Fields Based on Configuration Data](#).
- **Actions** – Let you personalize products with scripts that can automate a repetitive tasks or perform advanced tasks. For more information, see [Customizing Products with Scripts Using Action Records and Functions](#).

After creating a building block instance, you can copy or delete it. By copying a building block instance, you can reuse information and save you manual work.

To copy a building block:

1. On the building block subtab, click the name of the building block you want to copy.
2. Select **Make a Copy** in the **Actions** dropdown menu.
3. Edit the building block name, code, and sequence number if available.
4. On the **Rules** or **Rule** subtab, update the rules to match the requirements for this new building block.
5. Make changes to other fields and subtabs, and add new data as needed.
6. Click **Save**.

If you no longer need a building block instance, you can delete it permanently. Edit the building block and select **Delete** under the **Actions** dropdown menu. Then, click **OK** in the popup window to confirm that you want to delete the building block.

Assigning Base Items to Products

A base item (formerly called a base generic product) is a NetSuite item used as a placeholder. After users submit the configuration, the base item is added to the transaction as the main configured item, regardless of the specific choices made in the configuration.

To create the base item, you only need to enter the basic information needed to save the item record. The configuration provides the rest of the information.

 **Note:** The base item must be an active item. An item is active when the **Inactive** box isn't checked. You can find this option under the **System Information** subtab on the item record.

To assign a base item to a product, go to CPQ > Configurator > Product Maintenance. Edit the product that requires a base item. To choose the base item, two options are available:

- In the **Base Item** field, select an item from NetSuite.
- In the **Base Item (Item Creation Record)** field, select an item creation record to create the item when users submit the configuration. For more information, see [Assigning Item Creation Records to Materials, Additional Items, or Products](#).

When you're finished, save the product record.

You can use the following item types as base items:

- inventory items
- assemblies
- kit/packages

- non-inventory items
- subscription plans

To create work orders for configured items, select an assembly as a base item. For more information, see [Creating Work Orders for Configured Items](#).

On the **Custom** or **CPQ** subtab of the item used as a base item, define the product used to edit configured items. For more information, see [Defining a Product to Edit Configured Items](#).

Defining a Product to Edit Configured Items

After adding a configured item to a transaction, users may want to make changes to it. To let users edit configured items from transactions, you need to define the product used for this purpose. You can either define a default product to edit all configured items, or assign a specific product to each base item.

 **Note:** If you set both a default and a specific product, users will edit the associated configured item with the specific product.

If you want all configured items to be edited in the same way, define a default product. To define a default product, go to CPQ > Configurator > Settings and select a product in the **Default Product** field. If you use a default product, make sure you also go to the **Custom** or **CPQ** subtab of the item used as the base item. Then, check the **Is configurable** box. By setting this option, you can edit the configured items associated with the base item using the default product. For more information about base items, see [Assigning Base Items to Products](#).

Alternatively, if you want to edit the configured item using a specific product, assign a product to each item used as a base item. To assign a product to the base item, go to the **Custom** or **CPQ** subtab of the item record. Then, select a specific product in the **CPQ Configurator Product** field.

Creating Product Categories

You can group, classify, and organize products based on their features and functionality using product categories. For example, a furniture company may have a product category for office desks and another for home desks.

With product categories, users can view a filtered version of the Configurable Items page from eligible transactions. The page will only show the products in that category. To view the configurable items for a specific product category, users can click the arrow next to the **Configure** button on a transaction and select their preferred category.

If the product category includes only one product, users directly access the product interface without going through the Configurable Items page. Using product categories, users can also filter items on the Configurable Items page and quickly find the one that best suits their needs.

To create product categories, go to CPQ > Configurator > Product Categories and click **New CPQC Product Category**. Provide a descriptive name for the product category and save it. After creating the category, use the **Product Category** field on the product record to assign a product to a category.

You can also use product categories to create buttons on eligible transactions. To create a button for the product category, check the **Display on transactions** box on the product category record. When users click a product category button, only the products assigned to that category are displayed on the Configurable Items page.

To modify a product category, click **Edit** on the product category list. If you want to copy the product category, click Make a copy in the **Actions** menu. If you no longer need a product category and you want to delete it, click **Delete** in the **Actions** menu.

Exporting Products

After creating products, you may want to export them for various reasons. You can combine export and import options to move products from your sandbox to your production account after testing. Through exporting, you can create duplicate products in your current account and use them as a base for new products. Exporting is also useful to create backup copies and restore products if you run into any issues.

You can export products at CPQ > Configurator > Settings. To export a product, select one in the **Export Product** field and click the **Export** button. The product is saved as a ZIP file on your computer.

In the exported file, you can include additional resources, such as images, CSV tables, and CSS styles. If these resources aren't in the target account, make sure you check the **Export product resources** box before exporting the product. However, if you export the product for testing purposes and you don't need these resources, leave the box unchecked.

Importing Products

Note: Keep the options page open until all product imports are either pending or in progress. Closing the page will cancel any product imports in the **Initializing** status.

After exporting a product, you can import it back into the same account or a different one at CPQ > Configurator > Settings. You can select and import multiple products at the same time and the imports will run in parallel. When selecting products, you should group together those that will be imported using the same options.

You can monitor the import progress in the product import summary. Import information stays available for 7 days. To remove it earlier, click the **Clear** button next to the corresponding import line. If the product import summary exceeds 50 lines, clear finished imports before starting new ones.

When importing a product, choose how to handle references to records. For more information, see [Importing References to Records](#).

You can also choose to import a new product version into an existing product, which is called target product. For more information, see [Importing a New Version into an Existing Product](#).

Note: To set up, prototype, or update large numbers of products, use CSV Import to create or modify multiple CPQ product configurations. For details, see the help topic [CPQ Product Import](#).

To import a product:

1. In NetSuite, go to CPQ > Configurator > Settings.
2. In the **Import Product** field, select one or multiple ZIP files with the products you want to import.
3. In the **Import Options for Record References** field, choose how to handle references to records when importing your product. For more information, see [Importing References to Records](#).
4. To automatically create placeholder items for any referenced item records missing in the target account, check the **Auto-create missing items** box.

Note: If you want missing items to be created automatically, make sure you select the **Import by name** option in the **Import Options for Record References** field.

5. Click **Import**.

Importing References to Records

Imported products may include references to item records—also called links. For example, you create a reference to an item record when selecting a NetSuite item as a material. These referenced item records need to be retrieved in the target account. Before importing a product, choose how to handle item record references in the **Import Options for Record References** field:

- **Do not import** – Leaves all fields with a record reference blank.
Choose this option when you're importing products into a sandbox account for testing purposes and you don't need the referenced items.
- **Import by ID** – Assigns record references based on the internal ID of the item. This is the default option.
Choose this option when importing products within the same account to optimize the import process.
- **Import by name** – Finds the item internal ID by looking for the item name and places it in the link field.
Because items with the same name can have different internal IDs, choose this option when importing products across different accounts.
Some items may be referenced by imported products but don't exist in the target account. When importing record references by name, you can automatically create placeholder items to replace missing items that are linked to materials or additional items (formerly called breakout items). Placeholder items use the same ID and name or number as the original item, and they have a fixed price of 10 USD. To enable this option when importing products for testing purposes, check the **Auto-create missing item** box.

Importing a New Version into an Existing Product

You can update a product by importing a new product version into an existing one—called target product. The updated product keeps the original internal ID, but the rest of the data is replaced with information from the new product.

 **Note:** When working with a target product, start each import process separately.

To use this option, select the product you want to import in the **Import Product** field. Then, select the product where you want to import data in the **Target Product** field and click the **Import** button.

When working with a target product, you can continue to use the **Import Options for Record References** and **Auto-create missing items** fields.

When importing a new version into an existing product, options and features of the configurable item may change. If favorite configurations reference options or features that no longer exist or have changed in the new version, they may no longer work with the new imported version. If this is the case, check the **Unlink favorites from target product** box before importing the product. By enabling this option, favorites configurations will no longer be associated with the target product, but they'll remain in your account.

Deleting Products

If you no longer need a product, you can delete it from your account. Deleted products don't appear on the Configurable Items page and can't be configured on new transactions. Previous transactions maintain a reference to the deleted product, but you won't be able to view or open the configuration for the product.

Before deleting a product, take the following important points into account:

- Deleted products are permanently deleted from your NetSuite account and can't be restored. All subrecords (such as questions, answers, or tabs) are also deleted.
- Check for unfinished transactions that have a saved configuration for the product. After you delete the product, you won't be able to edit the configuration of the product from the transaction unless you specify a replacement product.
- If you think you might need to use the product again at a future date, you should export a copy of the product before deleting it. See [Exporting Products](#) for more information.

 **Note:** Instead of deleting a product, you might want to make it inactive—see [Inactivating Products](#).

After you delete a product, you can specify a replacement product—see [Replacing Deleted or Inactive Products](#).

To delete a product:

- Go to CPQ > Configurator > Product Maintenance and select **Edit** next to a product.
- Click **Delete**.

 **Warning:** In the next step, you'll confirm the deletion. You won't be able to undo this action. The product record and all its subrecords will be deleted from your NetSuite account.

- Type **DELETE** and click **Confirm** to confirm the operation.

Inactivating Products

If you want to make a product unavailable temporarily or you no longer sell a product, you can make it inactive. Similar to deleted products, inactive products don't appear on the Configurable Items page and can't be configured on new transactions. If there are unfinished transactions with a configuration for an inactive product, a user can still view or update the configuration from a transaction line. You can override the default behavior, however, to prevent users editing the configuration of an inactive product.

To inactivate a product:

- Go to CPQ > Configurator > Product Maintenance and click **Edit** next to a product.
- Check the **Inactive** box.
- If you want to prevent users editing the configuration from a transaction line, check the **Disallow edits on existing configurations** box.
- Click **Save**.

 **Note:** You can make a product active again at any time by clearing the **Inactive** box.

After you inactivate a product, you can specify a replacement product—see [Replacing Deleted or Inactive Products](#). If you want to permanently remove a product from your account, you can delete it instead of making it inactive—see [Deleting Products](#).

Replacing Deleted or Inactive Products

When you delete or inactivate a product, you can choose a different product to replace the inactive or deleted product. Specifying a replacement product can make it easier for users to view or edit a

configuration from a transaction line on an existing transaction. Provided that the replacement product has the same configuration properties as the original product, users can continue to configure a product even though the original product is no longer available.

Note: Users can only edit configurations on transactions that are still in progress and haven't been processed.

Replacement products should have the same or similar configuration properties as the original product. If there are differences in properties between the original product and the replacement product, some information will be lost. For example, if the replacement product is missing some configuration properties from the original product, those properties won't appear on the product interface. On the contrary, if the replacement product contains additional properties, they'll appear on the product interface, but they'll typically be blank or unpopulated.

You can use the same replacement product for multiple inactive or deleted products, but an inactive or deleted product can have one replacement product only.

When a user edits a configured item from a transaction line and then saves the configuration, the transaction is updated with the information entered on the configuration of the replacement product.

Example 1. Example

A product configuration for a bicycle includes three questions: frame size, frame color, and wheel type. An administrator makes the product inactive and specifies a replacement product that includes the questions: frame size, frame color, and handlebars type. Because the question for wheel type isn't in the replacement product, the question isn't displayed on the replacement product interface. The question for handlebars type is displayed on the configuration page, but its value isn't populated because it did not exist in the original product.

To specify a replacement product:

1. Go to CPQ > Configurator > Settings and select the **Product Replacements** subtab.
The Status column indicates whether the original product was made inactive or was deleted.
2. In the row of the product you want to replace, choose a replacement product in the **Replace With** column. You can only choose products that are active.
3. Click **OK** and then click **Save**.

Activating or Hiding Building Blocks with Rules

Your business relies on a well-defined logic to run successfully. For example, certain item configurations may be compatible only with specific materials. NetSuite CPQ Configurator helps you capture and enforce this logic using rules assigned to its building blocks, such as questions, answers, images. These rules represent the restrictions that limit the possible combinations of features and options users can choose for your configurable items. For example, you may prevent users from selecting a water-based finish for a metal desk frame because that could cause rust.

Rules are conditional statements based on combinations of questions and answers. Every time a user makes a selection or enters information in the product interface, the system verifies the conditions in the rules. When verifying the rules, the system compares the answers specified in the rules with the choices users have made on the product interface. If there's a match, the system activates the building blocks and performs the assigned task, such as displaying an image or including an additional item.

You can set up rules in the rule field of most building blocks. Some building blocks can have more than one type of rule. To write a rule, use question and answer codes and insert a forward slash to separate them. Each code is a unique, shortened version of a building block's name. When writing rules, always

enter the question code first. For example, you may use A/1 as the rule of an image. In this rule, A is the question code, and 1 is the answer code. This code combination is the condition that the system verifies. For example, if users select answer 1 for question A, then the image is displayed on the product interface. If a question type doesn't have answers, write the rule by entering the question code only.

You can also use operators to combine multiple conditions in a rule. For example, you can assign the condition A/1&B/2 to the image. The ampersand represents the AND operator, and the system will only display the image if both conditions are true. If users select answer 1 for question A and answer 2 for question B, then the image is displayed on the product interface.

When writing rules, you can choose codes and operators from fields with predefined lists that include all available combinations and options. These fields are named **Reference Codes** and **Operators**. For more information about operators, see [Using Rule Operators](#).

Some building blocks also have start and end dates. These dates affect how the system evaluates rules. If you define a start and end date or a start date only, the building block will only perform its assigned task if both the date range and the rule condition are met.

You can include additional conditions in your rules by using predefined answers, which are fixed questions and answers combinations that contain information from your account. For example, if you want to display a tab only for administrators, you can use this predefined answer in the tab rule `SOLE/ROLEID["administrator"]`. For more information, see [Including Additional Conditions in Rules and Actions with Predefined Answers](#).

NetSuite CPQ Configurator also supports additional rules named rule categories. When you assign a rule category to a building block, both the building block's rule and the category rule must be true for the building block to perform its task. For more information, see [Working with Rule Categories](#).

Example

Suppose your business offers a desk as a configurable item. This desk is available in various shapes, such as rectangular, U-shape, and L-shape. You may have a question like "Choose the desk shape" with the code `DESK_SHAPE`. One of the possible answers may be "L-shape" with L as a code. If users choose an L-shaped desk on the product interface, the system will display the image that contains `DESK_SHAPE/L` in the rule field.

Let's say the desk surface can also have assorted finishes, such as oak, cherry, or walnut. If users choose an L-shaped desk with an oak finish, the system displays the image with the rule `DESK_SHAPE/L&TOP_FINISH/OAK`. `TOP_FINISH` and `OAK` are the question and answer codes, respectively. The AND operator (ampersand character) indicates that users must choose both the L-shape and the oak finish to view that image.

Working with Rules

Depending on the building block to which a rule is assigned, NetSuite CPQ Configurator can perform one of the following tasks:

Note: To perform the task, the answers selected by users on the product interface must match the condition in the building block rule.

- Display an image on the product interface.
- Display a validation message on the product interface.
- Include an additional item (formerly called a breakout item) in the configuration and, later, in the transaction.
- Add a price to the total price of the configurable item.
- Add a material to the configuration and, later, to the work order.

 **Note:** Install NetSuite CPQ Manufacturing to create work orders. For more information, see the help topic [Installing NetSuite CPQ SuiteApps](#).

- Add a routing step to the configuration and, later, to the work order.

 **Note:** Install NetSuite CPQ Manufacturing to create work orders. For more information, see the help topic [Installing NetSuite CPQ SuiteApps](#).

- Run an action.
- Enable a rule category.
- Create an item required by the configuration.
- Create records required by the configuration.
- Set transaction body or line fields based on configuration data.

Special rule types are also available for questions, answers, tabs, and groups:

- **Hide rule** – If the rule's condition is met, the question, answer, tab, or group is hidden from the product interface. This rule prevents users from viewing unavailable features and options for their current selection.
- **Read-only rule** – If rule's condition is met, the question or answer remains visible but is dimmed on the product interface. This rule informs users that other options may become available if they make a different selection.
- **Incompatible rule** – If the rule's condition is met, the answer is dimmed and marked with a small red cross. If the answer was previously selected, its label becomes red. This rule notifies users that the selected combination of options or features doesn't meet the configuration requirements.

Working with Rule Categories

NetSuite CPQ Configurator has additional rules named rule categories. Rule categories help you better organize and maintain your business logic. If you frequently use a rule, you can create a category with that rule and apply it to specific building block instances. Rule categories can also be used to group building block instances that will be activated or inactivated based on the rule of the category. You can apply rule categories to the following building blocks: images, pricing, additional items (formerly called breakout items), materials, and actions. You can also share rule categories across all available building blocks.

When you assign a rule category to a building block instance, the system automatically combines the rule of the category with the building block rule—as if you had added the rule of the category directly to the **Rule** field of the building block using the AND operator (ampersand character). For example, if a rule applies to multiple images, you can create a category and assign it to each image record. Those image records will inherit the rule of the category.

A building block with a category assigned will only perform its task if its rule and the rule of the category match the answers selected by users on the interface.

To create a rule category:

1. Go to CPQ > Configurator > Product Maintenance.
2. Edit the product you want to modify.
3. On the **Rule Categories** subtab, click **New CPQC Rule Category**.
4. Enter a descriptive name for the rule category.
5. Select the rule category type.

To apply the rule category to all eligible building blocks, select **Common**. Otherwise, select a specific building block from the list.

 **Note:** If you leave this field blank, the rule category can be applied to all eligible building blocks as if you'd selected **Common**.

6. Provide a sequence number.
7. In the **Rule** field, provide the condition that determines whether this rule category is active.
Enter question and answer codes separated by a forward slash, for example, A/1. Use operators to combine multiple conditions, for example, A/1&B/2. For more information, see [Activating or Hiding Building Blocks with Rules](#).
8. Click **Save**.

Using Rule Operators

The following table lists the operators that you can use in rules to combine multiple conditions. In the examples, A represents the question code, and 1 is the answer code.

Operator	Description	Example	Meaning
/	Forward slash – Separates the question code from the answer code.	A/1	The answer to question A is 1.
&	Hard AND – Both conditions must be true for the entire rule to be true.	A/1&B/2	The answer to question A is 1 and the answer to question B is 2.
+	Soft AND – Works like the hard AND, but you use it with two or more answers to the same question.	A/1+2	The answers to question A are 1 and 2.
#	Hard OR – At least one condition must be true for the entire rule to be true.	A/1#B/2	The answer to question A is 1 or the answer to question B is 2.
,	Soft OR – Works like the hard OR, but you use it with two or more answers to the same question.	A/1,2	The answer to question A is 1 or 2.
()	Parentheses – Group conditions that the system evaluates together.	(A/1&B/2)#C/3	Either the answer to question A is 1 and the answer to question B is 2, or the answer to question C is 3.
[]	Square brackets – Used to verify the value of an answer when using the following operators: ■ equals to ■ greater than ■ less than ■ begins with ■ contains ■ ends with The tilde symbol (~) identifies a range of numbers.	A/1["oak"] A/1[^="o"]. A/1["*="a"]. A/1["\$="k"] A/1[=10] A/1[>10] A/1[<10] A/1[>=10] A/1[<=10]	The value entered for question A, answer 1 is oak. The value entered for question A, answer 1 begins with o. The value entered for question A, answer 1 contains a. The value entered for question A, answer 1 ends with k. The value entered for question A, answer 1 is 10.

Operator	Description	Example	Meaning
	To evaluate the answer against a sequence of characters (letters and numbers), enclose the value in quotation marks. You can verify the value for: ■ All types of input box answers. ■ QTable date and time fields and popup lists (formerly called picklist dropboxes). You can combine square brackets with resolve expressions, except when using the tilde symbol (~). For example, A/1["{Q/A}"]. Always enclose the resolve expressions in double quotation marks. For more information about resolve expressions, see Obtaining Answer Data with Resolve Expressions.	A/1[10~20]	The value entered for question A, answer 1 is greater than 10. The value entered for question A, answer 1 is less than 10. The value entered for question A, answer 1 is greater than or equal to 10. The value entered for question A, answer 1 is less than or equal to 10. The value entered for question A, answer 1 is between 10 and 20.
X()	NOT – Used to negate a condition. If the condition is false, the rule is true.	X(A/1)	Answer 1 isn't selected for question A.
*	Wildcard – Represents one or more unknown characters at the beginning, the end, or in the middle of an answer code.	A/1* A/*1 A/*1*	The selected answer code for question A begins with 1. The selected answer code for question A ends with 1. The selected answer code for question A contains 1.
%%	Unanswered question – Identifies a question where users didn't select any answers.	A/%%	No answer is selected for question A.
/	Any question and answer combination – The rule is always true. For example, the image is always displayed on the product interface. If the asterisk replaces only the answer code, the rule is true if the user selects any answer to the question.	*/* A/*	The rule is always true, regardless of the question and answer combination. Question A is answered.
_	Underscore – Used after the answer code to specify a qTable row. The row number follows the underscore. If the @ symbol follows the underscore, the underscore operator identifies the qTable cells in the same row. For example, you can make the content of a cell read-only based on a cell in the same row.	A/B_1 A/B_@	In the qTable A, the answer is the column B cell in row number 1. In the qTable A, the answer is the column B cell that is in the same row.
[x;x=""']	Identifies a sequence of characters, the number of characters in the sequence, and their position.	A/1[3;4="BCDE"]	The code of answer 1 for question A is the four-character value of BCDE, beginning in position 3.

Working with the Product User Interface

Each product is provided with an interface where users can configure complex items based on their preferences. The following image shows a simplified product user interface with its main elements.

You can find the following elements on a product user interface:

1. **Top bar** – Displays the subtotal (the price of a single configured item), the quantity of the main configured item, and the total price (the subtotal multiplied by the quantity).
When users finish configuring the item, they can click one of the **Submit** buttons to add it to a transaction. Depending on which submit button they click, users can close the interface, start configuring another item, or duplicate the configuration.
2. **Audit menu** – Contains information about every building block that has been activated from the user interface and included in the current configuration. This menu is updated every time users select options and features on the user interface.
The Audit menu includes information about prices, materials, additional items, items to be created, records to be created, product images, routing steps, actions, and transaction fields to be set. The **Data** option provides information about every activated rule in the configuration. You can also search the rules by question and answer codes or by the answer value.
3. **Favorites menu** – Lets you save a configuration to use later or define a default configuration that loads when users open a specific product. For more information, see [Creating Favorite And Default Configurations](#).
4. **Tab line** – Displays the tabs that organize questions in different panes. For more information, see [Organizing Questions into Tabs and Groups](#) and [Changing the Appearance of Tabs, Groups, and the Image Container](#).
5. **Question space** – Displays the questions and answers that help you gather information about configurable items and guide the configuration process. For more information, see [Gathering Information with Questions and Answers](#) and [Organizing the Question Space](#).
6. **Information bar** – Displays the configurable item image, the validation messages, and the summary. For more information about changing the information bar appearance, see [Changing the Information Bar Width and Position](#)

7. **Configurable item image** – Shows the configurable item while it's being configured (composite image) or a gallery of images. For more information, see [Displaying Preview Images of the Configurable Item](#).
8. **Validation messages** – Provide information to guide users in the configuration process. This section is only displayed if there's at least one validation message. For more information, see [Displaying Validation Messages on the User Interface](#).
9. **Summary** – Provides detailed information about the price for a single main configured item. The subtotal in the summary is copied to the **Subtotal** field in the top bar. For more information, see [Calculating the Final Pricing for the Configurable Item](#).

For information about changing the user interface look and feel, see [Changing the Product Interface Look and Feel](#).

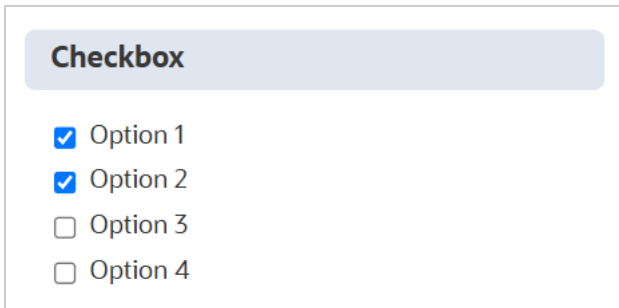
Gathering Information with Questions and Answers

With NetSuite CPQ Configurator, you can gather and display information about the configurable item with questions and answers and guide users through the configuration process. Questions represent the item features, aspects, or characteristics that require user input. For each question, answers show the available options to users and let them express their preferences.

The question types available are:

- **Checkbox** – Lets users select multiple answers from a predefined list by checking the box next to each answer. This question type is useful when users can express multiple preferences from a limited set of answers.

The following image shows a checkbox question:



Checkbox

- ☒ Option 1
- ☒ Option 2
- ☐ Option 3
- ☐ Option 4

For more information, see [Checkbox and Radio Button Questions](#).

- **Radio button** – Lets users select a single answer from a predefined list of mutually exclusive options by choosing the radio button next to each answer. This question type is helpful when only one option or feature is available within a limited answer list.

The following image shows a radio button question:



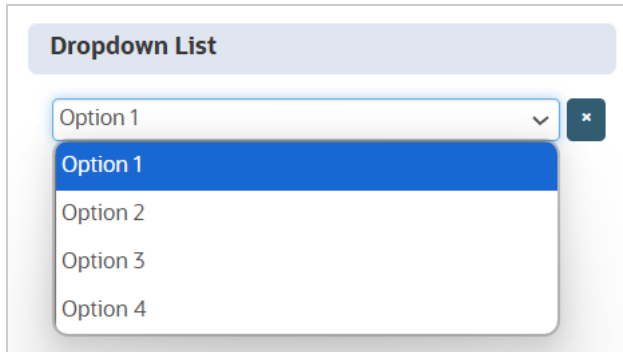
Radio Button

- ☒ Option 1
- ☐ Option 2
- ☐ Option 3
- ☐ Option 4

For more information, see [Checkbox and Radio Button Questions](#).

- **Dropdown list** – Lets users select a single answer from a predefined list, which appears when they click the arrow. When there is a long list of answers, you can choose this question type to save space on the user interface. For more information, see [Dropdown List Questions](#).

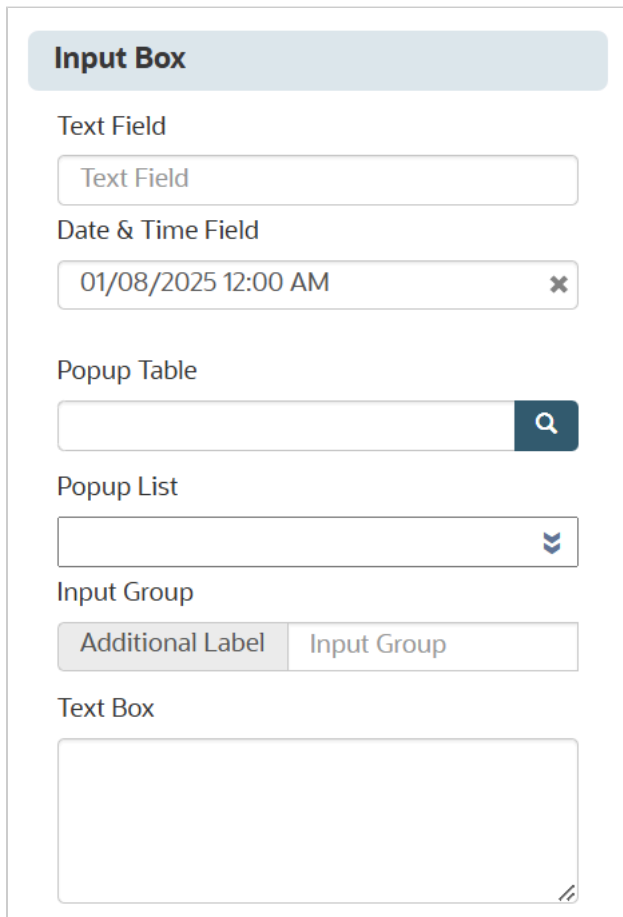
The following image shows a dropdown list question:



The image shows a 'Dropdown List' question interface. It features a light blue header with the text 'Dropdown List'. Below the header is a dropdown menu with a white background and a blue border. The menu is currently open, showing a list of four options: 'Option 1', 'Option 2', 'Option 3', and 'Option 4'. The first option, 'Option 1', is highlighted with a blue background. To the right of the dropdown menu is a small dark blue button with a white 'x' icon.

- **Input box** – Lets you create various types of fields where users can enter text, such as text fields or text boxes, or select options from a popup table or a popup list (formerly called a picklist and a picklist dropdown). This question type is useful to let users freely express their preferences, give feedback, or choose an item feature from an extensive lists of combined options stored in product tables.

The following image shows an input box question with its answer types:



The image shows an 'Input Box' question interface. It features a light blue header with the text 'Input Box'. Below the header are several input fields, each with a label and a corresponding input type:

- Text Field**: A simple text input field with the placeholder text 'Text Field'.
- Date & Time Field**: A date and time input field showing '01/08/2025 12:00 AM' and a dark blue button with a white 'x' icon.
- Popup Table**: A search input field with a dark blue button containing a white magnifying glass icon.
- Popup List**: A dropdown menu with a dark blue button containing a white double arrow icon.
- Input Group**: A group of two input fields, one labeled 'Additional Label' and the other 'Input Group'.
- Text Box**: A large text area input field with a dark blue button containing a white double arrow icon.

For more information, see [Input Box Questions](#).

- **QTable** – Lets you create a question in the form of a table where columns represent the answers. Columns can store varied answer types, such as text fields, checkboxes, and dropdown lists. This question type is useful for gathering information about various aspects of the configurable item at the same time or to add a long list of additional items to transactions.

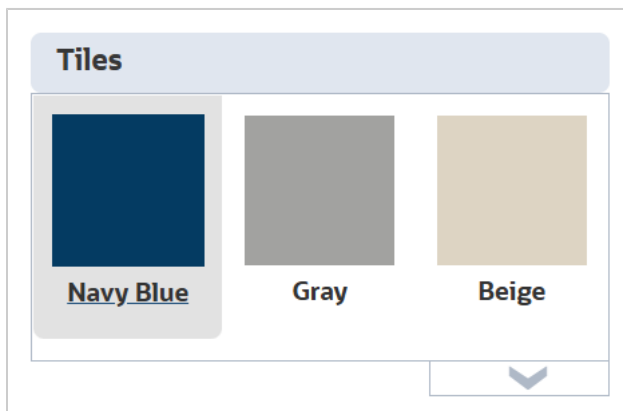
The following image shows a qTable question with columns corresponding to its different answer types:

QTable						
	Text Field	Popup List	Checkbox	Dropdown List	Date & Time Field	Custom HTML
1	ITEM-AB01	Navy Blue	☒	Option 1	13/10/2024 09:21 AM	Link 1
2			☐			Link 1
3			☐			Link 1
4			☐			Link 1
5			☐			Link 1

For more information, see [QTable Questions](#).

- **Tile** – Lets users select one or multiple answers from a collection of labeled images. To select multiple answers, users can hold the Ctrl key. You can use this question type when the item options or features have a relevant visual element. For example, tiles are useful when choosing colors or textures.

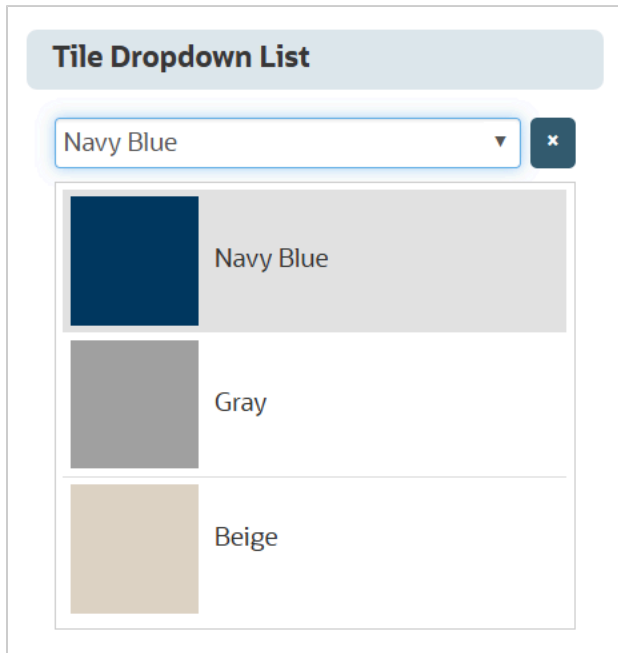
The following image shows a tile question type:



For more information, see [Tile and Tile Dropdown List Questions](#).

- **Tile dropdown list** – Lets users select one or multiple answers from a dropdown list with labeled images. To select multiple answers, users can hold the Ctrl key. When you have a long list of visual elements, this question type is helpful to save space on the user interface.

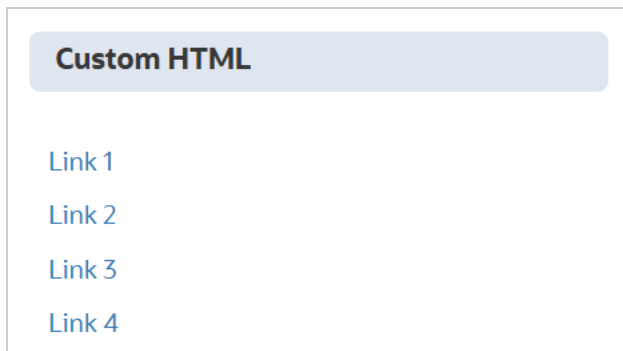
The following image shows a tile dropdown list question:



For more information, see [Tile and Tile Dropdown List Questions](#).

- **Custom HTML** – Lets users view the custom content that you created with HTML code. For example, you can add images or links.

The following image shows an inline HTML question:



For more information, see [Custom HTML Questions](#)

Depending on the question type you create, the answer options vary accordingly. For example, you can add a text field to a checkbox answer or show placeholder text in an input box answer. After saving a question or an answer, they're automatically displayed on the product user interface.

For more information, see the following help topics:

- [Working with Questions](#)
- [Working with Answers](#)
- [Working with Question and Answer Rules](#)
- [Working with Specific Question and Answer Types](#)

To change label and header colors for all questions in a product, see [Changing Label and Header Colors for Questions, Tabs, and Groups](#).

Working with Questions

With NetSuite CPQ Configurator, you can create questions to gather user preferences about the features and options of the configurable item.

Each question has an internal name and a label for the user interface. You can use a shortened version of the name as a code to identify questions in actions and rules. The sequence number determines the order in which the question is displayed on the product user interface. Questions with a lower sequence number are displayed higher on the user interface.

The question space on the user interface is divided into 12 columns. Depending on the question type, you may need to specify how much space the question can occupy on the user interface. You can choose whether to arrange questions vertically or horizontally on the product record. For more information about the question layout, see [Organizing the Question Space](#).

To create a question:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Click **Edit** next to the product you want to modify.
3. On the **Questions** subtab, click **New CPQC Question**.
4. Enter a descriptive name for the question.
This name is for internal reference and is automatically copied to the **Label** field. To change the name users see on the interface, edit the text in the **Label** field.
5. In the **Code** field, enter a unique identifier for this question.
You can use a shortened version of the name as the code to identify the question in rules and actions. The code can only contain uppercase letters, numbers, and underscore characters.
6. (Optional) In the **Description** field, enter a short explanation of the question.
The description isn't displayed on the user interface.
7. Provide a sequence number to set the order in which the question is displayed on the user interface.
You can enter a negative or positive number—for example, -2 or 5.
8. In the **Type** field, select the type of question you want to create, such as checkbox, radio button, or dropdown lists.
9. Depending on the question type, you may need to choose how many answers users can select on the product interface. In the **Answer Selection Type** field, define whether you want users to select one or multiple answers.

 **Note:** The **Default** option is no longer used.

10. Depending on the question type, you may need to select the order in which users see the answers on interface. In the **Sort Answers By** field, select whether you want to sort the answers by name or by sequence number.
11. In the **UI Columns** field, enter a number between 1 and 12 to define how much space the question can occupy on the user interface. The number 6 corresponds to half the question space.
12. In the **Label** field, enter the text users see on interface for this question.
You can format the text by changing the paragraph font and size or making the text bold, italic, or underlined. You can also include HTML code to add images or links. However, script tags aren't supported.
13. (Optional) Select a tab or a tab and group combination where the question is displayed.
14. On the **Rules** subtab, set up the rules to hide or make the question read-only when users select certain features or options. For more information, see [Working with Question and Answer Rules](#).

15. On the **Appearance** subtab, define the question look and feel. For more information, see [Changing the Question Look and Feel](#).
16. On the **CPQ AI Assistance** subtab, define the AI description for use by the NetSuite CPQ AI Assistant. For more information, see the help topic [Prepare Configurable Items for NetSuite CPQ AI Assistant](#).
17. Click **Save**.

After saving a question, you can open it from the **Questions** subtab and edit it or add information. To copy or delete questions, see [Working with Product Building Blocks](#). When you copy a question, answers are not copied with the question. When you delete a question, the answers related to that question are deleted as well.

Changing the Question Look and Feel

With NetSuite CPQ Configurator, you can change the question look and feel on the **Appearance** subtab of the question record. For example, if you only want to use an image to represent the question, you can hide the question label and header on the user interface.

You can add images to most question types to help users better understand features or options. You can select an image from the File Cabinet or enter the URL of the image. The image is displayed between the question header and the answers.

At the question level, you can also define whether to display the answers in one column (vertically) or multiple columns (horizontally). For example, choose a vertical layout for long answers, and a horizontal one for shorter answers.

To change the question look and feel:

1. On the **Questions** subtab, open a question for editing.
2. Click the **Appearance** subtab.
3. To hide the question label, check the **Hide label** box.
4. To hide the question background color, check the **Hide label background color** box.
5. To add an image to the question, two options are available:
 - In the **Image** field, select an image from the File Cabinet. For more information, see the help topic [Uploading Files to the File Cabinet](#).
 - In the **Image URL** field, enter the URL of the image.

If you select an image and also provide a URL, only the image from the URL is displayed on the user interface.

6. In the **Answer Layout** field, select an option to define how you want to display the answers:
 - **Vertical (single column)** – Displays the answers vertically.
 - **Horizontal (up to 12 columns)** – Displays the answers horizontally.

If you want to display the answers horizontally, enter the number of columns to arrange the answers in the **Answers per Row** field.



Note: Take into account the following:

- Depending on how many columns you assign to the question in the **UI Columns** field, the number of answers per row may be automatically adjusted to fit the available space.
- If you leave the **Answers per Row** field blank, three answers per row will be used.

7. Click **Save**.

Working with Answers

With NetSuite CPQ Configurator, you can create answers to introduce users to the item features or options and let them express their preferences. The options for creating answers change according to the question type. For example, you can add a text field to a checkbox answer or show placeholder text in an input box answer.

Each answer has an internal name and a label for the user interface. Optionally, you can add a tooltip—a short message that helps users better understand the answer. This message is displayed when users point to the answer. You can format the label or tooltip text by changing the paragraph font and size or making the text bold, italic, or underlined. You can also include HTML code to add links or images. However, script tags aren't supported.

You can use the sequence number to order the answers to a question. Answers with a lower sequence number are displayed higher on the user interface. To identify answers in rules and actions, you can use a shortened version of the name as a code.

You can change the answer look and feel on the **Appearance** subtab of the answer record. For example, you can show the answer label but hide its graphical element, such as the checkbox or the radio button. This option is useful for displaying a list of features or options that users should read but don't need to interact with.

The space for each answer depends on the columns assigned to the question record in the **UI Columns** field. If the answers related to a question are distributed horizontally, you can manually assign the space occupied by each answer depending on the amount of text they contain.

Before manually assigning the space to answers, go to the question record and verify that the answer layout is horizontal and clear any value entered in the **Answers per Row** field. For more information, see [Changing the Question Look and Feel](#).

To create an answer:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Click **Edit** next to the product you want to modify.
3. On the **Questions** subtab, click name of the question for which you want to create an answer.
4. On the **Answers** subtab, click **New CPQC Answer**.
5. Enter a descriptive name for the answer.

This name is automatically copied to the **Label** field on the **Appearance** subtab. To change the name users see on the product interface, edit the text in the **Label** field.

In answers for tiles and tile dropdown lists, screen readers use the name as an alternate text and read it aloud to describe the images.

6. In the **Code** field, enter a unique identifier for this answer.
You can use a shortened version of the name as a code to identify the answer in rules and actions. The code can contain only uppercase letters, numbers, and underscore characters.
7. (Optional) In the **Description** field, enter a short explanation of the answer.
The description isn't displayed on the user interface.
8. Provide a sequence number to sort the answer within the question space on the user interface.
You can enter a negative or positive number—for example, -2 or 5.
9. On the **Rules** subtab, set up the rules to hide, make the answer read-only, or mark it as incompatible with the current configuration when users select certain features or options. For more information, see [Working with Question and Answer Rules](#).

10. Click the **Appearance** subtab.
11. In the **Label** field, enter the text users see on the interface for this answer.
12. (Optional) In the **Tooltip** field, enter text to provide additional information about the answer and help users understand it.
13. To display the answer label and hide the graphical element, check the **Show label only** box.
14. To hide the answer label and show only the graphical element, check the **Hide label** box.
15. In the **UI Columns** field, enter a number between 1 and 12 to define how many columns the answer can occupy in the question space.

 **Note:** This option doesn't apply to dropdown lists, tile dropdown lists, and input box questions.

16. On the **CPQ AI Assistance** subtab, define the AI description for use by the NetSuite CPQ AI Assistant. For more information, see the help topic [Prepare Configurable Items for NetSuite CPQ AI Assistant](#).
17. Click **Save**.

After saving an answer, you can edit it from the **Answers** subtab to modify it or add information. To copy or delete answers, see [Working with Product Building Blocks](#).

Working with Question and Answer Rules

When users configure an item, you can dynamically guide them through the configuration process based on the conditions you added to rules. With rules, you can display only features and options that are relevant and available based on the answers currently selected.

Several options to work with question and answer rules are available. You can hide questions or answers, you can make them read-only, or you can mark answers as incompatible.

You can set up rules on the **Rules** subtab of question and answer records. In the rule fields, you write the conditions by entering question and answer codes separated by the forward slash character. For example, you can add A/1 to the read-only rule of question B. If users select answer 1 for option A, then question B becomes read-only. To combine multiple conditions, you can use operators—for example, A/1&B/2. For more information, see [Activating or Hiding Building Blocks with Rules](#) and [Using Rule Operators](#). When you're finished, save the question or answer record.

For more information, see:

- [Hiding Questions or Answers](#)
- [Making Questions or Answers Read-Only](#)
- [Marking Answers as Incompatible](#)

Hiding Questions or Answers

When hiding questions or answers with rules, you can prevent users from viewing unavailable item features or options based on the choices they've made. Using an hide rule, you can also avoid displaying a question before users answer another one. When creating the hide rule, replace the answer code with the double percentage operator (%%). For example, you can add the condition A/%% to the rule of question B. Question B is hidden until users answer question A.

In the **Hide Rule** field on the **Rules** subtab, enter the question and answer combination that defines the condition to hide the question or answer.

When applying hide rules, you can clear questions or answers selected by users if they become hidden. This option ensures that unavailable options or features don't remain selected when users change their choices. To apply this option, check the **Reset when hidden** box on the question or answer record. Questions or answers are also cleared when the parent tab or group becomes hidden.

Making Questions or Answers Read-Only

When making questions or answers read-only, you let users view item features or options that they should read but not interact with, or that may be available if they choose a different answer. Read-only questions or answers appear dimmed on the user interface and can't be selected.

In the **Read-Only Rule** field on the **Rules** subtab, enter the question and answer combination that defines the condition to make the question or answer read-only.

Marking Answers as Incompatible

When marking an answer as incompatible, you let users know that their current choice doesn't meet the configuration requirements.

On the user interface, answers marked as incompatible become dimmed, and their graphical element is crossed out. When a previously selected answer no longer meets the configuration requirements, the answer label becomes red.

You can also add a message to explain to users why the answer is incompatible with their current choices and how they can correct the configuration. This message is displayed when users point to the answer marked as incompatible.

In the **Incompatible Rule** field on the **Rules** subtab, enter the question and answer combination to determine when this answer doesn't meet the configuration criteria. In the **Incompatible Rule Message** field, you can add a message to help users understand why the selection doesn't meet the configuration requirements and what option to choose instead.

To guide users in the configuration process, you can also add validation messages to the information bar and make sure they don't submit incorrect or incomplete configurations. For more information, see [Displaying Validation Messages on the User Interface](#).

Working with Specific Question and Answer Types

NetSuite CPQ Configurator provides a variety of question types that you can use depending on the number of answers, how many answers users can select, and how you want to display the answers. For more information about when and why you should use one question type rather than another, see [Gathering Information with Questions and Answers](#).

When creating question types and their answers, you may need to complete or enable more fields and options than the basic ones. The following help topics support you in this task:

- [Checkbox and Radio Button Questions](#)
- [Dropdown List Questions](#)
- [Tile and Tile Dropdown List Questions](#)
- [Custom HTML Questions](#)
- [Input Box Questions](#)
- [QTable Questions](#)

For more information about the basic steps to create questions and answers, see [Working with Questions](#) and [Working with Answers](#).

Checkbox and Radio Button Questions

Checkboxes let users choose multiple answers in response to a question. Radio buttons are more restrictive and let users choose only one answer from multiple choices.

You can add a text field or text box below a checkbox or radio button option to gather additional information from users, such as comments or feedback. The choice between a text field or a text box depends on the additional information required.

Use a text field for short answers, and a text box if you need more detailed information. The text field is hidden by default until a user selects an answer, but you can choose to always show the field onscreen. In the text field, users can also select or enter the date and time based on the predefined format of your choice.

To create a checkbox or radio button question with answers:

1. On the **Questions** subtab, click **New CPQC Question**.
2. In the **Type** field, select Checkbox/Radio Button.
3. In the **Answer Selection Type** field, select one of the following options:
 - **Single Choice** – Creates a radio button question.
 - **Multiple Choice** – Creates a checkbox question.
4. In the **Sort Answers By** field, choose to sort the answers by name or by sequence number.
5. In the **UI Columns** field, enter a number between 1 and 12 to determine how many columns the question can occupy on the user interface.
6. Click the **Appearance** subtab.
7. In the **Answer Layout** dropdown list, choose if you want answers in one column (vertically) or multiple columns (horizontally). For more information, see [Changing the Question Look and Feel](#).
8. Click **Save**.
9. On the **Answers** subtab, click **Edit**.
10. To add a text field below the checkbox or radio button option, check one of the following boxes:
 - **Add text field** – Adds a single-line text field.
 - **Add text box** – Adds a multi-line text area.
11. (Optional) To let users provide the date and time in the text field associated with the checkbox or radio button:
 1. In the **Input Box Answer Type** field, select Date & Time Field.
 2. Select the date format.
 3. Select the time format.
 4. (Optional) Users can select the date and time using the calendar and time selector. To let users type the date and time while enforcing the predefined format of your choice, check the **Enable date mask** box.

 **Note:** When entering the date and time, users will be able to type only allowed characters. Fixed characters—such as slashes and hyphens—will be automatically completed as they type. Using the left and right arrow keys, they can move between the date and time elements. With up and down arrow keys, they can increase or decrease the date and time values. Users will still be able to select the date and time using the calendar and time selector.

12. To always show the text field or box even if users haven't selected an answer, check the **Always show text field or box** option.

13. Click **Save**.

Dropdown List Questions

Dropdown list questions let users select an answer from a predefined list when they click the arrow.

To create a dropdown list question:

1. On the **Questions** subtab, click **New CPQC Question**.
2. In the **Type** field, select Dropdown List.
3. In the **Sort Answers By** field, choose to sort the answers by name or by sequence number.
4. In the **UI Columns** field, enter a number between 1 and 12 to determine how many columns the question can occupy on the user interface.
5. Click **Save**.

To create dropdown list answers, follow the basic steps to create answers. For more information, see [Working with Answers](#).

Tile and Tile Dropdown List Questions

Tile questions let you assign images to answers, and tile dropdown questions include an image next to each list option.

Screen readers use the value of the **Name** field as an alternate text and read it aloud to describe the image.

To save space on the user interface the interface, you can collapse tile answers. Users can then click the arrow to expand and view the options.

To create a tile or tile dropdown question with answers:

1. On the **Questions** subtab, click **New CPQC Question**.
2. In the **Type** field, select Tiles or Tiles Dropdown List.
3. In the **Answer Selection Type** field, select one of the following options:
 - **Single Choice** – Lets users select only one answer.
 - **Multiple Choice** – Lets users select more than one answer.
4. In the **Sort Answers By** field, choose to sort the answers by name or by sequence number.
5. In the **UI Columns** field, enter a number between 1 and 12 to determine how many columns the question occupies on the user interface.
6. Click the **Appearance** subtab.
7. In the **Answer Layout** dropdown list, choose to display the answers in one column (vertically) or across multiple columns (horizontally). For more information, see [Changing the Question Look and Feel](#).
8. To collapse the answers in a tile question, check the **Collapse tiles** box.
9. Click **Save**.
10. On the **Answers** subtab, click **New CPQC Answer**.
11. On the **Appearance** subtab, two options are available to add an image:
 - In the **Image** field, select an image from the File Cabinet.
 - In the **Image URL** field, enter the URL of the image.

If you select an image and also provide a URL, only the image from the URL is displayed on the user interface.

12. Click **Save**.

Custom HTML Questions

With custom HTML questions, you can add HTML code to the **Label** field of the answers. For example, you can add an image or a link.

 **Note:** Script tags aren't supported.

To create a Custom HTML question:

1. On the **Questions** subtab, click **New CPQC Question**.
2. In the **Type** field, select Custom HTML.
3. In the **Sort Answers By** field, choose to sort the answers by name or by sequence number.
4. In the **UI Columns** field, enter a number between 1 and 12 to determine how many columns the question occupies on the user interface.
5. Click the **Appearance** subtab.
6. In the **Answer Layout** dropdown list, choose to display the answers in one column (vertically) or across several columns (horizontally). For more information, see [Changing the Question Look and Feel](#).
7. Click **Save**.

Input Box Questions

Input box questions let you create various fields where users can enter data directly or select options from a popup table or a popup list (formerly called a picklist and picklist dropbox).

You can create the following types of input box answers:

- **Text field** (formerly called a free-form text) – Lets users enter text to provide a short open-ended response.
- **Text box** (formerly called a text area) – Lets users enter text in a multiline field to provide a long open-ended response.
- **Date and time field** – Lets users select date and time. Users can also enter the date and time directly in the field based on the predefined format of your choice.
- **Input group** – Lets users enter text and provides information with an additional label on the side of your choice.
- **Popup table and popup list** (formerly called a picklist and a picklist dropbox) – Let users select one or multiple options from a popup table or a popup list when they click the input box button or arrow. Popup tables and lists source options from product tables. For more information, see [Working with Popup Table and Popup List Answers](#) and [Storing Answer Options in Tables](#).

You can also show placeholder text in the input box. Users view this placeholder text before entering data or clicking the input box button or arrow. The placeholder text is the same as the label.

To create an input box question with answers:

1. On the **Questions** subtab, click **New CPQC Question**.
2. In the **Type** field, select Input box.

3. In the **Sort Answers By** field, choose whether sort the answers by name or by sequence number.
4. In the **UI Columns** field, enter a number between 1 and 12 to determine how many columns the question occupies on the user interface.
5. Click the **Appearance** subtab.
6. In the **Answer Layout** dropdown list, choose to display the answers in one column (vertically) or across multiple columns (horizontally). For more information, see [Changing the Question Look and Feel](#).
7. Click **Save**.
8. On the **Answers** subtab, click **New CPQC Answer**.
9. In the **Input Box Answer Type** field, select the type of the input box answer.
10. To create a date and time field, enter the date and time formats.
11. (Optional) Users can select the date and time using the calendar and time selector. To let users type the date and time while enforcing the predefined format of your choice, check the **Enable date mask** box.

Note: When entering the date and time, users will be able to type only allowed characters. Fixed characters—such as slashes and hyphens—will be automatically completed as they type. Using the left and right arrow keys, they can move between the date and time elements. With up and down arrow keys, they can increase or decrease the date and time values. Users will still be able to select the date and time using the calendar and time selector.

12. To create an input group, enter the label you want to display on the side of the input box in the **Input Group Label** field.
The label is added on the left by default. If you want the label on the right, check the **Show input group label on the right** box.
13. To display the content of the **Label** field as placeholder text in the input box, click the **Appearance** subtab and check the **Show label as placeholder** box.
14. Click **Save**.

QTable Questions

Note: QTables are different from product tables, which store the answer options for popup tables and popup lists. For more information, see [Storing Answer Options in Tables](#) and [Working with Popup Table and Popup List Answers](#).

A qTable is a question presented as a table with each column corresponding to an answer. You can assign a different answer type to each column, and a column can only store cells of the same answer type.

You can use this question type to save time when your product includes a considerable amount of question and answer combinations. Instead of remembering thousands of codes for rules, a number preceded by an underscore is automatically assigned to each column code for each cell. For example, if your qTable has the question code QTABLE and an column with code CHECKBOX, the cell rules will be QTABLE/CHEKBOX_1, QTABLE/CHEKBOX_2, and so on.

The following answer/column types are available for qTable questions:

- **Text field** (formerly called text input) – Lets users enter short open-ended responses in each cell.
- **Checkbox** – Lets users check a box in each cell.
- **Dropdown list** – Lets users select a single option from a dropdown list in each cell by clicking the arrow.

- **Date and time field** – Lets users select date and time in each cell. Users can also enter the date and time directly in the cell based on the predefined format of your choice.
- **Custom HTML** – Lets users view the HTML code you've added to the answer/column in each cell. For example, you can add an image or a link. This answer/column type is read-only, and script tags aren't supported.
- **Popup list** (formerly called a picklist dropdown) – Let users select an option from a popup list when they click the input box arrow in each cell. Popup lists source options from product tables. For more information, see [Working with Popup Table and Popup List Answers](#) and [Storing Answer Options in Tables](#).

In addition to fields common to all questions and answers, complete the fields specific to the qTable question type and its answers.

To create a qTable question:

1. On the **Questions** subtab, click **New CPQC Question**.
2. In the **Type** field, select QTable.
3. In the **Sort Answers By** field, choose to sort the answers by name or by sequence number.
4. In the **UI Columns** field, enter a number between 1 and 12 to determine how many columns the question occupies on the user interface.
5. Click the **QTable** subtab.
6. In the **Initial Rows** field, enter the initial number of rows for the qTable.
7. (Optional) In the **Maximum Visible Rows** field, enter the maximum number of rows for the qTable.
If the number of rows exceeds this limit, a scroll bar appears to let view the rest of the qTable on the product interface.
8. (Optional) The qTable header row is fixed and remains visible as users scroll down. To unlock it and let it move out of view as they scroll down past the visible rows, check the **Unfreeze top row** box.



Note: A maximum number of visible rows must be set in the **Maximum Visible Rows** field.

9. To allow users to insert or delete rows, check the **Allow row number editing** box.
If checked, users can right-click the numbered column in the qTable, and a menu with the following options opens:
 - **Insert** – Adds a row at the bottom of the qTable.
 - **Delete** – Deletes the last row of the qTable.
10. To allow users to edit data in the qTable, check the **Allow cell editing** box.
Clear this box to make the entire qTable read-only.
If you only want certain columns to be read-only, check the **Allow cell editing** box and set a read-only rule for the non-editable answers/columns. If those columns should always be read-only, you can enter the operator */* in the read-only rule. For more information, see [Activating or Hiding Building Blocks with Rules](#).
11. Click **Save**.
12. On the **Answers** subtab, click **New CPQC Answer**.
13. In the **QTable Column Type** dropdown list, select the qTable answer type.
14. To add a date and time column, enter the date and time formats.
15. Users can select the date and time using the calendar and time selector. To let users type the date and time while enforcing the predefined format of your choice, check the **Enable date mask** box.

Note: When entering the date and time, users will be able to type only allowed characters. Fixed characters—such as slashes and hyphens—will be automatically completed as they type. Using the left and right arrow keys, they can move between the date and time elements. With up and down arrow keys, they can increase or decrease the date and time values. Users will still be able to select the date and time using the calendar and time selector.

16. To add a custom HTML column, enter the HTML code in the **QTable Custom HTML Content** field on the **QTable Cells** subtab.
17. To add a dropdown list column, click the **QTable Cells** subtab and click the open icon next to the **QTable Dropdown List Options** field.

In the popup window:

1. Double-click the text field and enter the first option.
2. Click the plus icon and create as many options as required.
3. To save the options, click **OK**.

The options are automatically copied to the **QTable Dropdown List Options** field in JSON format. The syntax is:

```
1 | ["option 1","option 2","option 3"]
```

18. (Optional) To keep the qTable column visible and pinned to the left side as users scroll horizontally to the right, check the **Freeze column** box on the **QTable Cells** subtab.
19. Click **Save**.

Working with Popup Table and Popup List Answers

Popup tables and popup lists (formerly called picklists and picklist dropboxes) are answer types that display filtered content from product tables as options. For more information about the basic steps to create answers, see [Working with Answers](#).

You can create popup tables for input box questions, and popup lists for both input box and qTable questions. For more information, see [Storing Answer Options in Tables](#), [Input Box Questions](#), and [QTable Questions](#).

When creating a popup table or list, you must specify the product table that stores the options and the column that provides the values for the input box field. When users make a selection in a popup table or list, they select an entire row of the product table. However, the input box displays only the value of the column you chose during the setup. You can map the rest of the values to other fields. For more information, see [Completing Questions or Answers Dynamically from Popup Tables or Lists Selections](#).

Users open the popup table or list by clicking the input box or the double arrow button on the product interface. You can let users select multiple options, and the values will appear separated by commas in the input box. To select multiple options on the product interface, users must hold down the Ctrl key. You can add a tooltip to let them know that they can perform this action and how to do it. Search boxes are available to help users find specific values in the popup tables or lists.

If you make a popup table, a popup list, or the associated input box question read-only, the popup table or list won't show up for users. You can still let them view the popup table or list when they click the input box button. This option is useful to show features and options that may be available if users make a different selection.

For more information about popup tables and lists see, [Working with Popup Table and List Answers for Input Box Questions](#) and [Working with Popup List Answers for QTable Questions](#).

Working with Popup Table and List Answers for Input Box Questions

When creating popup tables or lists as answers for input box questions, follow the basic steps mentioned in this help topic.

To work with a popup table or list:

1. On the **Answers** subtab of an input box question, click **New CPQC Answer**.
2. In the **Input Box Answer Type** field, select one of the following options:
 - **Popup Table** – Creates a popup table (formerly called a picklist).
 - **Popup List** – Creates a popup list (formerly called a picklist dropdown).
3. Click the **Popup Tables & Lists** subtab.
4. Go to the **General Information** section.
5. In the **Reference Table** field, select the product table you want to associate with the popup table or list.
6. In the **Reference Column** field, select the column that provides the value to display in the input box.
7. To let users select multiple options in a popup table or list, check the **Allow multiple selections** box.

 **Note:** This option isn't available for qTable popup lists.

8. To let users view popup tables or lists by clicking the button or the double arrow when the field is read-only, check the **View when read-only** box. Users still won't be able to enter data in the field.

 **Note:** This option isn't available for qTable popup lists.

9. Click **Save**.

After finishing these steps shared by popup tables and popup lists, also complete additional steps specific to each answer type. For more information, see the following help topics:

- [Defining the Popup Table Content](#)
- [Filtering the Popup Table Content](#)
- [Creating a Popup Table Filter for Users](#)
- [Defining the Popup List Content](#)

Defining the Popup Table Content

When creating a popup table as an answer to an input box question, you must specify the product table columns you want to display to users. You can also add labels to the columns.

To define the product table columns to display in the popup table:

1. Edit the popup table answer.
2. Click the **Popup Tables & Lists** subtab.
3. Click the open icon next to the **Select Product Table Columns** field.
4. In the popup window, select the column from the product table that you want to display in the popup table.
5. Provide a label for the column. This label is displayed to users.

6. To display the column in the popup table, check the **Visible** box next to the product table column selected.
If you want to map the column but not display it, leave the box unchecked.
7. To add more product table columns in the popup table, click the **Add Column** button. Repeat steps 4 – 6.
8. After adding all the required columns, click **Done** to add the information to the **Select Product Table Columns** field.
Your preferences are saved to this field in JSON format. This is the syntax:


```
[{"data": "column1", "title": "label1", "visible": true},
{"data": "column2", "title": "label2", "visible": true}]
```
9. Click **Save**.

Filtering the Popup Table Content

After choosing the product table and the columns to display in the popup table, you can filter the product table by specific values. The popup table will display the matching results.

Two options are available to provide the values for filtering:

- Manually entering the value.
- Taking the value from a question or an answer.

This option dynamically adapts the content of the popup table depending on specific user inputs.

You can create multiple filters with a variety of conditions, such as containing or being equal to the value for filtering. To associate the filter with checkboxes or radio buttons, one of the product table columns must contain true and false as values.

To filter popup tables by value:

1. Edit the popup table answer.
2. Click the **Popup Tables & Lists** subtab.
3. Click the open icon next to the **Product Table Filter** field.
4. In the popup window, select the product table column you want to filter.
5. Select a condition, such as **contains** or **equals to**.
6. To select the value to filter the column, two options are available:
 - In the **Custom Value** field, enter a value.
 - Select a question or an answer that contains text.
7. To add another filter:
 - Click the **Add Filter** button.
 - Select a logical operator, such as AND or OR.
 - Repeat steps 4 – 6.
8. To save your filtering preferences, click **Done**.
9. Click **Save**.

Creating a Popup Table Filter for Users

You can create a filter to let users narrow down the popup table content. The filter will list the product table columns you included in the popup table, and the column names will match the labels you assigned to them.

The filter is displayed above the popup table on the product interface. Users can filter the popup table by checking the boxes next to the available options for each column.

To create a popup table filter for users:

1. Edit the popup table answer.
2. Click the **Popup Tables & Lists** subtab.
3. To define the columns that users can select for filtering, click the open icon next to the **UI Popup Table Filter** field.
4. In the popup window, select the column you want to include in the product interface filter.
In this field, you can view the column names from the product table. However, the filter will display the labels you assigned to columns when creating the popup table.
5. In the **Filter Type** column, select the **Multiple Choice** option.
Users will always be able to select multiple options from the filter.
6. To add other columns to the user interface filter, click **Add Column** and repeat steps 4 – 5.
7. To save your filtering preferences, click **Done**.
8. Click **Save**.

Defining the Popup List Content

When creating a popup list (formerly called a picklist dropdown), you must specify the product table columns that provide the options for the popup list. The popup list displays the values of the selected columns in a dropdown list on the product interface.

To ensure that column values are properly displayed:

- Enclose the column names between curly braces, for example {Column Name}.
- Enter the column names exactly as they appear in the product table.

For example, you have the following product table:

Color Name	Color Code
Navy Blue	NVY
Gray	GRY
Gold	GLD

Suppose you want to display the color name and the color code in a popup list. You also want the color code in parentheses. To achieve this result, use the following syntax:

```
1 | {Color Name} ({Color Code})
```

The options displayed in the list would be:

- Navy Blue (NVY)
- Gray (GRY)
- Gold (GLD)

You can also define the look and feel of the options by applying formatting to the column names. You can change the font and size or make the text bold, italic, or underlined. To check the formatting and view the HTML code, click the **Source** button. When adding formatting, place the tags outside the curly braces. See the following example:

```
1 | <b>{Column Name}</b>
```

After the popup list loads on the product interface, users can filter it using the search box. If the popup list is associated with a dynamic table with a search filter, you can let users search it before it loads. The popup list will only return the options matching the value entered in the search box. This option saves users time when working with large lists that may take a while to load. For more information about dynamic tables, see [Storing Answer Options in Tables](#).

To define the popup list information:

1. Edit the popup list answer.
2. Click the **Popup Tables & Lists** subtab.
3. In the **Popup List Options** field, enter the column names between curly braces.
4. To let users search a popup list before it loads, check the **Allow search before loading** box.

Note: You can only use this option if the popup lists is linked to a dynamic table with a search filter.

5. Click **Save**.

Working with Popup List Answers for QTable Questions

You can create popup lists (formerly called picklist dropboxes) as answers for qTable questions. QTable popup lists work the same way as popup lists for input boxes. For more information, see [Defining the Popup List Content](#).

To create a popup list answer for a qTable question:

1. On the **Answers** subtab, click **New CPQC Answer**.
2. In the **QTable Column Type** dropdown list, select **Popup List**.
3. Click the **Popup Tables & Lists** subtab.
4. Go to the General Information section.
5. In the **Reference Table** field, select the product table associated with the qTable popup list.
6. In the **Reference Column** field, select the column that provides the value displayed in the qTable cell.
7. In the **Popup List Options** field, define the product table columns you want to display in the popup list.

To enter the column names, use the following syntax:

```
1 | {Column Name}
```

Note: When adding formatting, place tags outside the curly braces. See the following example:

```
<b>{Column Name}</b>
```

You can check the formatting by clicking the **Source** button.

8. Click **Save**.

Completing Questions or Answers Dynamically from Popup Tables or Lists Selections

When users make a selection in a popup table or list (formerly called picklists and picklist dropboxes), they select an entire row from the product table. The input box field will display the value of the column you chose during the setup. You can map other values from the same row to other questions or answers. For more information about product tables, see [Storing Answer Options in Tables](#).

Using mapping options, you can dynamically complete questions or answers based on user selections on popup tables or popup lists. This option eases the configuration process for users and ensures that questions and answers are filled in according to the established requirements.

To complete questions and answers dynamically by using popup table or list data, go to the Mapping section on the **Popup Tables & Lists** subtab of the answer record.

For more information, see the following help topics:

- [Mapping Popup Table or List Selections to Questions or Answers](#)
- [Mapping Popup Table Selections to QTables](#)
- [Mapping QTable Popup List Selections to Questions or Answers](#)

Mapping Popup Table or List Selections to Questions or Answers

You can map product table values to questions and answers based on what users select in popup tables or lists. When setting up the mapping, specify:

- The product table column that provides the values for the popup table or list.
- The question or answer where you want map the additional values from the same row.

To map popup table or list selections to questions and answers:

1. Edit the popup table or list answer.
2. On the **Popup Tables & Lists** subtab, go to the Mapping section.
3. Click the open icon next to the **Mapping to Questions/Answers** field.
4. In the popup window, select the product table column from which you want to take the value.
5. Select the question or answer where you want to map the value.
6. To map multiple columns to question or answers, click **Add Mapping**. Repeat steps 4 – 5.
7. To clear the mapped value and add a new one each time users select an option in the popup table or list, check the **Clear mapped questions/answers on reset** box.
8. To save your mapping preferences, click **OK**.
9. Click **Save**.

Mapping Popup Table Selections to QTables

You can map product table values to qTables based on user selections on popup tables (formerly called picklists).

Each user selection in the popup table fills a different cell in the selected qTable column—one cell after the other. To have user selections always appear in the same cell and delete the previously mapped value, check the **Clear qTable cell before mapping** box.

To map a popup table selection to a qTable:

1. Edit the popup table answer.
2. On the **Popup Tables & Lists** subtab, go to the Mapping section.

3. In the **Target QTable** field, select the qTable where you want to map values.
4. Click the open icon next to the **Mapping to QTable** field.
5. In the popup window, select the product table column you want to map to the qTable.
6. Select the qTable column in which you want to map the product table column.
7. To add more columns, click **Add Column** and repeat steps 5 – 6.
8. To save your preferences, click **Done** in the popup window.
9. Click **Save**.

Mapping QTable Popup List Selections to Questions or Answers

You can map values from product table columns to questions or answers based on what users select on qTable popup lists.

To map a qTable popup list selection to questions and answers:

1. On the **Answers** subtab, edit a qTable popup list.
2. Click the **Popup Tables & Lists** subtab.
3. In the Mapping section, click the open icon next to the **Mapping to Questions/Answers** field.
4. In the popup window, select the product table column from which you want to take the values.
5. Select the question or answer where you want to map the values.
6. To add more columns, click **Add Mapping** and repeat steps 4 – 6.
7. To save your preferences, click **Done**.
8. Click **Save**.

Storing Answer Options in Tables

When you want to offer users long lists of related options, you can store answers in product tables. By storing answers in tables, you can simplify the process of adding, removing, and updating the options for a question. For example, if your business sells desks, you can add several new desk types at the same time without creating a separate answer for each desk type.

These answer types can access product tables:

- Input box popup tables (formerly called picklists)
- Input box popup lists (formerly called picklist dropboxes)
- QTable popup lists (formerly called picklist dropboxes)

For more information about popup tables and popup lists, see [Working with Popup Table and Popup List Answers](#).

These types of tables are available:

- **Static tables** – Let you manually create a table or import data from a CSV file stored in the File Cabinet. Static tables store data in the table record. These tables work well with a limited amount of data that only requires occasional updates. For more information, see [Creating Static Tables](#).
- **CSV tables** – Let you access data from CSV files stored in the File Cabinet. For more information, see [Creating CSV Tables](#). CSV tables don't store data in the table record and are useful when working with large amounts of data that require occasional updates. For more information about uploading a CSV file to the File Cabinet, see the help topic [Uploading Files to the File Cabinet](#).
- **Dynamic tables** – Let you create tables using data from any record type available in your account. Each record field becomes a column in the table. Dynamic tables automatically update when data

change in your account. This table type is useful when working with account data that only needs to be accessed occasionally. For more information, see [Creating Dynamic Tables](#).

- **Function-generated tables** (formerly called CE-generated tables) – Let you create tables that you can fill in using NetSuite CPQ Configurator functions. You can use this table type when working with data that requires frequent access and updates. For more information, see [Creating Tables Using Functions](#).

To create a table:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Click **Edit** next to the product you want to modify.
3. On the **Tables** subtab, click **New CPQC Table**.
4. Enter a descriptive name for the table.
The name is used for internal reference only.
5. In the **Code** field, enter a unique identifier for the table.
The code can contain only uppercase letters, numbers, and underscores. You can enter a shortened version of the name as the code to identify the table in actions.
6. Select a table type.
7. Click **Save**.

After saving a table, you can edit it from the **Tables** subtab and modify its information or add new data. To copy or delete tables, see [Working with Product Building Blocks](#).

Creating Static Tables

Static tables let you manually create and maintain a table in the table record. You can also enter JSON data in the **JSON Data** subtab and the table will be automatically displayed in the editor. The table editor and the subtab with JSON Data are synchronized. When you apply changes to one subtab, the other one is updated as well.

 **Note:** Static tables have a one million character limit. To create large tables, use the CSV table type.

You can also create a static table by importing data from a CSV file. For more information, see [Importing Data Into Static Tables from CSV files](#).

Before completing the steps for this table type, follow the basic steps to create a table.

To create a static table:

1. On the **Tables** subtab, click **New CPQC Table**.
2. In the **Table Type** field, select **Static Table**.
3. To enter data for the table, two options are available:
 - In the **Table Editor** subtab, click a row and enter text.
Use the table editor buttons to add or remove rows and columns as needed.
 - In the **JSON Data** subtab, paste data in JSON format.
For example, a table with two columns and three rows would have these structure:

```
1 [{"ColumnName1": "Value", "ColumnName2": "Value"}, {"ColumnName1": "Value", "ColumnName2": "Value"}, {"ColumnName1": "Value", "ColumnName2": "Value"}]
```

4. Click **Save**.

Importing Data Into Static Tables from CSV files

Instead of entering data manually, you can import data for a static table from a CSV file stored in the File Cabinet. Data will be saved in the table record.

Before completing the steps for this table type, follow the basic steps to create a table.

To create a static table from a CSV file:

1. On the **Tables** subtab, click **New CPQC Table**.
2. In the **Table Type** field, select **Static Table**.
3. In the **CSV File** field, select the CSV file you want to use as the data source for the static table, for example, a comma or a semicolon.
4. In the **Column Separator** field, select the symbol that separates the columns in the CSV file, for example, a comma or a semicolon.
5. In the **Escape Characters** field, select the character used to indicate that the following character should be interpreted literally, or to enclose text with special characters.
6. Click **Save**.
7. On the **Tables** subtab, edit the table you've created to continue the setup.
8. To unlink the CSV file after loading its data, check the **Unlink static table from CSV file** box.
The static table will no longer be associated with the CSV file, and you can save the changes applied to the static table in the table record.
If you clear this box, any changes you make to the static table on the table record will be disregarded.



Note: Alternatively, you can unlink the static table from the CSV file by clearing the **CSV File** field after loading the data. Use this option if you don't need to keep track of the CSV file name.

9. Click **Save**.

Creating CSV Tables

CSV tables retrieve data from CSV files stored in the File Cabinet. Unlike static tables, CSV tables don't store data in the table record and you can use them for large tables to avoid the one-million character limit. To update a CSV tables, you can upload a CSV file with the same name to the File Cabinet.

Before completing the steps to create a CSV table, follow the basic steps to create a table.

To create a CSV table:

1. On the **Tables** subtab, click **New CPQC Table**.
2. In the **Table Type** field, select **CSV Table**.
3. In the **CSV File** field, select the CSV file you want to use as the data source.
4. In the **Column Separator** field, select the symbol that separates the columns of the CSV file, for example, a comma or a semicolon.
5. In the **Escape Characters** field, select the character used to indicate that the following character should be interpreted literally, or to enclose text with special characters.
6. Click **Save**.

Creating Dynamic Tables

Dynamic tables retrieve data from the record type you specify. You can choose any record type available in your account, and the table content automatically updates whenever those records change. When users click the button of the popup table or list associated with the table, answer options are dynamically loaded from the table into these answer types.

To add content to a dynamic table, specify the record fields you want to include. Each field becomes a column in the table. If you don't specify any fields, the dynamic table remains empty. After defining the table content, you can filter the dynamic table by field value to narrow down the results and display only information that matches the criteria defined in popup tables or lists.

 **Note:** Record fields from sublists aren't available.

You can preview the content of the dynamic table by clicking the **Test Table** button.

Before completing the steps to create a dynamic table, follow the basic steps to create a table.

To create a dynamic table:

1. On the **Tables** subtab, click **New CPQC Table**.
2. In the **Table Type** field, select **Dynamic Table**.
3. Select the record type from which you want to retrieve data.
4. To add columns, click the plus icon next to the **Record Fields** field and select the record fields you want to include.
5. To narrow down the table results and retrieve a subset of records, click the open icon next to the **Search Filter** field.

In the popup wizard:

- Select the record field you want to use for filtering.
- Select an operator.
The available operators depend on the field selected.
- Enter the filter text.
- Click **OK**.

6. To add multiple filters and retrieve a more specific subset of records, repeat step 5.

When adding another filter, you must select a logical operator (AND or OR) to combine the filters. AND produces a more restrictive condition, whereas OR results in a less restrictive condition.

The filters are automatically copied to the **Search Filter** field in JSON format. The syntax is:

```
1 | [{"fieldID1","operator","value1"},"and",["fieldID2","operator","value2"]]
```

7. To preview the dynamic table content, click the **Test Table** button.
8. Click **Save**.

Creating Tables Using Functions

Function-generated tables (formerly called CE-generated tables) are dynamically created when you open a catalog. After creating this table type, you can use functions to fill it in when the product loads.

To create a function-generated table:

1. Go to the **Tables** subtab.

2. Click **New CPQC Table**.
3. In the **Table Type** field, select **Function-Generated Table**.
4. Click **Save**.

Obtaining Answer Data with Resolve Expressions

With NetSuite CPQ Configurator, you can use resolve expressions to fill in record fields when you set up your product and dynamically retrieve information for the configuration from answers users select on the product interface. For example, you can use answers provided by users to specify the item name or number for a material, instead of entering a fixed name in the material record and creating a separate record for every possible material. The material item name or number will depend on the answers users select on the product interface, and you can use one material record to retrieve multiple items used as materials.

To write resolve expressions in the record fields that support them—called resolvable fields—use question and answer codes separated by a forward slash and enclosed in curly braces, for example, {A/1}. If you want to retrieve only part of the information dynamically, you can add characters outside the curly braces to generate the final result, for example, {A/1}-XXX-{B/2}-XX.

Using the codes in the curly braces, NetSuite CPQ Configurator finds the answer value from the user interface or other information from the answer record, such as its code, name, label, or description.

For multiple-choice questions, the resolve expression returns the data of all selected answers, separated by commas and ordered by their sequence number.

Types of Resolve Expressions

The following table identifies the available resolve expressions. Q represents the question code, and A is the answer code. Replace Q and A with codes from your product.

Resolve Expression	Description
{Q/A}	Returns the answer value. When questions can have only true or false as values, like checkboxes or radio buttons, this resolve expression returns true or an empty string.
{Q}	Returns the answer code. If users select multiple answers, this resolve expression returns the codes for all selected answers, separated by commas and ordered by sequence number.
{Q:ALT_NAME}	Returns the value of the Name field from the answer record. When users select multiple answers, this resolve expression returns the record name of all selected answers, separated by commas and ordered by sequence number.
{Q:NAME}	Returns the value of the Label field from the answer record. When users select multiple answers, this resolve expression returns the label of all selected answers, separated by commas and ordered by sequence number.
{Q:DESC}	Returns the value of the Description field from the answer record. When users select multiple answers, this resolve expression returns the description of all selected answers, separated by commas and ordered by sequence number.

Performing Calculations on Numbers Returned by Resolve Expressions

You can perform calculations on numbers returned by resolve expressions using basic mathematical and rounding operations to obtain the final value for resolvable fields. By performing calculations in resolvable

fields, you can dynamically determine and adjust values—such as quantities, prices, and multipliers—based on answers users select on the product interface.

To perform calculations in resolvable fields, enclose resolve expressions in the `calc()` function and combine them with mathematical operators. For example, you can enter `calc(({A/B}+{B/C})*3)` in the **Quantity** field of an additional item record. With this resolve expression, you can dynamically determine how many of those additional items a transaction will include based on user choices.

You can use the `calc()` function in resolvable fields that require numbers, such as **Quantity** and **Price** fields, as well as in text fields that support resolve expressions, such as the **Message** field for validation records. Text fields can include multiple `calc()` functions combined with text.

If the resolve expression doesn't contain numbers or the `calc()` function is written incorrectly, the result will be **NaN**, which stands for not a number. You'll find more information about the issue in the console.

Supported Operations and Operators

The following mathematical operations and operators are available.

Operator Symbol	Description
()	Group calculations and specify their order. Operations inside parentheses are performed first. In the example <code>calc(({A/B}+{B/C})*3)</code> , the addition happens before the multiplication.
+	Addition – Sums numbers.
#	Subtraction – Returns the difference between two numbers.
*	Multiplication – Multiplies two or more numbers.
/	Division – Divides one number by another.
**	Power – Raises a number to the power of the specified exponent. For example, <code>calc({Q/A}**3)</code> raises the value returned by <code>{Q/A}</code> to the third power.

Using Rounding Functions

When resolve expressions return decimal numbers, you can round the results using the `round()`, `ceil()`, or `floor()` functions inside the `calc()` function, for example, `calc(round({Q/A}+{Q1/A1}))`.

Using the round() Function

The `round()` function rounds a value to the specified position. The syntax is:

```
1 | calc(round({Q/A}, position))
```

The rounding position can be a number from 1 to 100, both inclusive. 0 is the default rounding position and can be omitted. If the digit to the right of the rounding position is 5 or greater, this function rounds up the value to the nearest number or rounding position; otherwise, it rounds down the value.

The following table shows examples of the rounding operation with numeric values.

Rounding Operation	Rounded Value
<code>calc(round(3.4))</code>	3
<code>calc(round(3.5))</code>	4
<code>calc(round(3.6))</code>	4

Rounding Operation	Rounded Value
calc(round(-3.4))	-3
calc(round(-3.5))	-4
calc(round(-3.5))	-4
calc(round(2.36, 1))	2.4
calc(round(4.5+3.2))	8
calc(round((4.5+3.2)/2,1))	3.9

Using the ceil() Function

The function ceil() rounds up a value to the nearest number with no decimals. The syntax is:

```
1 | calc(ceil({Q/A}))
```

The following table shows examples of the rounding-up operation with numeric values.

Rounding-Up Operation	Rounded Up Value
calc(ceil(3.64))	4
calc(ceil(3.65))	4
calc(ceil(3.66))	4
calc(ceil(-3.64))	-3
calc(ceil(-3.65))	-3
calc(ceil(-3.66))	-3
calc(ceil(4.5+3.2))	8
calc(ceil((4.5+3.2)/2))	4

Using the floor() Function

The function floor() rounds down the value to the nearest number with no decimals. The syntax is:

```
1 | calc(floor({Q/A}))
```

The following table shows examples of the rounding-down operation with numeric values.

Rounding-Down Operation	Rounded Down Value
calc(floor(3.64))	3
calc(floor(3.65))	3
calc(floor(3.66))	3
calc(floor(-3.64))	-4
calc(floor(-3.65))	-4
calc(floor(-3.66))	-4

Rounding-Down Operation	Rounded Down Value
calc(floor(4.5+3.2))	7
calc(floor((4.5+3.2)/2))	3

Organizing Questions into Tabs and Groups

With NetSuite CPQ Configurator, you can use tabs and groups to organize questions related to the same topic in a single place and guide users in the configuration process. Tabs create separate panes for questions. Groups organize questions into sections within each tab. You can create as many groups as required for each tab.

Tabs and groups have an internal name and a label for the product interface. You can format the label by changing the font, size, or making the text bold, italic, or underlined.

The sequence number determines the order in which tabs and groups are displayed on the product interface. Tabs and groups with a lower sequence number are displayed first. You can use negative or positive numbers. For example, -2 or 5.

You can hide tabs and groups on the product interface using rules. Hiding tabs or groups prevents users from viewing questions and answers for features or options that aren't available. To write a rule, enter question and answer codes separated by the forward slash character. For example, A/1. Use operators to combine multiple question and answer sets. For example, A/1&B/2. For more information, see [Activating or Hiding Building Blocks with Rules](#).

To assign a question to a specific tab or group, select the tab or the group in the **Tab/Group** field on the question record. In this field, groups are displayed as Tab Name/Group Name. For more information about questions, see [Working with Questions](#). If you want to use a tab in a rule field, enter the predefined answer `SOLE/TAB["TAB_CODE"]`. For more information about predefined answers, see [Including Additional Conditions in Rules and Actions with Predefined Answers](#).

Note: Assign at least one question to each tab or group to make them visible on the product interface.

For more information about tabs and groups, see [Creating Tabs](#) and [Creating Groups](#).

You can change the look and feel of all tabs and groups for a product. For more information, see [Changing Label and Header Colors for Questions, Tabs, and Groups](#).

Additionally, you can copy tabs and groups, and delete them when they are no longer needed. For more information, see [Working with Product Building Blocks](#).

Creating Tabs

By organizing questions into tabs, you can help users go through the different steps of the configuration process and focus on one step at a time.

When working with popup table answers (formerly called picklists), you can add tabs to the popup window on the product interface. If you add tabs on the popup table window, the table is displayed under the **General** tab by default.

To create a tab:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Click **Edit** next to the product you want to modify.
3. On the **Tabs** subtab, click **New CPQC Tab**.

4. Enter a descriptive name for the tab.
This name is used for internal reference and isn't visible to users.
5. In the **Code** field, enter a unique identifier for this tab.
The code can contain only uppercase letters, numbers, and underscore characters. You can enter a shortened version of the name as the code.
6. Provide a sequence number to determine the order in which the tab is displayed on the product interface. Tabs with a lower sequence number are displayed before those with a higher number.
7. In the **Label** field, enter the text that is displayed on the product interface for this tab.
8. In the **Hide Rule** field, provide the question and answer combination that determines whether the tab should be hidden on the product interface.
9. To add the tab to a popup table window, select the popup table answer where you want to add a tab in the **Add to popup table** field.
10. Click **Save**.

Creating Groups

By dividing questions into groups within tabs, you can guide users through each substep of the configuration process. You can also collapse groups to save space on the product interface and help users focus on the most relevant questions at each substep.

To create a group:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Click **Edit** next to the product you want to modify.
3. On the **Tabs** subtab, click the tab name where you want to create a group.
4. On the **Groups** subtab, click **New CPQC Group**.
5. Enter a descriptive name for the group.
This name is used for internal reference only and isn't visible on the product interface.
6. In the **Code** field, enter a unique identifier for this group.
You can enter a shortened version of the name as the code. The code can contain only uppercase letters, numbers, and underscore characters.
7. Provide a sequence number to determine the order in which the group appears on the product interface.
Groups with a lower sequence number are displayed higher within the tab.
8. In the **Label** field, enter the text displayed on the product interface for this group.
9. In the **UI Columns** field, enter a number between 1 and 12 to determine how many columns the group can occupy on the product interface.
10. In the **Hide Rule** field, provide the question and answer combination that determines whether the group is hidden on the product interface.
11. The group header and borders are displayed by default. If you don't want to display them, clear the **Show group box** box.
12. To collapse a group, two options are available:
 - To let users collapse the group on the interface, check the **Allow collapsing** box.
 - To display the group as collapsed and let users expand it on the interface, check the **Collapsed by default** box.
13. Click **Save**.

Displaying Validation Messages on the User Interface

You can show users validation messages to guide them through the configuration process, provide additional information, as well as communicate any inconsistencies in the configuration. Using validation messages, you can help users resolve issues, submit accurate configurations, and reduce errors.

Validation messages are displayed at the top of the information bar and below the configurable item image. You can format the message by changing the font and size, or by making the text bold, italic, or underlined. In the message, you can also include information from questions and answers using resolve expressions.

To include resolve expressions, use question and answer codes separated by a forward slash and enclosed in curly braces. For example, {A/1}. The answer value will be retrieved using the codes. For more information, see [Obtaining Answer Data with Resolve Expressions](#).

The following validation message types are available:

- **Note** – Provides information or suggestions to users. This validation message doesn't prevent submitting the configuration.
- **Incomplete configuration** – Informs users that required information or selections are missing. If you check the **Prevent submitting incomplete configurations** box on the product record, this message prevents submitting the configuration.
- **Incompatible configuration** – Notifies users that the configuration doesn't meet the established requirements. If you check the **Prevent submitting incompatible configuration** box on the product record, users can't submit a configuration with this validation message.

If users need to complete the task in phases and plan to update the configuration later, you can let them submit configurations even if there are incomplete or incompatible validation messages.

To create a validation message:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Click **Edit** next to the product you want to modify.
3. On the **Validations** subtab, click **New CPQC Validation**.
4. Enter a descriptive name for the validation message.
This name is used for internal reference only and isn't displayed on the product interface.
5. Select the validation message type.
6. Provide a sequence number to determine the order in which the validation message is displayed in the validation message container on the product interface.
Validation messages with a lower sequence number are displayed before those with a higher sequence number. You can enter a negative or positive number. For example, -2 or 5.
7. (Optional) To define a specific date range in which the validation message is active and displayed on the user interface:
 - In the **Effective Start Date** field, select a start date.
Before this date, the validation message isn't displayed.
 - (Optional) In the **Effective End Date** field, select an end date.
After this date, the validation message is no longer displayed.

 **Note:** If you define a date range, the validation message will only be displayed if both the date range and rule conditions are met.

8. Provide a message to display to users.

9. In the **Rule** field, enter the condition that determines whether this validation message is displayed to users.
Enter question and answer codes separated by a forward slash. For example, A/1. Use operators to combine multiple conditions. For example, A/1&B/2. For more information, see [Activating or Hiding Building Blocks with Rules](#).
10. (Optional) To include an additional rule in the validation message rule, select a rule category. The validation message will be displayed only if both its rule and the category rule match the questions or answers selected by users. For more information, see [Working with Rule Categories](#).
11. Click **Save**.

To copy or delete a validation message record, see [Working with Product Building Blocks](#).

Displaying Preview Images of the Configurable Item

You can display images of the configurable item on the product user interface. Adding images can help users:

- Better understand the item they're configuring.
- See how features and options fit together.
- Preview the final look and feel of the configured item.

The image container is displayed above the summary in the information bar. If no image is available, the container remains hidden. Images are added to the image container when their rules match the answers users select on the product interface. For more information about the image container, see [Changing the Appearance of Tabs, Groups, and the Image Container](#).

Inside the image container, you can create either a composite image or an image gallery. Composite images display the item while it's being configured. A composite image is made by layering multiple images with transparent backgrounds on top of each other. Each image includes only one feature or option of the configurable item. With composite images, you can mix and match features and options that vary in color, size, material, and so on without needing an image for every possible combination.

Alternatively, you can show users the configurable item using an image gallery. To display images one after another in a gallery, check the **Enable image gallery** box on the product record.

When working with composite images or image galleries, you can group the images to show the configurable item from multiple views or angles, such as front and back views. Users can browse through the images by clicking the arrows and switch between views by clicking the dots.

Each image has a sequence number that determines its order in the image container. In an image gallery, images with a lower sequence number are displayed before those with a higher number. In a composite image, images with a lower sequence number are placed closer to the background image. For example, you may assign 1 as a sequence number to the background image and 2, 3, 4 for layers applied on top of the background image. Layer 4 is displayed over the other layers.

All images should be the same size, especially for composite images. Using images with consistent dimensions ensures that each layer is correctly positioned in the image container. The system automatically resizes images to fit the width of the image container by scaling them up or down. The height of the images is dynamically adjusted to maintain the original proportions.

 **Note:** The image container is as wide as the information bar. For more information, see [Changing the Information Bar Width and Position](#).

You should upload the images to the Images folder in the File Cabinet at Documents > File Cabinet > Images. In the Images folder, you can create a folder for NetSuite CPQ Configurator and add a subfolder

for each product. By uploading images to the Images folder, images are available to the NetSuite CPQ SuiteCommerce Integration without login. For more information about uploading files to the File Cabinet, see the help topic [Uploading Files to the File Cabinet](#).

To add an image:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Click **Edit** next to the product you want to modify.
3. On the **Images** subtab, click **New CPQC Image**.
4. Enter a descriptive name for the image.
This name is displayed under the Images option in the Audit menu of the product interface. Screen readers also use this name as alternate text and read it aloud to describe the image.
5. Enter a sequence number to determine the order in which the image is displayed on the composite image or the image gallery.
You can use negative or positive numbers. For example, -2 or 5.
6. To choose the image, two options are available:
 - In the **Image** field, select an image from the File Cabinet.
 - In the **Image URL** field, enter the URL of the image.
7. To create a view, provide a view code. Assign the same code to all images that belong to the same view.
The sequence number determines the order of the images within the same view.
8. In the **Rule** field, provide the condition that determines whether the image is displayed on the product interface.
Enter question and answer codes separated by a forward slash. For example, A/1. Use operators to combine multiple conditions. For example, A/1&B/2.
To make an image always visible, enter the syntax */*. For example, you can use this syntax to always display the background image in a composite image. For more information, see [Activating or Hiding Building Blocks with Rules](#).
9. (Optional) To include an additional rule in the image rule, select a rule category. The image will be displayed only if both its rule and the category rule match answers selected by users. For more information, see [Working with Rule Categories](#).
10. (Optional) To add the image as a logo on a composite image, you can resize it and position it in the top-left corner of the image container.
Enter the size and the offsets of the logo as percentages. The active image with the greatest height—before any resizing or positioning—is used as the base to calculate the logo's size and offsets. Leave out the percentage symbol when entering the values.

 **Note:** An image is active if its rule matches the answers users select on the product interface.

To resize and position the logo, enter values in the following fields:

- **Height** – The logo height.
- **Width** – The logo width.
- **Offset from Left** – The space between the logo and the left border of the image container.
- **Offset from Top** – The space between the logo and the top border of the image container.

You can use resolve expressions to provide values for these fields. For more information, see [Obtaining Answer Data with Resolve Expressions](#).

You could enter the image height, width, and offsets in pixels by adding px after the number—for example, 10px. However, take into account that pixels don't maintain proportions on different screens. Using percentages ensures that the image remains proportional on any device.

11. Click **Save**.

To copy or delete images, see [Working with Product Building Blocks](#).

Creating Favorite And Default Configurations

With NetSuite CPQ Configurator, users with an administrator role and employees can save frequently used or unfinished configurations as favorites and reload whenever they need. By loading a saved set of features and options, favorites help save time and create accurate configurations.

Favorites are organized by product. To create a favorite configuration, go to the Favorites menu on the product interface and select **Create Favorite**. Provide a name for your favorite and, optionally, a short and a long description. Only the short description is visible to users when viewing the favorite list. When you're finished entering information, click the **Save** button. You can create as many favorite configurations as needed.

To view your list of favorite configurations, go to the Favorites menu and select **View Favorites**. You can click the **Load** button next to any favorite to reload a favorite configuration. To delete a favorite configuration from the list, click the **Delete** button.

Favorite Configurations for Administrators

Users with an administrator role can perform additional tasks related to favorites:

- **Allow employees to view favorite configurations created by any user** – Go to the **Access** subtab on the employee record. In the **CPQ Configurator Permissions** field, select **Advanced** and click the **Save** button. This option encourages knowledge sharing in your business. Users with basic permissions can view only their own favorites by default.
- **Create favorites from the product preview** – You can create favorites without opening products from transactions. To access the product preview, click the **Run Standalone** button on the product record. Then, create favorites following the usual steps.
- **Create public favorite configurations** – You can create predefined or commonly used configurations and make them available to all users from the Favorites menu. To create a public favorite configuration, check the **Public Favorite** box before saving your favorite. All users can view public favorite configurations, regardless of the option set in the **CPQ Configurator Permissions** field on their employee record.
- **Create a default configuration for each product** – A default configuration is a set of predefined features and options that automatically loads whenever users start a new configuration or click the **Submit & New** button. To create a default favorite configuration, check the **Default Favorite** box before saving your favorite.

The default favorite is automatically saved as a public favorite, and you can view it from the favorite list. You can create only one default favorite for each product. If you create a new default configuration, the replaced favorite remains available in the list as a public favorite.

 **Note:** The default favorite doesn't load when users edit an existing configuration or click the **Submit & Copy** button.

Changing the Product Interface Look and Feel

Depending on your business needs, you can change the following aspects of the product interface with NetSuite CPQ Configurator:

- **Information bar position and width** – You can define the position and the number of columns the information bar can occupy. For more information, see [Changing the Information Bar Width and Position](#).
- **Question organization** – Questions can be displayed horizontally or vertically. You can also organize the question space using horizontal lines or line breaks. For more information, see [Organizing the Question Space](#).
- **Tab line, image container, and group appearance** – Tabs can be displayed at the top pr on the left side of the question space. You can also set the space between tabs (in pixels), remove the space between the product name bar and the tab line, and remove borders from the image container and groups. For more information, see [Changing the Appearance of Tabs, Groups, and the Image Container](#).
- **Label and header colors** – You can define the color of the label and the label background for questions, tabs, and groups. For more information, see [Changing Label and Header Colors for Questions, Tabs, and Groups](#).

To change the product interface look and feel, go to CPQ > Configurator > Product Maintenance. Edit a product and set your preferences on the **Appearance** subtab. When you're finished, save the product record.

Changing the Information Bar Width and Position

You can think of the product interface as a 12-column grid. The following image shows how this grid is arranged on the product interface.

The diagram illustrates the layout of the product interface as a 12-column grid. The interface is divided into several sections:

- Top Header Bar:** Contains 'Subtotal: 0.00 Total: 0.00', 'Quantity: 1', and 'Cancel'/'Submit & Close' buttons.
- Product Name Section:** Contains the 'Product Name' label and 'Audit'/'Favorites' buttons.
- Tab Line:** A horizontal line separating the product name from the question space.
- Question Space:** A large area for displaying questions, occupying 8 columns.
- Image:** A section for displaying an image, occupying 4 columns.
- More information:** A section containing 'Validation Messages' and 'Summary'.

The information bar is placed to the right of the question space and occupies one-third of the product interface—4 out of the 12 columns.

To change the width and the position of the information bar, go to the **Appearance** subtab on the product record. In the **Information Bar Width** field, select a number between 1 and 11 to define the

columns the information bar should occupy. You should set the width to a maximum of 5 or 6 columns to leave enough space for questions.

To hide the information bar, select 0. To display the information bar below the question space, select 12. The information bar will occupy the full width of the product interface.

Organizing the Question Space

You can change the way questions are displayed on the product interface. Questions can be arranged in a single column (vertically) or multiple columns (horizontally). To define the question orientation, select your preferred option in the **Question Layout** field on the product record. If you choose a horizontal layout, the number of questions that can fit in a row is automatically determined based on their width.

You can set the width of each question in the **UI Columns** field on the question record. This option determines how many columns the question can occupy on the product interface. You can think of the question space as a 12-column grid. The following image shows how the grid is arranged on the question space.

When using the horizontal layout, you can add line breaks (formerly called breaks) and horizontal lines (formerly called separators) to the question space. Line breaks and horizontal lines help organize the question space and make questions and answers easier to read. Line breaks force the next question to start on a new line. You can use horizontal lines to separate groups of questions on the product interface.

To create a line break or a horizontal line:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Click **Edit** next to the product you want to modify.
3. On the **Questions** subtab, click **New CPQC Question**.
4. Provide a descriptive name for the line break or the horizontal line.

5. In the **Code** field, enter a unique identifier to use in rules and actions.
The code can contain only uppercase letters, numbers, and underscores. You can use a shortened version of the name as the code.
6. Provide a sequence number to determine the order in which the line break or the horizontal line is displayed on the product interface. Line breaks or horizontal lines with a lower sequence number are displayed higher on the product interface.
7. In the **Type** field, select **Line Break** or **Horizontal Line**.
8. (Optional) Select a tab or a tab and group combination where you want to display the line break or the horizontal line.
9. Click **Save**.

Changing the Appearance of Tabs, Groups, and the Image Container

You can change the position and the appearance of tabs, groups, and the image container. To do this open the product record and go to the **Appearance** subtab.

You can display the tab line at the top or on the left side of the question space. In the **Tab Line Position** field, select your preferred option. To remove the space between the tab line and the product name bar, check the **Remove space around the tab line** box. You also can define the space between tabs in pixels. Enter a number between 0 and 5 in the **Tab Spacing (px)** field.

The height and shape of the image container are determined by the active image with the greatest height before any resizing or positioning. The width of the image container matches the width of the information bar. To remove the borders around the image container, check the **Remove image borders** box.

To remove the borders around the groups, check the **Remove group borders** box.

Changing Label and Header Colors for Questions, Tabs, and Groups

You can change the label and label background colors for questions, tabs, and groups to match color scheme of your business. The colors you choose apply to all questions, tabs, and groups in the product interface.

You can select a color from the palette or enter the hexadecimal value—for example, #FFFFFF for white. To change colors, use the following fields:

- **Question Label Color** – Sets the color for question labels.
- **Question Header Color** – Sets the color of the background behind question labels.
- **Tab Label Color** – Sets the color for tab labels.
- **Tab Color** – Sets the color of the background behind tab labels.
- **Active Tab Color** – Sets the color for the selected tab header.
- **Group Label Color** – Sets the color for group labels.
- **Group Header Color** – Sets the color of the background behind group labels.

Translating Custom Labels for Product Interface Elements

You can translate custom labels of certain product interface elements to match the language of the NetSuite user interface and make the configuration process simpler and faster for users. These elements' labels can be translated:

- Questions
- Answers

- Validations
- Pricing
- Tabs
- Groups

You can add the translations under the **Translations** section or subtab on any of these records.

Prerequisites for Translating Custom Labels

Before translating the custom labels, complete these steps:

1. Enable the NetSuite Multi-Language feature to display the product interface in different languages. Go to Setup > Company > Setup Tasks > Enable Features. In the International section under the **Company** subtab, check the **Multi-Language** box and click **Save**.
2. Define the translation languages. Go to Setup > Company > Preferences > General Preferences. Under the **Languages** subtab, enable any system languages that NetSuite supports and click **Save**. For a list of available languages, see the help topic [Configuring Multiple Languages](#).
3. Enable the custom label translations for the product interface. Go to the options record at CPQ > Configurator > Settings. Then, check **Show label translations** box and click **Save**.

Adding Languages

Before translating custom labels, make sure the language you want to use is available in NetSuite CPQ Configurator. Go to the **Translations** section or subtab of any record that supports translations for custom labels. Under the **Language** column, click a field to view the list of available languages.

If the language isn't available, add any previously enabled NetSuite system-supported language. You can verify the enabled languages under the **Languages** subtab at Setup > Company > Preferences > General Preferences.

To add a language:

1. Go to the **Translations** section or subtab.
2. Under the **Language** column, click **New**.
3. Provide a name for the language.
4. Select a NetSuite system-supported language from the dropdown list.

 **Note:** The list only includes the NetSuite system-supported languages previously enabled under the **Languages** subtab at Setup > Company > Preferences > General Preferences.

5. Provide a code.

 **Note:** Codes follow the ISO 639 and ISO 3166 standards. Enter the language and country codes separated by an underscore, for example, en_US or fr_FR.

6. Click **Save**.

Adding Translations

After verifying that the language is available, you can add the translation.

To add a translation:

1. Go to the **Translations** section or subtab.
2. In the **Language** field, select your preferred language.
3. In the **Label Translation** field, enter the translation for the label.
4. Click **Add**.
5. Click **Save**.

Calculating the Final Pricing for the Configurable Item

The configurable item price can be dynamically calculated and updated based on the features and options users select to ensure consistent and accurate pricing. All active pricing records are combined to provide the final price for the configurable item. An active price has a rule that matches answers users select on the product interface.

Note: The price of item groups used as base items can't be dynamically calculated. Their price depends on the prices of their member items, and NetSuite CPQ Configurator doesn't handle those prices.

Pricing records have an internal name and a label for the product interface. You can use the sequence number to sort prices in the product interface summary. Prices with a lower sequence number are displayed higher in the summary. You can also apply multiplier and discount values to create a flexible price structure that adapts to changes in quantities and rates.

Note: Use a dot to add decimals to price, discount, and multiplier values.

Price amounts can be sourced from materials using the **Source Price from Material** field. Materials derive their price from the associated item record. For more information, see [Sourcing Price Information from Materials](#).

You can view pricing information on the product interface and use the **Subtotal** from the product interface as the base price. For more information, see [Viewing Price Amounts on the Product Interface](#) and [Using the Subtotal as the Base Price](#).

To create a price:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Click **Edit** next to the product you want to modify.
3. On the **Pricing** subtab, click **New CPQC Pricing**.
4. Enter a descriptive name for this price.
This name is displayed under Pricing in the Audit menu of the product interface.
5. Provide a sequence number to sort the prices in the summary.
You can enter a negative or positive number. For example, -2 or 5.
6. (Optional) To define a specific date range during which this price is included in the calculations for the final item price and displayed on the product interface:
 - In the **Effective Start Date** field, select a start date.
Before this date, the price is inactive.
 - (Optional) In the **Effective End Date** field, select an end date.
After this date, the price is inactive.

Note: If you define a date range, the price must meet both the date range and the rule conditions to be active.

7. In the **Label** field, enter the price label displayed in the summary and in the Audit > Pricing menu on the product interface. You can format the text by changing the paragraph font and size or by making the text bold, italic, or underlined.

Note: You can also include information from answers users select on the interface by using resolve expressions. Enter question and answer codes separated by the forward slash character and enclosed in curly braces. For example, {A/1}. Question and answer codes are replaced with the corresponding answer value. For more information, see [Obtaining Answer Data with Resolve Expressions](#).

8. To provide the price amount, two options are available:
 - In the **Price** field, enter a number without the currency symbol. You can also obtain the price value using resolve expressions.
 - In the **Source Price from Material**, select the material from which you want to source the price amount. For more information, see [Sourcing Price Information from Materials](#).

Regardless of the option you choose, the price amount can be multiplied by the multiplier and the discount.

The currency depends on the one assigned to the customer selected on the transaction.

9. In the **Discount** field, enter a number or a resolve expression to apply a reduction to the usual price. For example, 0.5.
If you leave this field blank, the default value is 1.
10. In the **Price Multiplier** field, you can enter a number or a resolve expression. The price is multiplied by this value.
For example, you can use a resolve expression to multiply the price by the quantity selected by users in an answer. If you leave this field blank, the default value is 1.
11. In the **Rule** field, provide the condition that determines whether the price is added to the summary and the **Subtotal** field of the top bar.
Enter question and answer codes separated by the forward slash character—for example, A/1. Use operators to combine multiple question and answer sets—for example, A/1&B/2. For more information, see [Activating or Hiding Building Blocks with Rules](#).
If you source the price from a material, the material rule is applied to the pricing record through the **Material Rule** field.
12. (Optional) To include an additional rule in the pricing rule, select a rule category. The price is active if its rule and the rule of the category match an answer selected by users. For more information, see [Working with Rule Categories](#).
13. To include this price in the **Total** field on the product interface top bar one time only, check the **One time charge** box.
For example, you can use this option for shipping charges.
14. To hide the price amount in the summary, check the **Hide price amount** box. The price label is still displayed on the product interface, and the amount is included in all pricing calculations.
15. To hide both the price label and amount in the summary, check the **Hide price line** box. The price amount is still included in all pricing calculations.
16. Click **Save**.

To copy or delete prices, see [Working with Product Building Blocks](#).

Sourcing Price Information from Materials

Note: Prices sourced from materials aren't available when opening a product in preview mode by:

- Clicking the **Run Standalone** button on the product record.
- Accessing the **Configurable Items** page at CPQ > Configurator > Standalone.

Price amounts can be sourced from materials using the **Source Price from Material** field. Materials derive their price from the associated item record. If you use the Multiple Prices and the Multiple Currencies features, item records include price levels organized by currency where you can specify price amounts for each level. If the item record include multiple price levels and currencies, the price level and currency for the material depends on those assigned to the customer specified on the transaction. For more information, see the help topics [Setting Up Item Pricing](#) and [Using Multiple Pricing](#).

If you use the Multiple Quantity feature, also the quantity ranges specified on the item record are taken into account for the material price. For more information, see the help topic [Using Quantity Pricing](#).

Note: When prices change based on the quantity you sell for a material, the **Subtotal** amount for a single configured item may change when users increase or decrease the quantity on the product interface.

Viewing Price Amounts on the Product Interface

On the product interface, all active prices are displayed in the summary with their amount—unless you hide a particular price amount or an entire price line. Users can find more information about active prices under the **Pricing** option in the Audit menu. The price for a single configured item is displayed in the **Subtotal** field in both the summary and the top bar.

After users submit the configuration, the transaction receives the values of the **Subtotal** and **Total** fields from the top bar. The **Total** field takes into account the quantity selected for the configured item. For more information about pricing data on the product interface, see [Working with the Product User Interface](#).

Using the Subtotal as the Base Price

You can dynamically obtain the price for a single configured item from the **Subtotal** field in top bar of the product interface. This subtotal amount can be used in a pricing record as the base for other calculations.

For example, you want the shipping charges to correspond to a certain percentage of the price of a single configured item. To create a pricing record that depends on the subtotal, check the **Calculate subtotal** box. After enabling this option, use the subtotal in the **Price** field by entering the {SOLE/MAT_PRICE} resolve expression. Then, you can multiply the subtotal by the multiplier, the discount, or both and follow the basic steps to create a price.

Including Additional Items Related to the Configurable Item

You can add items related to your main configurable item to transactions. Additional items (formerly called breakout items) complete the main item and are usually used or sold together. For example, if your

configurable item is a desk, you may have a lamp or a desk mat as additional items. Depending on your business needs, you can also create products that add only additional items to transactions, without a main configurable item.

 **Note:** Additional items must be active items. An item is active when the **Inactive** box isn't checked. This option is placed under the **System Information** subtab on the item record.

You can specify an additional item using one of the following options:

- An existing item in NetSuite
- A resolve expression
- An item creation record

If you use a resolve expression, the item name or number is obtained from answers users select on the product interface. With a resolve expression, you can retrieve different items depending on users' choices with a single additional item record. For more information about resolve expressions, see [Obtaining Answer Data with Resolve Expressions](#).

If, on the other hand, you use an item creation record, the item will be automatically created when users submit the configuration. For more information, see [Creating Items Required by the Configuration](#).

An additional item is included in configurations—and later in transactions—if its rule matches answers that users select on the product interface. For more information about rules, see [Activating or Hiding Building Blocks with Rules](#).

You can view active additional items under Additional Items in the Audit menu on the product interface. After submitting the configuration, active additional items are assigned dedicated transaction lines and are listed below the main configured item.

Additional items are priced separately from the configurable item. For more information, see [Working with the Price of Additional Items](#).

Different additional items can be linked to the same item record. When submitting the configuration to transactions, you can consolidate the corresponding additional item lines into a single line. For more information, see [Consolidating Additional Item Lines on Transactions](#).

When including additional items in transactions, you can set line fields using mapping records. For more information, see [Setting Transaction Line Fields for Additional Items](#).

To create an additional item:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Click **Edit** next to the product you want to modify.
3. On the **Additional Items** subtab, click **New CPQC Additional Item**.
4. Enter a descriptive name for the additional item.
This name is displayed under Additional Items in the Audit menu of the product interface. The name that appears on transactions belongs to the item record.
5. (Optional) In the **Description** field, enter a brief explanation of the additional item.
The description is displayed under Additional Items in the Audit menu and on the transaction line.
6. Provide a sequence number to sort the additional item on the transaction lines.
Additional items are sorted from lowest to highest sequence number. You can enter a negative or positive number—for example, -2 or 5.

7. (Optional) To define a specific date range in which this additional item is active and included in configurations and later in transactions:

- In the **Effective Start Date** field, select a start date.
Before this date, the additional item is inactive.
- (Optional) In the **Effective End Date** field, select an end date.
After this date, the additional item is inactive.

 **Note:** If you define a date range, the additional item must meet both the date range and the rule conditions to be active.

8. To define the additional item, you have three options:

- In the **Item Creation Record** field, select the item creation record that stores the information to create the additional item.
The additional item is created if the rules for both the item creation record and the additional item match the answers that users select on the product interface. For more information, see [Creating Items Required by the Configuration](#).
- In the **Item** field, select an item from NetSuite.
If you use this option, the **Item Display Image** and the **Average Cost** fields are automatically completed with information from the item record (if available).
- In the **Resolve Item Name/Number**, enter a resolve expression to obtain the item name or number of the additional item from answers on the product interface.
To write the resolve expression, use question and answer codes separated by the forward slash character and enclosed in curly braces—for example, {A/1}. Question and answer codes are replaced with the corresponding answer value. For more information, see [Obtaining Answer Data with Resolve Expressions](#).

 **Note:** If you fill in more than one field to specify the additional item, only one will be used. Priority is given to the field for the item creation record, then the existing NetSuite item, and finally, the resolve expression if others are empty.

9. In the **QTY** field, enter a number or a resolve expression to define how many units of this additional item the configuration requires.
10. In the **Price** field, enter a number without the currency symbol, or a resolve expression. This value replaces the price from the item or the item creation records.

 **Note:** Use a dot to add decimals to the price value.

11. In the **Price Multiplier** field, you can enter a number or a resolve expression for a more flexible price structure. For example, you can use a resolve expression to multiply the price by the quantity users select in an answer.
12. (Optional) In the **Line Field Mapping Records** field, select the mapping records you want to use to set line fields for the additional item when it's added to transactions. For more information about mapping records, see [Setting Transaction Line Fields for Additional Items](#) and [Setting Transaction Body and Line Fields Based on Configuration Data](#).
13. In the **Rule** field, provide the condition that determines whether the additional item is included in the transaction. The condition is based on answers users select on the product interface.
Enter question and answer codes separated by a forward slash character—for example, A/1. Use operators to combine multiple conditions—for example, A/1&B/2.
14. (Optional) To add an extra rule to the additional item rule, select a rule category. If both the additional item and the category rules match the answers that users select on the product

interface, the additional item is included in the configuration and later in transactions. For more information, see [Working with Rule Categories](#).

15. Click **Save**.

To copy or delete additional items, see [Working with Product Building Blocks](#).

Working with the Price of Additional Items

Price calculations for the main configurable item exclude additional item prices. The price of each additional item is displayed only on transaction lines by default. However, you can also choose to display additional item prices in the Additional Items section of the Summary on the product interface.

By displaying additional item prices in the Summary, users can make more informed choices about which additional items to include based on their price and review all pricing details before adding the configured item to the transaction.

To display these prices on the product interface, check the **Show additional item prices in Summary** box on the product record.

Consolidating Additional Item Lines on Transactions

When submitting configurations to transactions, you can consolidate multiple additional item lines that reference the same item record (same internal ID) into a single line item. To apply the options for consolidating additional items, go to the product record during the setup and edit it. Three options are available:

- **Consolidate additional item lines** – Combines additional item lines with the same internal ID into one.

When additional item lines are combined, the quantity and the price of each additional item are summed. To consolidate additional item lines, check the box for this option.

- **Calculate rate for consolidated additional item lines** – Determines the rate for the consolidated line. If additional items linked to the same item record have different prices, the rate field on the transaction will display the average price. Take into account that the average may be a decimal number with multiple places—for example, 19.999999. To calculate the rate, check the box for this option.

If you use the price of the item record and multiple price levels are enabled in your account, the price level on the consolidated item line will be labeled as **Custom**. For more information about multiple prices and currencies in NetSuite, see the help topics [Using Multiple Prices or Currencies](#) and [Using Multiple Pricing](#).

- **Merge mappings for consolidated additional item lines** – Merges any mapping records from individual additional items during the consolidation. Individual mapping records are those listed in the **Line Field Mapping Records** field on the additional item record. If multiple mapping records include the same field, the value will be set based on the mapping record linked to the additional item with the higher sequence number. To merge mapping records, check the box for this option.

Setting Transaction Line Fields for Additional Items

When including additional items in transactions, you can set line fields by selecting mapping records in the **Line Field Mapping Records** field during the setup. In the **Common Line Field Mapping Records** field, you can view the mapping records that are automatically applied to all additional items. These mapping records have the **Apply to all additional items** box checked. For more information about

setting transaction line fields for additional items, see [Setting Transaction Body and Line Fields Based on Configuration Data](#).

Note: Before mapping records, you could set transaction line fields using the **Order Column Mapping** field. If you want to switch from this field to mapping records, move the field information to mapping records and clear the field. After clearing this field, it will become hidden.

Creating Work Orders for Configured Items

Note: Install NetSuite CPQ Manufacturing to create work orders using materials and routing steps from configured items. For more information, see the help topic [Installing NetSuite CPQ SuiteApps](#). Your account also needs manufacturing capabilities to install NetSuite CPQ Manufacturing. Contact your NetSuite representative for more information.

With NetSuite CPQ Manufacturing, you can create work orders for configured items that use an assembly as the base item (formerly called the base generic product). When creating work orders from sales orders, NetSuite CPQ Manufacturing uses materials and routing steps from configured items added to the sales order by NetSuite CPQ Configurator. For more information about work orders in NetSuite, see the help topic [Assembly Work Orders](#).

To create work orders for configured items, follow these steps:

1. Implement prerequisites. For more information, see [Implementing Prerequisites](#).
2. Create work orders from sales orders by clicking the **Convert WO** button. Alternatively, automatically create work orders based on custom conditions using a script.
For more information, see [Creating Work Orders from Sales Orders](#) and [Creating Work Orders for Configured Items Using Scripts](#).
3. Verify that work orders are correctly created. For more information, see [Testing and Debugging Work Orders for Configured Items](#).

Implementing Prerequisites

Before creating work orders for configured items, follow these steps:

1. Always maintain enabled the **Build WO** field (field ID: custcol_cpqm_item_col_buildwo) on sales orders. During the installation, NetSuite CPQ Manufacturing automatically creates and enables this custom line field on sales orders to let users create work orders.
2. Create an assembly to use as the base item by following the specific requirements. For more information, see [Creating Assemblies to Use as Base Items](#).

Note: If you created the assembly base item through an item creation record, you must create the work order following the standard NetSuite process. For more information, see the help topic [Creating Work Orders From a Sales Orders](#).

3. Assign the assembly as the base item to your NetSuite CPQ Configurator product. For more information, see [Assigning Base Items to Products](#).
4. In NetSuite CPQ Manufacturing, enable the **Convert WO** button for sales orders. For more information, see [Enabling the Convert WO Button](#).
5. Create materials and routing steps to build the configured item with NetSuite CPQ Configurator. Items used as materials must have the same subsidiary as the sales order. For more information, see [Defining Materials for the Configurable Item](#) and [Defining Routing Steps to Build the Configurable Item](#).

Creating Work Orders from Sales Orders

After submitting the configuration, the configured item is added under the **Items** subtab on the sales order.

After saving the sales order, users can edit it and click the **Convert WO** button to create a work order. This button is available at the top and bottom of the sales order, as well as on each configured line item. If users click the **Convert WO** button at the top or bottom, a work order is created for each configured item on the transaction. If users click the button on the configured line item, a work order is created only for that specific configured item.

When converting configured line items to work orders for an entire sales order:

- Configured line items without work orders are converted.
- Configured line items with up-to-date work orders are disregarded.
- Outdated work orders are removed, and configured line items with outdated work orders are converted again.

 **Note:** A work order becomes outdated when users edit and save the related configuration.

After creating the work order, users can click the work order number link in the **Create WO** line field to view its details. In the work order, materials are under the **Items** subtab and routing steps are under the **Operations** subtab.

Creating Assemblies to Use as Base Items

If you want to create work orders for configured items, assemblies used as the base item (formerly called the base generic product) must meet specific criteria. When creating the assembly, complete the following tasks to comply with the requirements:

1. If you use an advanced bill of materials, create a bill of materials and a revision with a placeholder component. Enter only the basic information required to save them. NetSuite CPQ Manufacturing uses this bill of materials and this revision as placeholders only. For more information, see the help topics [Creating a New BOM Record](#) and [Creating BOM Revisions](#).

If you don't use an advanced bill of materials, add the placeholder component and the basic information required to save the assembly.

 **Note:** The assembly and its placeholder component must have the same subsidiary. Make sure you let users know the subsidiary used because they must use the same subsidiary when creating the sales order.

2. Create a placeholder manufacturing routing, check the **Default** box, and add the basic information required to save it. For more information, see the help topic [Creating a Manufacturing Routing](#).
3. On the assembly, check the **WIP** box for the location selected on the sales order. Considering this is a placeholder assembly, you can mark all the available locations as **WIP**.

Enabling the Convert WO Button

Users can create work orders by editing a sales order and clicking the **Convert WO** button. To enable the **Convert WO** button on sales orders, go to CPQ > Manufacturing > Settings. Check at least one of these options:

- **Top/Bottom** – Displays the **Convert WO** button at the top and bottom of sales orders. If users click one of these buttons, a work order is created for each configured line item on the transaction.
- **Line Item** – Displays the **Convert WO** button on the configured line item. If users click this button, a work order is created only for that specific configured line item.

Defining Materials for the Configurable Item

Note: Install NetSuite CPQ Manufacturing to use materials from configured items in work orders. For more information, see the help topic [Installing NetSuite CPQ SuiteApps](#). Without NetSuite CPQ Manufacturing, you can still use materials to set up components for item groups and kit/packages.

Configurable items can be made of materials or components. For example, if your configurable item is a desk, you may have the top surface and side panels as materials. With NetSuite CPQ Configurator, you can specify a material using one of these options:

- an existing item in NetSuite
- a resolve expression
- an item creation record

If you use a resolve expression, the item number or number is obtained from answers users select on the interface. With a resolve expression, you can retrieve different items based on users' choices using a single material record. For more information, see [Obtaining Answer Data with Resolve Expressions](#).

If, on the other hand, you use an item creation record, the material will be automatically created when users submit the configuration. For more information, see [Creating Items Required by the Configuration](#).

You can use pricing information from the item associated with the material using the **Source Price from Material** field on the pricing record. For more information, see [Calculating the Final Pricing for the Configurable Item](#).

A material is added to configurations—and later to work orders—if its rule matches the answers users select on the product interface. For more information, see [Activating or Hiding Building Blocks with Rules](#). You can view active materials under the **Materials** option on the Audit menu on the product interface.

When users click the **Convert WO** button on the sales order or the configured item line, NetSuite CPQ Manufacturing adds each material as a line item on the work order under the **Items** subtab. For more information, see [Creating Work Orders for Configured Items](#).

When adding a material as a component to the work order, you can set line fields by selecting mapping records in the **Line Field Mapping Records** field during the setup. In the **Common Line Field Mapping Records** field, you can view the mapping records automatically applied to all materials. These mapping records have the **Apply to all materials** box checked. For more information about mapping records, see [Setting Transaction Body and Line Fields Based on Configuration Data](#).

Note: Before mapping records were introduced, you could set work order line fields using the **WO Column Mapping** field. If you want to switch to mapping records, move the field information to mapping records and clear the **WO Column Mapping** field. After clearing the field, it will become hidden.

To create a material:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.

2. Click **Edit** next to the product you want to modify.
3. On the **Materials** subtab, click **New CPQC Material**.
4. Enter a descriptive name for the material.

This name is displayed under **Materials** in the Audit menu on the product interface.

5. (Optional) Edit the description.

This field sources its content from the **Sales Description** field on the item record. However, you can change the content to adapt it to your business needs.

 **Note:** If you change the item used as a material:

1. The description will update with the content from the **Sales Description** field on the new item record.
2. Any manually added content will be overwritten.

6. Assign a routing step to specify the task in which this material will be used.
7. (Optional) To define a date range in which the material is active and added to configurations and later to work orders:
 - In the **Effective Start Date** field, select a start date.
Before this date, the material is inactive.
 - (Optional) In the **Effective End Date** field, select an end date.
After this date, the material is inactive.

 **Note:** If you define a date range, the material must meet both the date range and the conditions in its rule to be active.

8. To specify the material, three options are available:
 - In the **Item Creation Record** field, select the item creation record that stores the information to create the material.

 **Note:** The material is created if the rules of both the item creation record and the material match the answers users select on the product interface. For more information, see [Creating Items Required by the Configuration](#).

- In the **Item** field, select an item from NetSuite.
When you select this option, a series of fields is automatically completed (if the information is available on the selected item record): **Item Display Image**, **Description**, **Primary Sale Unit**, **Average Cost**, and **Purchase Price**.
- In the **Resolve Item Name/Number**, enter a resolve expression to obtain the item name or number from answers on the product interface.
To write a resolve expression, use question and answer codes separated by a forward slash and enclosed in curly braces. For example, {A/1}. Question and answer codes are used to retrieve the corresponding answer value. For more information, see [Obtaining Answer Data with Resolve Expressions](#).

 **Note:** If you fill in multiple fields to specify the material, only one field will be used. Priority goes to the field for the item creation record, then the existing NetSuite item, and finally, the resolve expression.

9. In the **Quantity** field, enter a number or a resolve expression to define how many units of this material are required by the configuration.

The quantity value is multiplied by the multiplier. On the work order, you can view the result in the **BOM Quantity** column on the **Items** subtab. The **BOM Quantity** field indicates the component quantity used to build an item, assuming no loss. For more information, see the help topic [Setting the Use Component Yield Preference](#).

10. For more flexibility in defining the quantity, enter a number or a resolve expression in the **Quantity Multiplier** field.

If you leave this field blank, the assumed value is 1.

11. (Optional) In the **Component Yield** field, enter the percentage of the material or component that survives the manufacturing process. Leave out the percentage symbol.

This value is displayed under the corresponding column in the work order. For more information, see the help topic [Component Yield Examples](#).

12. (Optional) In the **Primary Units Type** field, enter the internal ID of the unit type you use to purchase, stock, and sell this material. Examples of unit types are length, volume, and time.

This value is displayed in the **Units** column on the work order.

To use this field, enable the multiple units of measure feature. For more information about using multiple units of measure, see the help topic [Multiple Units of Measure](#).

13. If the material is a phantom assembly item to be built by assembling components in the account, check the **Phantom Material** box. If you check this box, the **Mark sub-assemblies phantom** box on the work order becomes checked. Use this option when building subassemblies of the main assembly item. For more information about phantom assemblies, see the help topic [Phantom Assemblies](#).

14. (Optional) In the **Line Field Mapping Records** field, select the mapping records you want to use to set line fields for the material when it's added as a component to the work order. For more information about mapping records, see [Setting Transaction Body and Line Fields Based on Configuration Data](#).

15. In the **Rule** field, provide the condition that determines whether the material is added to the configuration and later to the work order.

Enter question and answer codes separated by a forward slash, for example, A/1. Use operators to combine multiple conditions, for example, A/1&B/2. For more information, see [Activating or Hiding Building Blocks with Rules](#).

16. (Optional) To include an additional rule in the material rule, select a rule category. If both the material and the category rules match the answers users select on the interface, the material is added to the configuration and later to the work order. For more information, see [Working with Rule Categories](#).

17. Click **Save**.

To copy or delete materials, see [Working with Product Building Blocks](#).

Defining Routing Steps to Build the Configurable Item



Note: Install NetSuite CPQ Manufacturing to use routing steps from configured items in work orders. For more information, see the help topic [Installing NetSuite CPQ SuiteApps](#).

Routing steps are the tasks required to build a configurable item. If your configurable item is a desk, examples of routing steps may be cutting and polishing the wood for the top surface and the side panels.

To include a routing step in a work order, assign the routing step to a material through the **Routing Step** field on the material record. For more information, see [Defining Materials for the Configurable Item](#).

A routing step is added to configurations and later to work orders if its rule matches the answers users select on the product interface. For more information, see [Activating or Hiding Building Blocks with Rules](#). You can view active routing steps under the **Routing Steps** option in the Audit menu on the product interface.

When users click the **Convert WO** button on the sales order, routing steps are added under the **Operations** subtab of the work order. Each routing step becomes a manufacturing operation task, which is a subrecord of the work order. For more information about creating work orders, see [Creating Work Orders for Configured Items](#).

When adding a routing step as an operation task to the work order, you can set fields for the routing step by selecting mapping records in the **Line Field Mapping Records** field during the setup. In the **Common Line Field Mapping Records** field, you can view the mapping records automatically applied to all routing steps. These mapping records have the **Apply to all routing steps** box checked. For more information about mapping records, see [Setting Transaction Body and Line Fields Based on Configuration Data](#).

To create a routing step:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
 2. Click **Edit** next to the product you want to modify.
 3. On the **Routing Steps** subtab, click **New CPQC Routing Step**.
 4. Enter a descriptive name for this routing step.
This name is displayed under the **Routing Steps** option in the Audit menu on the product interface. It's also used as the operation name under the **Operations** subtab of the work order.
 5. In the **Code** field, enter a unique identifier for this routing step.
You can use a shortened version of the name as the code. The code can contain only uppercase letters, numbers, and underscores.
 6. (Optional) In the **Description** field, enter a brief explanation of this routing step.
This description is used for internal reference only.
 7. Complete the basic fields for the manufacturing operation task record:
 - The **Comments** field. Its content is displayed on the product interface under Audit > Routing Steps.
 - The fields in the Operation Properties section: **Operation Sequence**, **Setup Time (min)**, **Run Rate (min/unit)**, **Manufacturing Work Center**, **Manufacturing Cost Template**. For more information, see the help topic [Manufacturing Operation Tasks](#).
To obtain information from the configuration, you can enter resolve expressions in the **Comments**, **Setup Time (min)** and **Run Rate (min/unit)** fields. For more information, see [Obtaining Answer Data with Resolve Expressions](#).
 8. (Optional) To define a specific date range in which this routing step is active and can be added to the configuration and later to the work order:
 - In the **Effective Start Date** field, select a start date.
Before this date, the routing step is inactive.
 - (Optional) In the **Effective End Date** field, select an end date.
After this date, the routing step is inactive.
-  **Note:** If you define a date range, the routing step must meet both the date range and the conditions in its rule to be active.
9. (Optional) In the **Line Field Mapping Records** field, select the mapping records you want to use to set fields for the routing step when it's added as an operation task to the work order. For

more information about mapping records, see [Setting Transaction Body and Line Fields Based on Configuration Data](#).

10. In the **Rule** field, provide the condition that determines whether this routing step is added to the configuration and later to the work order.

Enter question and answer codes separated by a forward slash, for example, A/1. Use operators to combine multiple conditions, for example, A/1&B/2.

11. Click **Save**.

To copy or delete routing steps, see [Working with Product Building Blocks](#).

Creating Work Orders for Configured Items Using Scripts

Note: If your sales orders contain a high number of configured items to convert to work orders, switch from the scheduled script to the **CPQM-MR-WOC** map/reduce script. Don't use the scheduled script in new implementations.

NetSuite CPQ Manufacturing lets you create work orders for configured items using scripts. In this case, the **CPQM-MR-WOC** map/reduce script (customscript_cpqm_mr_woc) handles the work order creation process. For more information about map/reduce scripts, see the help topic [SuiteScript 2.x Map/Reduce Script Type](#).

To create work orders for sales orders using a script:

1. In SuiteScript 2.x, create one of these server-side script types to launch the **CPQM-MR-WOC** map/reduce script:
 - Suitelet
 - RESTlet
 - User event
 - Scheduled

Note: Because map/reduce scripts were introduced with SuiteScript 2.0, the triggering script must be written in SuiteScript 2.x.

2. Launch the **CPQM-MR-WOC** script from the created server-side script. As an example, see [Launching the Work Order Creation Process from a User Event Script](#).
3. Pass the internal ID of the sales order to the custscript_cpqm_tranid parameter.
4. To update existing work orders, use the custscript_cpqm_force parameter and set it to true. For more information, see [Converting Up-To-Date Work Orders](#).

For more information about the **CPQM-MR-WOC** map/reduce script, see [Work Order Creation Script](#).

Launching the Work Order Creation Process from a User Event Script

A common use case is launching the work order creation process from a user event script when users or other scripts save or edit the sales order. When working with user event scripts, launch the work order creation on the afterSubmit event. Work orders can only be created after the sales order has been saved. For more information about user event scripts, see the help topic [SuiteScript 2.x User Event Script Type](#).

To create the user event script:

1. Pass the internal ID of the sales order to the `custscript_cpqm_tranid` parameter.
2. The submit event type can be create, edit or both. If you want to:
 - Create work orders only for new sales orders, run the script for the create event only.
 - Create work orders after editing and resaving a sales order, run the script for both the create and edit events.
3. To define when work orders are created, set up a custom condition. Use a custom function that verifies sales order fields such as status, customer, and so on.

Example

This user event script launches the **CPQM-MR-WOC** script to create work orders for configured items. The script runs under these conditions:

1. When creating or editing sales orders (create and edit events).
2. When the **Memo** field on the sales order contains **auto-create-wo**.

```

1  /**
2   * @ApiVersion 2.1
3   * @NScriptType UserEventScript
4   */
5  define(['N/task', 'N/runtime'], function(_nTask, _nRuntime) {
6
7      function afterSubmit(context) {
8          const type = context.type,
9                record = context.newRecord,
10               recId = record?.id;
11          log.debug({ title: 'afterSubmit (${type})', details: recId });
12          if (allowedEventType(type) && meetLaunchCriteria(record)) {
13              try {
14                  log.debug({ title: 'Scheduling WO creation for SO', details: recId });
15                  const scriptTask = _nTask.create({
16                      taskType: _nTask.TaskType.MAP_REDUCE,
17                      scriptId: 'customscript_cpqm_mr_woc',
18                      params: {
19                          custscript_cpqm_tranid: recId // Pass the sales order ID
20                      }
21                  });
22                  const taskId = scriptTask.submit();
23                  log.debug({ title: 'WO creation scheduled', details: 'Task ID: ' + taskId });
24              } catch (e) {
25                  log.error({ title: 'Error scheduling WO creation', details: e.message });
26              }
27          }
28      }
29
30      // Work orders are created for new and edited sales orders
31      function allowedEventType(type) {
32          return type == 'create' || type == 'update';
33      }
34
35      // Custom condition to define whether work orders are created for the sales order
36      function meetLaunchCriteria(record) {
37          return record?.getValue('memo') === 'auto-create-wo';
38      }
39
40      return {
41          afterSubmit: afterSubmit
42      };
43  });

```

Work Order Creation Script

After a server-side script, launches it, the **CPQM-MR-WOC** map/reduce script performs these tasks:

1. Loads the sales order.
2. Makes a list of configured line items to process.

 **Note:** This list excludes configured line items with up-to-date work orders.

3. Creates a work order for each:
 - Configured line item without a work order.
 - Configured line item with an outdated work order. Outdated work orders are deleted. A work order becomes outdated when users edit and save the corresponding configuration.
4. Updates the sales order by marking configured line items as **converted** and saves it.
When users submit the configuration, the **Config status** field (custcol_cpqc_item_cfg_status) has **new** as a value. After creating the work order, the value changes to **converted**.
5. If the update event launches the map/reduced script, the script runs a second time. However, no new work order will be created because all configured line items already have up-to-date work orders.

Converting Up-To-Date Work Orders

The **CPQM-MR-WOC** map/reduce script can reconvert configured line items with up-to-date work orders. To reconvert up-to-date work orders, the server-side script that launches the **CPQM-MR-WOC** script must:

1. Include the `custscript_cpqm_force` parameter.
2. Set the `custscript_cpqm_force` parameter to true.

See the following example to force the conversion:

```
1 const scriptTask = _nTask.create({
2   taskType: _nTask.TaskType.MAP_REDUCE,
3   scriptId: 'customscript_cpqm_mr_woc',
4   params: {
5     custscript_cpqm_tranid: recId,
6     custscript_cpqm_force: true,
7   }
8 });
```

By setting this parameter to true:

- All configured line items will be converted to work orders—including those with up-to-date work orders. Existing work orders will be deleted.
- When forcing the work order conversion, the **CPQM-MR-WOC** script will run one time only and won't update the sales order. Because work orders are separate records, they'll be saved, and you'll see the link to the work order in the **Create WO** line field on the sales order.

Solving Possible Issues

If you experience issues when creating work orders using scripts:

- If work orders aren't created, verify the logs of your server-side script to make sure the custom criteria was met and the **CPQM-MR-WOC** map/reduce script was launched.
- If work orders aren't created or are created with errors, verify the logs of the **CPQM-MR-WOC** map/reduce script.

To view script logs, go to Customization > Scripting > Scripts. Open the script record and click the **Execution Log** subtab. To delete logs, click the **Remove All** button.

Testing and Debugging Work Orders for Configured Items

When setting up products, you can test and debug the work order creation process for configured items. With the debugger tool in NetSuite CPQ Manufacturing, you can identify and fix potential issues and eliminate the need for manual, repetitive actions like opening and saving sales orders. You can use the debugger tool to test both the standard work order creation process and work order creation using a script.

Based on the script type that can launch the work order creation process, four modes are available for testing and debugging:

- **Client script mode** – Launches the work order creation process from a client script. This mode is the same as clicking the **Convert WO** button on the sales order. You can view detailed logs directly in the console. Use this mode to ensure that materials and routing steps are properly set up to create work orders.
- **Suitelet script mode** – Launches the work order creation process from a Suitelet. The console only provides basic information about whether the work order process has started and completed. You can find detailed information in the logs of the **CPQM-ST-WOC** script. This mode helps identify issues when testing is successful in client mode but fails in map/reduce mode.
- **Scheduled script mode** – Launches the work order creation process from a scheduled script. Use this mode to test sales orders with a limited number of configured line items to be converted. The console only provides basic information about whether the process has started. You can find detailed information in the logs of the **CPQM-SC-CWO** script.

 **Note:** If your sales orders contain a high number of configured items to be converted to work orders, switch from the scheduled script to the **CPQM-MR-WOC** map/reduce script. Don't use the scheduled script in new implementations.

- **Map/reduce script mode** – Launches the work order creation process from a map/reduce script. Use this mode to test sales orders with a large amount of configured line items to convert. The console only provides basic information about whether the process has started. You can find detailed information in the logs of the **CPQM-MR-WOC** script.

Additionally, if you check the **Force conversion** box, up-to-date work orders are converted again. This option:

1. Deletes up-to-date work orders for the selected sales order.
2. Converts configured line items with up-to-date work orders.

After testing the work order creation process, make sure you remove the created work orders before continuing with further testing.

To create a testing task to create work orders from a sales order:

1. Go to CPQ > Manufacturing > Work Order Creator.
2. Click **New CPQM WO Creator**.
3. Enter a name for the testing task.
4. Select the sales order for which you want to create work orders.
5. Click **Save**.
6. Click **Edit** to start testing the work order creation process.

7. (Map/reduce script mode only) If you plan to run the testing task in map/reduce script mode, you can check the **Force conversion** box. This option reconverts up-to-date work orders.
8. Depending on the testing mode you want to use, click one of the following buttons to start the testing task:
 - **Client**
 - **Suitelet**
 - **Scheduled**
 - **Map/Reduce**
9. When the testing process has completed:
 1. Open the sales order from the testing task using the open icon next to the **Order** field.
 2. On the sales order, view the **Create WO** line field. If the field contains a link with a number, the work order has been created.
10. If you experience issues when creating work orders, verify the script logs.
To view script logs for server-side scripts, go to Customization > Scripting > Scripts. Open the script record and go to the **Execution Log** subtab. To delete logs, click the **Remove All** button under this subtab.
11. After verifying the work orders and the logs, go back to the testing task and delete the created work orders by clicking the **Remove WOs** button.
You can view logs about the deletion process in the console.

Creating Items Required by the Configuration

NetSuite CPQ Configurator can automatically create items required by the configuration using information from item creation records.

Item creation records store data for the potential new item to be created, such as the item name or number, the item type, and the rule that determines whether it will be created. With resolve expressions, you can use information from the product interface to set field values for the new item. For more information, see [Obtaining Answer Data with Resolve Expressions](#).

With item creation records, you no longer need to manually create every possible item that may be required by a configuration— whether it's a material item, additional item (formerly called a breakout item), or base item (formerly called base generic product). NetSuite CPQ Configurator will create these items for you based on users' choices during the configuration process. Using resolve expressions, you can create a different item from the same item creation record every time users submit a configuration.

Additionally, item creation records reduce manual work by letting you copy information from existing items and set only the fields you want to change for the new items to be created. You can also update existing items with new information from item creation records.

If the item creation rule matches the answers users selected on the product interface, the new item is created after users submit the configuration. If the new items will be used as materials or additional items, the rule of the linked material or additional item must also match the answers selected by users. After submitting the configuration, users can monitor the items while they are being created by following a progress bar on the product interface.

To work with item creation records, see [Working with Item Creation Records](#).

After saving an item creation record, you can assign it to a material, an additional item, or a product. In the configuration, these new items will become materials, additional items, or the base item, respectively. For more information, see [Assigning Item Creation Records to Materials, Additional Items, or Products](#).

Note: If your account has more than 300 items, new items are automatically added to transaction fields. If your account has fewer than 300 items and you send data directly to the transaction page, you need to follow specific steps to make the new items available for transactions. For more information, see [Making New Items Available for Transaction Fields](#).

NetSuite CPQ Configurator can't create the following item types:

- Download items
- Sales tax groups
- Sales tax items
- Shipping cost items
- Matrix items

To work with assemblies on item creation records, see [Working with Assemblies on Item Creation Records](#).

Note: Install NetSuite CPQ Manufacturing to create assembly items automatically. For more information, see the help topic [Installing NetSuite CPQ SuiteApps](#).

Making New Items Available for Transaction Fields

If your account has more than 300 items, new items created with item creation records are automatically added to transaction fields. However, if the account has fewer than 300 items and the **Submit to transaction page** box is checked on the options record, new items can't be added to transaction dropdown lists using item creation records. Instead, check the **Add new resources to transaction fields** box to ensure that new items will be available for transaction fields. You can find this field under the **Submit** subtab at CPQ > Configurator > Settings.

Note: This option may be incompatible with custom scripts deployed on transactions.

However, the **Add new resources to transaction fields** option can't be applied if all these conditions exist:

1. Users are assigned to the **Customer Center** role on the customer record.
2. The Customer Center has fewer than 300 items.
3. Products use item creation records to add the main configurable item and additional items to the transaction.

If you have such a setup, you'll need to replace item creation records with a standard base item and placeholder additional items. Then, you'll need to attach the item creation records to the product and the additional items with mapping records. For more information, see [Assigning Base Items to Products](#) and [Setting Transaction Body and Line Fields Based on Configuration Data](#).

When users save the transaction, placeholder items will be replaced with the items created using item creation records based on the information from mapping records. After saving the transaction, users will see the replaced line items and the correct price amounts.

Note: If you want users to see the correct price amounts for additional items on transactions after submitting the configuration, you should add prices to additional item records rather than sourcing them from item records.

To ensure that placeholder items are replaced with the correct items on transactions:

1. On the product record:
 1. Clear the **Base Item (Item Creation Record)** field.
 2. In the **Base Item** field, select the placeholder you want to use as the base item.
2. On additional item records:
 1. Clear the **Item Creation Record** field.
 2. In the **Item** field, select the placeholders for the additional items.



Note: If the **Consolidate additional item lines** option is enabled on the product record, each additional item requires its own placeholder item.

3. Create a hidden answer for each item creation record to store the internal ID of the newly created record.
4. On each item creation record, select the answer that will store the new item internal ID in the **Answer for Item ID** field.
5. Create a line field mapping record for each item creation record.
 1. Set required and optional fields as needed.
 2. In the **Rule** field, add the same rule as the related item creation record.
 3. Go to the **Transaction Line Fields** subtab.
 4. In the **Field Name** column, select the **Item Replacement ID** field (field ID: `custcol_cpqc_item_repl_id`).
 5. In the **Value** column, specify the answer where you stored the internal ID assigned to the new item. Enter question and answer codes as a resolve expression {Q/A}.
6. Attach the corresponding mapping record to the product record. In the **Line Field Mapping Records** field on the product record, select the corresponding line field mapping record.
7. Attach the corresponding mapping records to the additional items. In the **Line Field Mapping Records** field on the additional item record, select the corresponding line field mapping record.

Working with Item Creation Records

Item creation records store information about items that may be created when users submit a configuration.

To work with item creation records, follow these steps:

1. Enter the information about the new item. For more information, see [Entering Information about the New Item](#).
2. Set up the rule for the item creation record. For more information, see [Setting Up the Item Creation Record Rule](#).
3. Define how to handle existing items. For more information, see [Handling Existing Items](#).
4. Set values for item fields. For more information, see [Setting Values for Item Fields](#).
5. Set the price for items that require it. For more information, see [Setting Pricing](#).
6. Save the item creation record.
7. (Optional) Add components to any item types that require them. For more information, see [Adding Components to Items](#).

New items are created in batches to optimize performance. If you want to create a limited number of items one at a time, go to the options record at CPQ > Configurator > Settings. Under the **Submit** subtab,

check the **Create items one by one** box and click **Save**. After submitting the configuration, the progress bar on the product interface will update as each item is created.

To copy or delete item creation records, see [Working with Product Building Blocks](#).

Entering Information about the New Item

On the item creation record, you can either manually enter information about the new item to be created or copy information of an existing item. For more information, see [Entering Information about the New Item Manually](#) and [Entering Information by Copying an Existing Item](#).

Entering Information about the New Item Manually

To create a new item required by a configuration, select an item type on the item creation record, assign a name or number to the new item, and provide a sequence number. The sequence number determines the order in which items will be created when users submit the configuration.

Items with a lower sequence number are created before items with a higher sequence number. For example, an item related to an item creation record with sequence number 1 will be created before an item related to an item creation record with sequence number 2.

Components for assemblies, groups, kit/packages, and items part of subscription plans must have a lower sequence number than their parent item to be created first. If you enter the sequence numbers incorrectly, their order is automatically rearranged to create child items before their parent items.

If you want to use the new item in the configuration, you must specify the answer where to store its internal ID. The answer must be an input box text field, and you must create it before setting up the item creation record. Then, you can include the question and answer codes in a resolve expression like {Q/A} to obtain the internal ID. For example, you can use the resolve expression in NetSuite CPQ Configurator functions or to add the new item internal ID to transaction fields using mapping records. For more information, see [Input Box Questions](#) and [Obtaining Answer Data with Resolve Expressions](#).

To enter information about the new item manually:

1. On the **Item Creation** subtab, click **New CPQC Item Creation**.
2. Enter a descriptive name for the item creation record.
3. Provide a sequence number.
You can enter a negative or positive number, for example, -2 or 5.
4. Select the item type.
The available item types depend on your account.
5. Provide the item name or number you want to assign to the new item.



Note: You can also obtain the item name or number from answers users select on the product interface by using resolve expressions. For example, you could enter {A/1} as a resolve expression. Question and answer codes are used to retrieve the corresponding answer value.

6. In the **Answer for Item ID** field, select the answer that will store the internal ID of the new item.
Answers are presented as a combination of question and answer codes.

Entering Information by Copying an Existing Item

With item creation records, you can create an item required by the configuration by copying information from an existing item. The existing item is called a template item and works as the base for the newly

created item. Using template item data, you can set only the fields that you want to change instead of manually entering data for every item field.

On the item creation record, you can either select the item you want to copy or obtain the item name or number from a user selection on the product interface using resolve expressions. Then, add a suffix to the name or number of the copied item to make it unique, or assign a new name or number.

To copy an existing item:

1. On the **Item Creation** subtab, click **New CPQC Item Creation**.
2. Enter a descriptive name for the item creation record.
3. Provide a sequence number.
You can enter a negative or positive number. For example, -2 or 5.
4. To identify the item you want to copy, two options are available:
 - In the **Template Item** field, select an existing item.
 - In the **Resolve Template Item Name/Number** field, enter a resolve expression to obtain the item name or number from answers users select on the product interface.
Use question and answer codes separated by a forward slash and enclosed in curly braces. For example, {A/1}. NetSuite CPQ Configurator finds the item name or number using question and answer codes. You can also combine resolve expressions with additional characters to generate the final item name or number, for example, {A/1}-XXX-{B/2}-XX.
5. To create a unique identifier for the new item, two options are available:
 - In the **Item Name/Number Suffix** field, add a suffix to the existing item name or number.
 - In the **Item Name/Number** field, enter a new name or number.
6. In the **Answer for Item ID** field, select the answer that will store the internal ID of the new item.
Answers are presented as a combination of question and answer codes.

Setting Up the Item Creation Record Rule

After entering the information for the new item, set up the item creation rule to determine whether the new item will be created.

When users submit the configuration, the new item is created only if the item creation rule matches the answers users selected on the product interface. If the new item will be used as a material or additional item, the rule for the linked material or additional item records must also match the answers selected by users.

To set up the rule, go to the **Rule** subtab on the item creation record.

In the **Rule** field, provide the condition that determines whether the new item will be created.

Enter question and answer codes separated by a forward slash, for example, A/1. Use operators to combine multiple conditions, for example, A/1&B/2. For more information, see [Activating or Hiding Building Blocks with Rules](#).

Handling Existing Items

When creating a new item, the item name or number you choose may already be assigned to another item. During the item creation record setup, you can define how to proceed when finding an existing item when submitting a configuration.

If an item with the same name or number exists, you have three options in the **Existing Item Handling** field:

- **Do not create a new item** – Doesn't create a new item if the same name or number exists. This is the default option.
- **Update the existing item** – Updates the existing item with the new information from the item creation record.
- **Create the item with a new name/number** – Creates the new item by automatically adding a five-digit suffix to the name or number of the existing item. A hyphen precedes the suffix.

Note: For example, if the item name or number is AB, the new item will be AB-00001. If this item name or number exists, the new unique identifier will be AB-00002, then AB-00003, and so on.

Setting Values for Item Fields

After entering the basic information, you can provide additional details on the item creation record to set item fields for the new item to be created. If you choose to update an existing item or use a template item, only the fields included on the item creation record will be updated.

You can set item fields under the **Item Fields** subtab on the item creation record. The available fields depend on the item type you selected. Basic required fields are automatically displayed and can't be removed. Based on the fields you set, the system dynamically determines the additional required fields, the fields to be dimmed, and the fields to be filled in with default values.

- Note:** Take into account the following:
- Sublists for item records can't be set or edited, except for those sublists specifically mentioned in this [Creating Items Required by the Configuration](#) help topic.
 - You must fill in all required fields, unless you use a template item.

To set field values on an item creation record:

1. On the item creation record, go to the **Item Fields** subtab.
2. Click the **Select Fields** subtab.
3. Click **Add Field**.
4. In the **Field Name** column, select a field from the dropdown list.
5. To provide a value for the field, two options are available:
 - In the **Select Value** column, perform the required task for the field, such as selecting an option, entering a value, or checking a box.
This column shows the fields as they appear on the corresponding item record. For example, you may see a dropdown list, click the dropdown arrow, and select an option.
 - In the **Value** column, enter a resolve expression, for example, {A/1}. For more information, see [Obtaining Answer Data with Resolve Expressions](#).

Note: When using resolve expressions, the field value can't be predicted. To dynamically determine related fields, you must also set or select a placeholder value in the **Placeholder Value** column. Otherwise, you may experience issues when setting other fields.

6. Repeat steps 3 – 5 for each field you want to set.

Setting Pricing

If price is available for the item type you selected, you can set the price for the new item on the item creation record. For more information about pricing in NetSuite, see the help topic [Setting Up Item Pricing](#).

If you use multiple prices and currencies, NetSuite CPQ Configurator automatically recognizes and mirrors the price levels and currencies in your account. On the item creation record, you can set the base price and alternate price levels in multiple currencies for the item that will be created or updated. For more information about multiple prices and currencies in NetSuite, see the help topics [Using Multiple Prices or Currencies](#) and [Using Multiple Pricing](#).

NetSuite CPQ Configurator also supports the NetSuite Quantity Pricing feature and lets you apply different prices depending on the number of items you sell. For more information, see the help topic [Using Quantity Pricing](#).

After creating the item, the price amounts, quantities, and currencies you entered in the item creation record are reflected in the price matrix on the **Sales/Pricing** subtab of the new item record.

To set pricing information:

1. Go to the **Pricing** subtab.
2. If you use quantity pricing, enter the required quantity for each quantity-based price break in the **Qty** fields.
3. Click the currency subtab for which you want to set prices.
4. Enter a value for the base price under the **Qty 0** column.
If you use quantity pricing, also enter a base price for each quantity-based price break under the remaining **Qty** columns.
5. If you use multiple pricing, enter values for each alternate price level under the **Qty 0** column.
If you use quantity pricing, fill in the alternate prices for each quantity-based price break under the remaining **Qty** columns.
6. Repeat steps 4 – 6 for each available currency.
7. Click **Save**.

After providing pricing information, you have completed the setup for the item creation record.

Adding Components to Items

When working with assemblies, groups, or kit/packages on an item creation record, you can add components to the new items to be created. If you create an assembly with advanced BOM, the components are added to the BOM revision record of the newly created item. For more information about these item types, see the help topic [Groups, Assemblies, and Kit/Packages](#).



Note: The item type or template item type must be an assembly, item group, or kit/package on the item creation record.

As a component, you can choose:

- An item from NetSuite
- An item creation record
- A material from the product

Note: Components must be active items and match the subsidiary of the parent item.

An item is active when the **Inactive** box isn't checked. This option is under the **System Information** subtab on the item record.

If you have multiple materials, you can add all active materials as components of the configurable item. To do this, check the **Add all materials as components** box on the item creation record. Materials become active when users select answers that match their rule on the product interface. You can find active materials under the **Materials** option in the Audit menu on the product interface.

To add components to an item:

1. On the **Item Creation** subtab, click **Edit** next to the item creation record you want to modify.
2. If you work with assemblies with advanced BOM, check the **Bill of materials** box to view the **Components** subtab.
3. Click the **Components** subtab.
If you want to use only items from NetSuite as components, you can select the items directly in the sublist. Then, enter a quantity and a sequence number, and click **Add**. Repeat these steps for each component and click **Save**.
4. To add product materials or item creation records as components, click the **New CPQC ITCR Component** button.
If you don't see this button, save the item creation record first.
5. Provide a quantity for this component.
6. Enter a sequence number to determine the order of this component compared to the rest of the components.
7. To specify the component, three options are available:
 - In the **Item** field, select an item from NetSuite.
 - In the **Material** field, select a material from the product.
In this field, you can also choose a material linked to an item creation record.
 - In the **Item Creation Record** field, select an item creation record to create the component when users submit the configuration.
8. Click **Save**.

Assigning Item Creation Records to Materials, Additional Items, or Products

You can assign an item creation record to a material, an additional item (formerly called a breakout item), or a product. When users submit the configuration, the new item will serve as a material, additional item, or the base item (formerly called the base generic product) in the configuration.

On the product interface, materials and additional items associated with an item creation record are highlighted in their corresponding sections in the Audit menu. Their description let users know that these items will be created when submitting the configuration.

To assign item creation records, edit a product at CPQ > Configurator > Product Maintenance. For more information, see the following help topics:

- [Assigning Item Creation Records to Products](#)

- [Assigning Item Creation Records to Additional Items](#)
- [Assigning Item Creation Records to Materials](#)

Assigning Item Creation Records to Products

When you assign an item creation record to the product, the new item will become the base item (formerly called the base generic product). This approach is useful to create a unique main configured item with a new item name or number for each submitted configuration.

As the base item, the new item is displayed as the main configured item on the transaction after users submit the configuration. For more information about base items, see [Assigning Base Items to Products](#).

To assign an item creation record to a product, edit a product record at CPQ > Configurator > Product Maintenance. Then, select an item creation record in the **Base Item (Item Creation Record)** field and save the product record.

Assigning Item Creation Records to Additional Items

If you want to automatically create an additional item (formerly called a breakout item) for a configuration, assign an item creation record to an additional item. When users submit the configuration, the new item will become an additional item and will be assigned its own line on the transaction after the main configured item. For more information about additional items, see [Including Additional Items Related to the Configurable Item](#).

To assign an item creation record to an additional item:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Edit the product you want to modify.
3. Go to the **Additional Items** subtab.
4. Edit the additional item to which you want to assign the item creation record.
5. In the **Item Creation Record** field, select an item creation record.
6. Click **Save**.

Assigning Item Creation Records to Materials

If you want to automatically create a material for a configuration, assign the item creation record to the material. For more information about materials, see [Defining Materials for the Configurable Item](#).

When users create a work order after submitting the configuration, the newly created material is added as a component under the **Items** subtab on work order. For more information about work orders for configured items, see [Creating Work Orders for Configured Items](#).

To assign an item creation record to a material:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Edit the product you want to modify.
3. Go to the **Materials** subtab.
4. Edit the material to which you want to assign the item creation record.
5. In the **Item Creation Record** field, select an item creation record.
6. Click **Save**.

Working with Assemblies on Item Creation Records

Note: Install NetSuite CPQ Manufacturing to automatically create assembly items required by the configuration. For more information, see the help topic [Installing NetSuite CPQ SuiteApps](#).

Using item creation records, you can automatically create assembly items required by the configuration. If your account has the Advanced Bill of Materials feature enabled, you can create assemblies with an advanced BOM. In this case, select assembly as the item type. If this feature isn't enabled, the available item type is assembly/bill of materials, and you can't create assemblies with an advanced BOM. For more information about the advanced BOM in NetSuite, see the help topic [Enabling Advanced BOM](#).

To work with assemblies on item creation records:

1. Create an item creation record and complete the basic fields. For more information, see [Working with Item Creation Records](#).
2. Set fields for the bill of materials and the revision. For more information, see [Working with the BOM and the Revision](#).
3. Add components to the assembly. For more information, see [Adding Components to Items](#).
4. Set fields for the manufacturing routing. For more information, see [Working with the Manufacturing Routing](#).
5. Add routing steps to the manufacturing routing. For more information, see [Adding Routing Steps to the Manufacturing Routing](#).

Working with the BOM and the Revision

With item creation records, you can work with assemblies that use an advanced bill of materials. When working with an advanced BOM, you can set fields for:

- The bill of materials (BOM)
- The revision

If you want to update an existing BOM, make sure you keep the same name on the item creation record. When adding a new BOM to assembly items, an existing one—either standard or master default—will be overwritten:

- Adding a standard BOM to an assembly item will overwrite the first available standard BOM. If a standard BOM doesn't exist, the master default BOM will be overwritten.
- Adding a master default BOM to an assembly item will overwrite the existing master default BOM. If no master default BOM exists, the first standard BOM will be overwritten. The new BOM won't assume the master default status.

Note: Each assembly is limited to one master default BOM. If you add a new BOM with a unique name as the master default using the **Add as a new BOM** option, the **Master Default** box setting will be cleared from any BOMs on the assembly.

To preserve existing BOMs while adding a new one with a unique name, enable the **Add as a new BOM** option.

To set fields for the bill of materials and the revision:

1. On the **Item Creation** subtab, edit the item creation record you want to modify.

2. Check the **Bill of Materials** box.
3. On the **BOM** subtab, complete required and optional fields for the bill of materials and the revision as you would on their respective records. Examples of these fields are **Name**, **Memo**, and **Subsidiary**. For more information, see the help topics [Creating a New BOM Record](#) and [Creating BOM Revisions](#).
4. The **Restrict to current assembly** box is checked by default to limit the bill of materials to the current assembly.

When this option is enabled, only the current assembly is selected in the **Restrict to Assemblies** field of the BOM record.

To make the BOM available to all assemblies:
 - Clear the **Restrict to current assembly** box.
 - Check the **Available for all assemblies** box.
5. To add a new BOM to an assembly without overwriting existing ones, check the **Add as a new BOM** box.

 **Note:** Make sure you provide a unique name for the new BOM. If the name isn't unique, an existing BOM will be overwritten on the assembly record.

6. To add a master default BOM, check the **Master Default** box. If you check this box, the **Available for All Locations** box will be automatically enabled.

 **Note:** If you set this option, the **Master Default** box will be checked under **Manufacturing > Bill of Materials** on the assembly item record. When creating a work order, the master default BOM will appear in the **Bill of Materials** field.

7. Click **Save**.

Working with the Manufacturing Routing

On item creation records, you can set fields for the manufacturing routing. The manufacturing routing groups all the routing steps required to build your assembly.

To set fields in the manufacturing routing:

1. On the **Item Creation** subtab, edit the item creation record you want to modify.
2. Check the **Manufacturing Routing** box.
3. On the **Manufacturing Routing** subtab, fill in the fields related to the manufacturing routing as you would on the respective record. Examples of the fields are the name, memo, and location. For more information, see the help topic [Creating a Manufacturing Routing](#).
4. Click **Save**.

Adding Routing Steps to the Manufacturing Routing

Routing steps are the tasks required to build assemblies. They can be associated with materials or stand alone. On item creation records, you can choose the routing steps that will be added to the manufacturing routing. The following options are available:

- [Adding Stand-Alone Routing Steps](#)
- [Adding Routing Steps Associated with Materials](#)
- [Adding all Active Routing Steps in the Product](#)

You can combine stand-alone routing steps and those associated with materials. A routing step can be associated with multiple materials.

Note: Only active routing steps are added to the manufacturing routing. A routing step is active when its rule matches the answers selected by users on the product interface.

Adding Stand-Alone Routing Steps

Stand-alone routing steps are added to the manufacturing routing using ITCR routing step records.

Note: If you create an ITCR routing step record for a routing step already associated with a material, the ITCR routing step record is disregarded.

To create an ITCR routing step record:

1. Go to the **Routing Steps** subtab.
2. Click **New CPQC ITCR Routing Step**.
3. In the **Routing Step** field, select a stand-alone routing step from the product.
4. Click **Save**.

Adding Routing Steps Associated with Materials

You can add routing steps associated with materials to the manufacturing routing. First, assign a routing step to a material. To do this, go to the material record, select a routing step in the **Routing Step** field, and save the record. For more information, see [Defining Routing Steps to Build the Configurable Item](#).

Second, add the material under the **Components** subtab of the item creation record. If you check the **Add all materials as components** box, all routing steps associated with materials will be added to the manufacturing routing.

Adding all Active Routing Steps in the Product

You can add all active routing steps in the product to the manufacturing routing. A routing step is active when its rule matches the answers selected by users on the interface.

To use all active routing steps, check the **Use all routing steps** box. If you check this box, the manufacturing routing will include:

- Stand-alone routing steps.
- Routing steps associated with active materials.
- Routing steps associated with inactive materials.

Creating Records Required by the Configuration

Note: Use item creation records to create items required by the configuration. If you work with assembly items, item creation records can use existing material items in the bill of materials and the routing steps in the manufacturing routing. For more information, see [Creating Items Required by the Configuration](#) and [Working with Assemblies on Item Creation Records](#).

NetSuite CPQ Configurator can automatically create records required by the configuration using information from creation records. Creation records store the data for the potential new record to be

created, such as the type, the key field, and the rule that determines whether it will be created. Using resolve expressions, you can include information from the product interface to set field values for the new record. For more information, see [Obtaining Answer Data with Resolve Expressions](#).

Creation records are also provided with subrecords to let you include sublists and sublist lines in the new record to be created. These subrecords are called RCR sublist and RCR sublist line (formerly called an RCR sublist component).

With creation records, you no longer need to manually create every record that may be required by the configuration. After submitting the configuration, NetSuite CPQ Configurator will create them for you based on users' choices during the configuration process. Using resolve expressions, NetSuite CPQ Configurator can create a different record from the same creation record every time a user submits a configuration.

Additionally, NetSuite CPQ Configurator reduces manual work by letting you copy information from existing records and set only the fields you want to change for the new record. You can also update existing records with new information from creation records.

During the configuration process, users can view the active creation records under the **Record Creation** option in the **Audit** menu on the product interface. Active creation records have a rule that matches the answers selected by users on the interface.

When users submit the configuration, NetSuite CPQ Configurator creates the record only if the rule of the creation record matches the answers selected by users on the interface. After submitting the configuration, users can monitor the records through a progress bar on the product user interface.

Note: If your account includes more than 300 records of the type you want to create, new records are automatically added to the transaction fields. However, if your account has fewer than 300 records of the same type, you must enable an option to add the new records to transaction fields. Update the setup by going to CPQ > Configurator > Settings. Under the **Submit** subtab, check the **Add new resources to transaction fields** box. This option may be incompatible with custom scripts deployed on transactions.

Working with Creation Records

Creation records store the information about the records that may need to be created after users submit the configuration. To work with creation records, follow these steps:

1. Enter the information about the new record. For more information, see [Entering Information about the New Record](#).
2. Set up the rule for the creation record. For more information, see [Setting Up the Creation Record Rule](#).
3. Define how you want to handle existing records. For more information, see [Handling Existing Records](#).
4. Set values for record fields. For more information, see [Setting Values for Record Fields](#).
5. Save the creation record.
6. Add record sublists and set field values for sublist lines. For more information, see [Adding Sublists and Setting Field Values](#).

New records are created in batches to optimize performance. If you want to create a limited number of records one at a time, go to the options record at CPQ > Configurator > Settings. Under the **Submit** subtab, check the **Create records one by one** box and click **Save**. After submitting the configuration, the progress bar on the product interface will update as each record is created.

To copy or delete creation records, see [Working with Product Building Blocks](#).

Entering Information about the New Record

On creation records, you can either manually enter information about the new record to be created or copy details from an existing record. For more information about copying information from another record, see [Entering Information by Copying an Existing Record](#).

To enter information about the new record to be created, assign a name to the creation record, select a record type for the new record, and the key field—a record field that works as a unique identifier for the new record.

Creation records have a sequence numbers that determine the order in which records are created when users submit the configuration. Records with a lower sequence number are created before those with a higher sequence number. For example, a record with sequence number 1 will be created before a record with sequence number 2.

If you want to use the new record in a configuration, you must specify the answer where its internal ID will be stored. This answer must be an input box text field, which you must create before setting up the creation record. Then, you can create a resolve expressions with the question and answer codes, for example {Q/A}, and enter it as a field value in other records. For example, you can copy the internal ID to a transaction body or line field with mapping records, or use it in NetSuite CPQ functions. For more information about input box text answers and resolve expressions, see [Input Box Questions](#) and [Obtaining Answer Data with Resolve Expressions](#).

Creation records are processed after item creation records. If you need to process a creation record before item creation records, check the **Process before item creation** box on the creation record. This option is useful, for example, when the internal ID of the creation record must be copied to a field on an item to be created. In this case, the record must be created before the item.

To enter information about the new record:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Edit the product you want to modify.
3. On the **Record Creation** subtab, click **New CPQC Record Creation**.
4. Enter a descriptive name for the creation record.
5. Provide a sequence number.
You can enter a negative or positive number. For example, -2 or 5.
6. Select the record type.
The available record types depend on your account.
7. To define and assign a value to the key field:
 1. In the **Key Field**, select the field used as a unique identifier.
 2. On the **Record Fields** subtab, select the key field and assign it a value.
8. In the **Answer for Record ID** field, specify the answer that will store the internal ID of the new record to be created. Enter the question and answer codes separated by a forward slash, for example, Q/A.
9. To process the creation record before item creation records, check the **Process before item creation** box.

Entering Information by Copying an Existing Record

With creation records, you can create a new record by copying the information of an existing one—called a template record. The template record works as the base for the new record to be created. Using the template record data, you only need to update the fields that you want to change, instead of entering all the data manually.

On the creation record, you can either select the record you want to copy or use resolve expressions to identify the record from answers users select on the product interface. Then, you add a suffix to the key field of the copied record to create a unique identifier for the new record.

To copy the information of an existing record:

1. Follow the same steps used to enter information for a new record, but skip step 6. For more information, see [Entering Information about the New Record](#).
2. To identify the record you want to copy, two options are available:
 - In the **Template Record** field, select an existing record.
 - In the **Resolve Template Record Key Field** field, enter a resolve expression to obtain the key field value from the answers users select on the product interface.
 Use question and answer codes separated by a forward slash and enclosed in curly braces, for example, {A/1}. NetSuite CPQ Configurator finds the key field value using the question and answer codes. You can also combine resolve expressions with additional characters to generate the final key field value, for example, {A/1}-XXX-{B/2}-XX.
3. In the **Key Field Suffix** field, add a suffix to the existing key field value to create a unique identifier for the new record.
4. (Optional) To assign a new value to the key field:
 1. Go to the **Record Fields** subtab.
 2. Select the key field.
 3. Enter the new value.
 This new value will overwrite both the copied key field value and the suffix.

 **Note:** You can't skip assigning a suffix, even if it won't be applied.

Setting Up the Creation Record Rule

After entering basic information for the new record, set up the rule that determines whether the new record will be created. When users submit the configuration, the new record will be created only if the rule of the creation record matches the answers users selected on the product interface.

To set up the rule, go to the **Rule** subtab.

In the **Rule** field, provide the condition that determines whether the new record will be created.

Enter question and answer codes separated by a forward slash, for example, A/1. Use operators to combine multiple conditions. For example, A/1&B/2. For more information, see [Activating or Hiding Building Blocks with Rules](#).

Handling Existing Records

When creating a new record, another record with the same value for the key field may already exist. During the creation record setup, you can define how to proceed when finding an existing record with the same key field value using the **Existing Record Handling** field. Three options are available:

- **Do not create a new record** – Doesn't create a new record if the same key field value exists. This is the default option.
- **Update the existing record** – Updates the existing record with the new information from the creation record.

- **Create the record with a new key field value** – Creates the new record by automatically adding a five-digit suffix to the key field value of the existing record. A hyphen precedes the suffix.

Note: For example, if the key field value is AB, the new record is assigned AB-00001 as a unique identifier. If this key field value exists, the new unique identifier will be AB-00002, then AB-00003, and so on.

Setting Values for Record Fields

After entering the basic information, you can provide additional details on the creation record to set record fields for the new record to be created. If you choose to update an existing record or use a template record, only the fields included on the creation record will be updated.

You can set record fields under the **Record Fields** subtab on the creation record. The available fields depend on the record type you selected. Basic required fields are automatically displayed and can't be removed. Based on the fields you set, the system dynamically determines the additional required fields, the fields to be dimmed, and the fields to be filled in with default values.

To set field values on a creation record:

1. On the creation record, go to the **Record Fields** subtab.
2. Click the **Select Fields** subtab.
3. Click **Add Field**.
4. In the **Field Name** column, select a field.
5. To provide a value for the field, two options are available:
 - In the **Select Value** column, perform the required task for the field, such as selecting an option, entering a value, or checking a box.
This column shows the fields as they appear on the corresponding record. For example, you may see a dropdown list, click the dropdown arrow, and select an option.
 - In the **Value** column, enter a resolve expression, for example, {A/1}. For more information, see [Obtaining Answer Data with Resolve Expressions](#).

Note: When using resolve expressions, the field value can't be predicted. To dynamically determine related fields, you must also set or select a placeholder value in the **Placeholder Value** column. Otherwise, you may experience issues when setting other fields.

6. Repeat steps 3 – 5 for each required field and every other field you want to set.
7. Click **Save**.

Adding Sublists and Setting Field Values

After saving the creation record, you can work on the record sublists for the new record to be created. To set values for record sublists, follow these steps:

1. Create an RCR sublist record for each sublist type you want to use.
Each sublist record stores the list of the sublist lines for the new record to be created.

Note: Only one sublist record for each sublist type can be created.

2. Create an RCR sublist line record (formerly called an RCR sublist component) for each sublist line.

3. On the RCR sublist line record, select the sublist line fields and enter their values. For more information, see [Creating a Sublist Line Record](#).

Custom record types with the **Allow child record editing** option enabled can work as sublist lines. In that case, you can use these custom records as sublist lines also within RCR sublist records. You can enable this option on the custom record at Customization > Lists, Records, & Fields > Record Types. Edit the custom subrecord and ensure that the box for **Allow child record editing** is checked.

To create a sublist record:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Edit the product you want to modify.
3. On the **Record Creation** subtab, edit the creation record you want to modify.
4. Go to the **Sublists** subtab.

 **Note:** This subtab becomes available after saving the creation record.

5. Click **New CPQC RCR Sublist**.
6. Enter a descriptive name to identify the sublist record.
7. In the **Sublist ID** field, select one of the available sublists for the record type.
8. In the **Line Add Method** field, select how to add new sublist lines:
 - **Delete & add new lines** – Deletes all existing sublist lines and adds new ones. This is the default option.
 - **Append new lines** – Adds the new lines below the existing lines.

 **Note:** This option requires you to check the **Enable dynamic mode** box on the creation record.

- **Overwrite existing lines** – Writes the new lines over the existing one by starting from the first line. If the number of existing lines exceeds the new ones, exceeding existing lines remain unchanged.

 **Note:** On RCR sublist line records, you can remove all or some required fields to avoid overwriting line fields.

9. Click **Save**.

Creating a Sublist Line Record

After creating the RCR sublist record, you can create sublist lines using RCR sublist line records (formerly called RCR sublist components). On the RCR sublist line record, you can set the sublist line fields and their values according to your business needs. Each RCR sublist line record has a sequence number to order the sublist line within the sublist. Sublist lines with a lower sequence number will be displayed higher on the sublist.

To create a sublist line record:

1. Edit the RCR sublist record for which you want to add sublist lines.
2. Go to the **Sublist Lines** subtab.
3. Click **New CPQC RCR Sublist Line**.
4. Enter a descriptive name to identify the sublist line.

5. Provide a sequence number to order the sublist line within the sublist.

You can enter a negative or positive number. For example, -2 or 5.

6. On the **Sublist Line Fields** subtab, set field values for the sublist line as you would set values for record fields. For more information see, [Setting Values for Record Fields](#).

 **Note:** You can remove all or some required fields to avoid overwriting existing line fields.

7. Go to the **Rule** subtab.
8. In the **Rule** field, provide the condition that determines whether the sublist line will be added to the sublist and its fields will be set.

Enter question and answer codes separated by a forward slash, for example, A/1. Use operators to combine multiple conditions. For example, A/1&B/2. For more information, see [Activating or Hiding Building Blocks with Rules](#).
9. Click **Save**.

Setting Transaction Body and Line Fields Based on Configuration Data

With NetSuite CPQ Configurator, you can automatically set transaction fields based on users' choices and save time when working with eligible transactions. For more information about eligible transactions, see [Selecting Eligible Transactions](#).

If you've previously created custom fields to store configuration data, you can also copy this data to transactions. Other NetSuite capabilities can use this additional configuration data, for example, to create reports, launch workflows, or create documents.

Let's say your business includes a desk as a configurable item. You can add information about the wood type to the transaction, generate a report to see the type customers prefer, and increase its stock. For more information about creating custom transaction fields, see the help topics [Creating Custom Transaction Body Fields](#) and [Creating Custom Transaction Line Fields](#).

You can set transaction body and line fields on eligible transactions using mapping records. Mapping records store information about the fields to be set, such as the transactions to which the fields belong, the field values, and the rule that determines whether the fields will be set after users submit the configuration. A mapping record becomes active if its rule matches the answers users selected on the product interface.

Each mapping record has a sequence number that determines the order in which the fields will be set on transactions or create the work order. Mapping records with a lower sequence number are applied before those with a higher sequence number, regardless of whether they set body or line fields. Consequently, a mapping record may overwrite values set by a previous mapping record.

You have available a mapping record for each transaction field type:

- **Body field mapping record** – Sets transaction body fields.
- **Line field mapping record** (formerly called component field mapping record) – Sets transaction line fields.

You can set transaction line fields for the main configured item and additional items. Additionally, you can set line fields for materials and fields for routing steps when added as components and operation tasks to work orders. After creating mapping records, you must assign them to these resources to set fields on transactions. For more information, see [Setting Line Fields for Main Items and Additional Items](#) and [Setting Fields for Materials and Routing Steps](#).

Note: Install NetSuite CPQ Manufacturing to apply mapping records to work orders, materials, and routing steps. For more information, see the help topics [Installing NetSuite CPQ SuiteApps](#) and [Creating Work Orders for Configured Items](#).

You can set fields for multiple transaction types in a single mapping record. If a field is shared across transaction types, the same value is applied to all the selected transaction types. To set different values for the same field on different transaction types, create a mapping record for each transaction.

Don't use mapping records to edit:

- Transaction fields automatically set by NetSuite CPQ Configurator, such as the item name, quantity, and rate.
- Custom fields used by NetSuite CPQ Configurator to work properly. For more information, see [Maintaining Enabled Specific Custom Transaction Body and Line Fields](#).

Note: Previously, transaction line fields could be set using these fields:

- **Order Column Mapping** (additional item record).
- **WO Column Mapping** (material record).

If you want to switch from these fields to mapping records, move the field information to mapping records and leave these fields blank. After clearing these fields, they will become hidden.

To create a transaction body or line field mapping record:

1. In NetSuite go to CPQ > Configurator > Product Maintenance.
2. Edit the product for which you want to create a mapping record.
3. Go to the **Data Mapping** subtab.
4. Depending on the type of mapping record you want to create, click the related button:
 - To create a body field mapping record, click **New CPQC Body Field Mapping**.
 - To create a line field mapping record, click **New CPQC Line Field Mapping**.

Note: The steps for creating body and line field mapping records are similar, but line field mapping records include a few additional options.

5. Provide a descriptive name for the mapping record.
6. Provide a sequence number to determine the order in which the mapping record is applied.
You can enter a negative or positive number, for example, -2 or 5.
7. Select the transaction types to which you want to apply the mapping record.
The available fields depend on the selected transaction type.
8. To apply a line field mapping record to all additional items in the product, check the **Apply to all additional items** box.
These mapping records will be displayed in the **Common Line Field Mapping Records** field on additional item records.
9. To set fields for work orders on a line field mapping record:
 - In the **Work Order Target** field, select one of these options:
 - **Component** – Sets line fields for the component associated with the material.
 - **Operation Task** – Sets fields for the operation task associated with the routing step.

- To apply a line field mapping record to all materials in the product, check the **Apply to all materials** box.
These mapping records will be displayed in the **Common Line Field Mapping Records** field on material records.
 - To apply a line field mapping record to all routing steps in the product, check the **Apply to all routing steps** box.
These mapping records will be displayed in the **Common Line Field Mapping Records** field on routing step records.
 - 10. In the **Rule** field, provide the condition that determines whether body or line fields will be set on transactions using mapping records.
Enter question and answer codes separated by a forward slash, for example, A/1. Use operators to combine multiple conditions, for example, A/1&B/2. For more information, see [Activating or Hiding Building Blocks with Rules](#).
 - 11. On the **Transaction Body Fields** or **Transaction Line Fields** subtab, click **Add Field** to set values for transaction body or line fields.
 - 12. In the **Field Name** column, select the field you want to set.
 - 13. To provide a value for the field, two options are available:
 - In the **Select Value** column, perform the required task for the field, such as selecting an option, entering a value, or checking a box.
This column shows fields as they appear on the corresponding transaction. For example, you may see dropdown list, click the dropdown arrow, and select an option.
 - In the **Value** column, enter a resolve expression, for example, {A/1}. For more information, see [Obtaining Answer Data with Resolve Expressions](#).
-  **Note:** When using resolve expressions, the field value can't be predicted. To dynamically determine related fields, you must also set or select a placeholder value in the **Placeholder Value** column. Stepping this step may cause issues when setting other fields.
- 14. Repeat steps 11 – 13 to set values for as many fields as required.
 - 15. Click **Save**.

Setting Line Fields for Main Items and Additional Items

If you want to set transaction line fields for the main configured item or an additional item, you need to assign the mapping records you previously created to the product or the additional item. To set line fields for the main configured item, edit the product record and select the mapping records in the **Line Field Mapping Records** field. The main configured item is represented by the base item. For more information about base items, see [Assigning Base Items to Products](#).

When setting line fields for additional items, you can apply mapping records to all additional items for a product or only to specific ones. To apply mapping records to a specific additional item, edit the additional item record and select the mapping records in the **Line Field Mapping Records** field. For more information about linking an additional item to mapping records, see [Setting Transaction Line Fields for Additional Items](#).

On the additional item record, you can also view the shared mapping records in the **Common Line Field Mapping Records** field. These mapping records have the **Apply to all additional items** box checked.

After users submit the configuration, the associated mapping records set field values for the main configured item or the additional item line included in the transaction.

Setting Fields for Materials and Routing Steps

If you want to set fields for a material or a routing step when it's added as a component or operation task to a work order, assign the mapping records you've previously created to the material or routing step. You can apply mapping records to all materials or routing steps in a product or only to specific ones. When users create a work order, mapping records set:

- Line fields for the component associated with the material.
- Fields for the operation task associated with the routing step.

For more information about work orders for configured items, see [Creating Work Orders for Configured Items](#).

On the material or the routing step records, you can view the shared mapping records in the **Common Line Field Mapping Records** field. These mapping records have one of these boxes checked:

- **Apply to all materials.**
- **Apply to all routing steps.**

To apply mapping records to a specific material or routing step, assign the mapping records to the material or the routing step using the **Line Field Mapping Records** field. For more information about materials and routing steps, see [Defining Materials for the Configurable Item](#) and [Defining Routing Steps to Build the Configurable Item](#).

Working with Product Configurations

The product configuration gathers the selections made by users for a specific configurable item. Configurations are stored on the configuration record. To view the configuration record, click the **Access Config Record** button on the configured item line on transactions.

Any user with advanced permissions can view the configuration record and its ID. To set role permissions, go to the **Access** subtab on the employee record. Then, select **Advanced** in the **CPQ Configurator Permissions** field and save the record.

 **Note:** Users with an administrator role are assigned advanced permissions by default without setting this field.

You can also view the configuration ID directly on transactions. In the **Configurator Data** line field, look for the value of the configId parameter.

The configuration record stores the configuration in a compressed format. You can use a RESTlet endpoint to retrieve the compressed configuration using custom client and server scripts. After taking the configuration ID as a parameter, the endpoint searches for the record, and returns the decompressed configuration as a string. You can then convert this string into a JSON object if needed.

In client scripts, use the `_nHttps.post()` method from the `N/https` module. The syntax is:

```
1 const response = _nHttps.post({
2   url: _nUrl.resolveScript({
3     scriptId: 'customscript_cpqc_rs_maint',
4     deploymentId: 'customdeploy_cpqc_rs_maint'
5   }),
6   body: {
7     task: 'getConfiguration',
8     configurationId: id_number
9   }
10 });
```

See the following example:

```

1 | const response = _nHttps.post({
2 |   url: _nUrl.resolveScript({
3 |     scriptId: 'customscript_cpqc_rs_maint',
4 |     deploymentId: 'customdeploy_cpqc_rs_maint'
5 |   }),
6 |   body: {
7 |     task: 'getConfiguration',
8 |     configurationId: 501
9 |   }
10 | });
11 |
12 | console.log('Parsed configuration: ', JSON.parse(response.body));

```

In server scripts, use the `_nHttps.requestRestlet()` method from the `N/https` module. The syntax is:

```

1 | const response = _nHttps.requestRestlet({
2 |   scriptId: 'customscript_cpqc_rs_maint',
3 |   deploymentId: 'customdeploy_cpqc_rs_maint',
4 |   method: 'POST',
5 |   body: {
6 |     task: 'getConfiguration',
7 |     configurationId: id_number
8 |   }
9 | });

```

See the following example:

```

1 | const response = _nHttps.requestRestlet({
2 |   scriptId: 'customscript_cpqc_rs_maint',
3 |   deploymentId: 'customdeploy_cpqc_rs_maint',
4 |   method: 'POST',
5 |   body: {
6 |     task: 'getConfiguration',
7 |     configurationId: 501
8 |   }
9 | });
10 |
11 | console.log('Parsed configuration: ', JSON.parse(response.body));

```

Including Additional Conditions in Rules and Actions with Predefined Answers

Predefined answers (formerly called system Q/As or system QAs) are fixed questions and answer combinations included in NetSuite CPQ Configurator. You can use predefined answers as variables to include additional conditions in rules and actions when activating or hiding building blocks and performing tasks. These conditions are based on information from your NetSuite account.

In rules, use this syntax to reference a predefined answer:

```
1 | Q/A["value"]
```

For example, you can include a predefined answer in a tab rule to display this tab only for administrators:

```
1 | SOLE/ROLEID["administrator"]
```

Predefined answers with SOLE as the question code contain information about the configuration. The NS question code is used for predefined answers with information about the transaction and your account. Most predefined answers are read-only, but a few can be set or edited using actions and functions.

You can view most predefined answers and their values under the **Data** option in the **Audit** menu on the product interface.

For more information, see the lists of predefined answers grouped by the type of information they store:

- [Predefined Answers for User Information](#)
- [Predefined Answers for Configuration Information](#)
- [Predefined Answers for Configuration Information Storing JSON Objects](#)

Predefined Answers for User Information

The following table introduces predefined answers that store user session information.

Predefined Answer	Description	Value Example	Read-Only
SOLE/USER_NAME	User name	Name Surname	Yes
SOLE/USER_LOCATION	User location ID	—	Yes
SOLE/USER_EMAIL	User email	name.surname@oracle.com	Yes
SOLE/USER_DEPARTMENT	User department ID	—	Yes
SOLE/USER	Logged in employee internal ID	—	Yes
SOLE/ROLEID	User role name	administrator	Yes
SOLE/ROLECENTER	User role center name	BASIC	Yes

Predefined Answers for Transaction Information

The following table introduces predefined answers that store transaction information.

Predefined Answer	Description	Value Example	Read-Only
NS/CURRENCY	Currency name	USD	Yes
NS/CURRENCYCODE	Currency internal ID	—	Yes
NS/ENTITYID	Customer Internal ID (Customer selected on the transaction)	—	Yes
NS/RECTYPE	Transaction type name	salesorder	Yes
NS/SODATE	Transaction Date	2/5/2024	Yes

Predefined Answers for Configuration Information

The following table introduces predefined answers that store configuration information.

Predefined Answer	Description	Value Example	Read-Only
SOLE/IMAGES	The array of JSON objects with the active images. Its content is sourced from the image preview on the product interface.	[{"url":"folder/folder2/folder3","style":{"}}]	No

Predefined Answer	Description	Value Example	Read-Only
SOLE/MAT_PRICE	The price of a single configured item. The value is sourced from the Subtotal field on the product interface.	—	Yes
SOLE/QUANTITY	The quantity of the configured item. The value is sourced from the Quantity field on the product interface.	—	No
SOLE/QUANTITY_CHANGE	Detects when users manually change the value of the Quantity field on the product interface. If users change the value, this predefined answer becomes true and resets to false on the next mouse click.	true	Yes
SOLE/TOTAL_PRICE	The total price from the Total field on the product interface.	—	Yes
SOLE/ITEMID	The internal ID of the base item.	—	Yes
SOLE/TAB	The active tab code. The value is sourced from the Code field on the tab record.	TAB1	Yes

Predefined Answers for Configuration Information Storing JSON Objects

These predefined answers store configuration information as arrays of JSON objects:

- **SOLE/ITEMS** – Stores the list of active additional items. For more information, see [SOLE/ITEMS](#).
- **SOLE/MATERIALS** – Stores the list of active materials. For more information, see [SOLE/MATERIALS](#).
- **SOLE/ROUTING** – Stores the list of active routing steps. For more information, see [SOLE/ROUTING](#).

Note: A building block is active when its rule matches the answers users select on the product interface.

These predefined answers aren't displayed under the **Data** option in the **Audit** menu on the product interface. Additional items, material items, and routing steps have dedicated entries in the **Audit** menu. You can also use these predefined answers to create table sections in NetSuite CPQ Proposal Generator templates. For more information, see the help topic [Adding Content to Sections](#).

SOLE/ITEMS

The SOLE/ITEMS predefined answer stores an array of JSON objects that represent the list of active additional items in the configuration. See an example with one additional item:

```
1 | [
2 |   {
3 |     "id": "124",
```

```

4      "item": "2180",
5      "ext": 0,
6      "itemid": "LAMP-A1000-01",
7      "itemQA": "",
8      "qty": 1,
9      "price": 300,
10     "desc": "Desk lamp",
11     "pricemul": 1,
12     "name": "Desk Lamp",
13     "itemname": "Desk Lamp",
14     "itemQA_res": "LAMP-A1000-01",
15     "displayItemId": "LAMP-A1000-01",
16     "colmap": {
17     }
18   }
19 }
20 ]

```

The following table summarizes the JSON properties for the SOLE/ITEMS predefined answer. Most properties reference fields on the additional item record. If a property reference a field on another record or the source record may be unclear, the record name is explicitly stated. Field names are marked in bold. For more information about additional items, see [Including Additional Items Related to the Configurable Item](#).

JSON Property	Description
id	The internal ID of the additional item record.
item	The internal ID of the item record associated with the additional item.
ext	The additional item Quantity multiplied by the Average Cost from the related item record.
itemid	The Item Name/Number of the item record associated with the additional item.
itemQA	The resolve expression entered in the Resolve Item Name/Number field.
qty	Quantity
price	Price
desc	Description
pricemul	Price Multiplier
name	The Item Display Name field of the corresponding item record.
itemname	Name
itemQA_res	The resolved value from the Resolve Item Name/Number field.
displayItemId	The Item Name/Number of the item record associated with the additional item.

SOLE/MATERIALS

The SOLE/MATERIALS predefined answer stores an array of JSON objects that represent the list of active material items in the configuration. See an example with one material item:

```

1  [
2    {
3      "id": "212",
4      "item": "2185",
5      "qty": 1,
6      "route": "105",
7      "phantom": false,

```

```

8      "itemQA": "",
9      "units": "",
10     "comp_yield": "97",
11     "colmap": {},
12     "desc": "Side panel with clear gloss finish",
13     "itemQA_res": "SIDE-PANEL-24X12",
14     "displayItemId": "SIDE-PANEL-24X12"
15   }
16 ]

```

The following table summarizes the JSON properties for the SOLE/MATERIALS predefined answer. Most properties reference fields on the material item record. If a property reference a field on another record or the source record may be unclear, the record name is explicitly stated. Field names are marked in bold. For more information about materials, see [Defining Materials for the Configurable Item](#).

JSON Property	Description
id	The internal ID of the CPQ material item record.
item	The internal ID of the item record associated with the material item.
qty	Quantity
route	The internal ID of the CPQC routing step record associated with the material.
phantom	Phantom Material The property value can be true or false.
itemQA	The resolve expression entered in the Resolve Item Name/Number field
units	Primary Units Type
comp_yield	Component Yield
desc	Description
itemQA_res	The resolved value from the Resolve Item Name/Number field.
displayItemId	The Item Name/Number of the item record associated with the material item.

SOLE/ROUTING

The SOLE/ROUTING predefined answer stores an array of JSON objects that represent the list of active routing steps. See an example with one routing step:

```

1  [
2    {
3      "id": "125",
4      "name": "Desk Assembly",
5      "comments": "Assemble desk",
6      "opseq": "1",
7      "r_opseq": "1",
8      "stime": "15",
9      "rrate": "10",
10     "mwc": "1728",
11     "mct": "1",
12     "colmap": {}
13   }
14 ]

```

The following table summarizes the JSON properties for the SOLE/ROUTING predefined answer. These properties refer to fields on the routing step record. Field names are marked in bold. For more information about routing steps, see [Defining Routing Steps to Build the Configurable Item](#).

JSON Property	Description
id	The internal ID of the CPQC routing step record.
name	Name
comments	Comments
opseq	Operation Sequence Doesn't display the returned value of resolve expressions.
r_opseq	Operation Sequence Displays the returned value from resolve expressions.
stime	Setup Time (min)
rrate	Run Rate (min/unit)
mwc	Manufacturing Work Center
mct	Manufacturing Cost Template

Working with Configurable Subscriptions

With SuiteBilling, you can manually create standard subscriptions and their related records. For more information, see the help topic [SuiteBilling Overview](#). Alternatively, you can automatically create configurable subscriptions and associated records with NetSuite CPQ Configurator. Subscriptions and related records will be created in your account after users submit the configuration.

When working with configurable subscriptions, the most common use case is to manually create the associated subscription plan with SuiteBilling. If your business frequently adds new services and you want to create them automatically, you can use NetSuite CPQ Configurator to create configurable subscription plans.

Automatically creating subscriptions and associated records involves several NetSuite CPQ capabilities. Before working with configurable subscriptions, become familiar with these NetSuite CPQ capabilities:

- Item creation records. For more information, see [Creating Items Required by the Configuration](#).
- Creation records. For more information, see [Creating Records Required by the Configuration](#).
- Mapping records. For more information, see [Setting Transaction Body and Line Fields Based on Configuration Data](#).

To automatically create subscriptions with NetSuite CPQ Configurator, follow the steps listed in the table. You can also learn more about standard subscriptions by reading the corresponding SuiteBilling help topics.

Step Number	Name	Description	NetSuite CPQ Help Topic	SuiteBilling Help Topic
1	Prerequisites	Implement prerequisites: ■ Enable the SuiteBilling features. ■ Check the Add new resources to transaction fields	Implementing Prerequisites	SuiteBilling Setup

Step Number	Name	Description	NetSuite CPQ Help Topic	SuiteBilling Help Topic
		box in NetSuite CPQ Configurator.		
2	Subscription plan	Create a standard subscription plan or use an existing one. Alternatively, create an item creation record to work with a configurable subscription plan.	■ Creating a Standard Subscription Plan ■ Creating a Configurable Subscription Plan	Creating Subscription Plans
3	Subscription	Work with a creation record to create a configurable subscription.	Creating a Configurable Subscription	■ Creating a New Stand-alone Subscription ■ Creating a Subscription from a Transaction
4	Link between the subscription and the subscription plan line	Create a mapping record to link the subscription to the subscription plan line on the transaction.	Associating the Subscription with the Subscription Plan Line	—

Implementing Prerequisites

Before automatically creating configurable subscriptions:

1. Make sure your account includes the SuiteBilling features.



Note: Contact your NetSuite representative to have the SuiteBilling features provisioned for your account.

2. In NetSuite CPQ Configurator, go to the options record at CPQ > Configurator > Settings. Under the **Submit** subtab, check the **Add new resources to transaction fields** box.

This option makes the automatically created subscription plans and their associated records available in transaction dropdown fields that contain fewer than 300 options.



Note: The option may be incompatible with custom scripts deployed on transactions.

After implementing the prerequisites, work with the subscription plan—either standard or configurable. For more information, see [Creating a Standard Subscription Plan](#) and [Creating a Configurable Subscription Plan](#).

Creating a Standard Subscription Plan

You can combine a standard subscription plan with configurable subscriptions. When working with a standard subscription plan, you'll manually create the subscription plan and any dependent records with SuiteBilling. You must also follow the specific requirements outlined in this help topic. These requirements ensure that the subscription plan is compatible with configurable subscriptions.

The following table sums up the steps to combine a standard subscription plan with configurable subscriptions.

Step Number	Name	Description	This Help Topic	SuiteBilling Help Topic
1	Subscription plan member items	Manually create the member items for the subscription plan. Alternatively, you can use existing items from your account.	Creating Member Items for the Standard Subscription Plan	Creating Items for Subscription Plans
2	Subscription plan	Create a standard subscription plan. Alternatively, you can use an existing subscription plan in your account.	Creating the Standard Subscription Plan	Creating Subscription Plans
3	Price book	Create a standard price book.	Creating the Price Book and Price Plan for the Standard Subscription Plan	Creating Price Books
4	Price plan	Create a standard price plan for each price book line.	Creating the Price Book and Price Plan for the Standard Subscription Plan	Creating Price Plans
5	Link between the standard subscription plan and the product or an additional item	Assign the standard subscription plan as the base item for the product. Alternatively, you can assign the subscription plan to an additional item.	Assigning the Standard Subscription Plan to the Product or an Additional Item	—

After working with the standard subscription plan, create the configurable subscription. For more information, see [Creating a Configurable Subscription](#).

Creating Member Items for the Standard Subscription Plan

Manually create the items that you'll select as member items for the subscription plan. These member items will be inherited by all configurable subscriptions created from that subscription plan. If no item should be inherited, create at least one placeholder item with the minimum information required to save it. Without member items, the subscription plan can't be saved. For more information, see the help topic [Creating Items for Subscription Plans](#).

Creating the Standard Subscription Plan

Manually create a standard subscription plan with SuiteBilling. Go to Lists > Subscriptions > Subscription Plans > New. Under the **Lines** subtab, add the required member items or the placeholder item. Complete all required fields and any additional fields as needed. For more information, see the help topic [Creating Subscription Plans](#).

If you use a placeholder line item, don't check the **Required** box. If you don't set this option, the placeholder line item won't be added to the transaction. However, it'll still be inherited by the subscription.

Creating the Price Book and Price Plan for the Standard Subscription Plan

After creating the subscription plan, manually create the price book for the subscription plan with SuiteBilling. Add a price plan for each price book line. During the setup, complete all required fields and any additional fields as needed. For more information, see the help topics [Creating Price Plans](#) and [Creating Price Books](#).

Assigning the Standard Subscription Plan to the Product or an Additional Item

After creating the standard subscription plan and all associated records, assign the subscription plan to the product as the base item. Use this option to add a single subscription plan to the transaction.

In NetSuite CPQ Configurator, edit a product record. In the **Base Item** field, select the standard subscription plan you previously created. For more information about base items, see [Assigning Base Items to Products](#).

Alternatively, if you need to add multiple subscription plans to the transaction, create an additional item (formerly called a breakout item) for each subscription plan. On the additional item record, you can find the **Item** field. In this field, select the subscription plan you want to link the additional item. For more information about additional items, see [Including Additional Items Related to the Configurable Item](#).

Creating a Configurable Subscription Plan

You can automatically create a configurable subscription plan and any associated records with NetSuite CPQ Configurator. The following table shows the steps required to work with a configurable subscription plan.

Step Number	Name	Description	This Help Topic	SuiteBilling Help Topic
1	Answers to store record IDs	Create the answers to store the internal IDs of the newly created records: - subscription plan - price plans - price book	Creating Answers for the Record IDs	—
2	Subscription plan member items	Create an item creation record for each member item of the subscription plan you want to create automatically. Alternatively, use existing items or manually create them.	Creating Member Items for the Configurable Subscription Plan	Creating Items for Subscription Plans
3	Subscription plan	Create an item creation record for the configurable subscription plan.	Working with a Subscription Plan on an Item Creation Record	Creating Subscription Plans
4	Link between the subscription plan and its member items	Create an ITCR component record for each member	Adding Items to the Configurable Subscription Plan	Creating Subscription Plans

Step Number	Name	Description	This Help Topic	SuiteBilling Help Topic
		item to add them to the subscription plan.	with ITCR Component Records	
5	Price plans	Create a creation record for each price plan that will be associated with a price book line.	Working with Price Plans on Creation Records	Creating Price Plans
6	Price book	Create a creation record for the price book.	Working with a Price Book on a Creation Record	Creating Price Books
7	Link between the configurable subscription plan and the product	Assign the configurable subscription plan to the product as the base item.	Assigning the Configurable Subscription Plan to the Product or an Additional Item	—

After working with the configurable subscription plan, create the configurable subscription. For more information, see [Creating a Configurable Subscription](#).

Creating Answers for the Record IDs

Each new record requires its own answer to store the internal ID. Before working with configurable subscription plans, create the answers to store the internal IDs of the following records:

- subscription plan
- price plans
- price book

The answer must be an input box text field. For more information about answers and input boxes, see [Gathering Information with Questions and Answers](#) and [Input Box Questions](#).

Creating Member Items for the Configurable Subscription Plan

If you want to automatically create the member items for the configurable subscription plan, create an item creation record for each member item using NetSuite CPQ Configurator. For more information, see [Creating Items Required by the Configuration](#).



Note: Member items must be created before the subscription plan. Make sure item creation records for member items have a lower sequence number than the item creation record of the subscription plan.

Alternatively, you can use existing items in your account or create them manually. For more information, see the help topic [Creating Items for Subscription Plans](#).

Working with a Subscription Plan on an Item Creation Record

If you want to automatically create a subscription plan, use an item creation record. For more information, see [Creating Items Required by the Configuration](#).

To work with a subscription plan on an item creation record:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Edit a product.
3. Go to the **Item Creation** subtab.
4. Click **New CPQC Item Creation**.
5. In the **Item Type** field, select **Subscription Plan**.
Alternatively, select a subscription plan in the **Template Item** field to copy an existing one.
6. In the **Answer for Item ID** field, select the answer where you want to store the internal ID for the new subscription plan.
7. Complete all required fields and any additional fields as needed.
8. Click **Save**.
9. On the item creation record, go to the **Components** subtab.
10. Create an ITCR component record for each member item of the subscription plan.
For more information, see [Adding Items to the Configurable Subscription Plan with ITCR Component Records](#).

Adding Items to the Configurable Subscription Plan with ITCR Component Records

When creating a subscription plan with an item creation record, you must add at least one member item to the subscription plan. The member item can also be a placeholder item that includes only the information required to save it. You can associate member items to the subscription plan using ITCR component records. Before adding items to the subscription plan, save the item creation record.

You can add items from NetSuite, from product materials, or from item creation records. After choosing the item, you can define or update the item details on the **Select Columns** subtab.

The member item details are the same as those you'd enter on the **Lines** subtab of a standard subscription plan. Examples of item details are the subscription type and the payment mode.

Based on the information you enter under the **Select Columns** subtab, NetSuite CPQ Configurator dynamically determines the additional required fields, the fields to be dimmed, and the fields to be filled in with default values.

To add an item to a subscription plan:

1. On the **Item Creation** subtab, click **Edit** next to the item creation record you want to modify.
2. Go to the **Components** subtab.
3. If you want to add only items from NetSuite to the subscription plan:
 1. Select the item directly in the subtab.
 2. Enter a quantity.
 3. Enter a sequence number.
 4. Click **Add**.
 5. Repeat steps 1 – 4 for each member item.
 6. Click **Save**.
 7. Skip the remaining steps.
4. To add product materials or item creation records as member items of the subscription plan, click **New CPQC ITCR Component**.

If you don't see this button, save the item creation record first.

5. Enter a sequence number to determine the order of the member item in the subscription plan.
6. To specify the item to be added to the subscription plan, three options are available:
 - In the **Item** field, select an item from NetSuite.
 - In the **Material** field, select a material from the product.

 **Note:** You can choose a material linked to an item creation record.

- In the **Item Creation Record** field, select the item creation record that stores the information for the member item.
7. On the **Select Columns** subtab, set the fields as you'd complete on the **Lines** subtab of a standard subscription plan record.
 8. To provide values for the fields, two options are available:
 - In the **Select Value** column, perform the required task for the field, such as selecting an option, entering a value, or checking a box.

This column shows the fields as they appear on the corresponding item record. For example, you may see a dropdown list, click the dropdown arrow, and select an option.
 - In the **Value** column, enter a resolve expression, such as {A/1}. For more information, see [Obtaining Answer Data with Resolve Expressions](#).

 **Note:** When using resolve expressions, the field value can't be predicted. To dynamically determine related fields, you must also set or select a placeholder value in the **Placeholder Value** column. If you skip this step, you may experience issues when setting other fields. When selecting placeholder values, take into account dependencies between fields.

9. Click **Save**.

Working with Price Plans on Creation Records

After creating a configurable subscription plan, use creation records to automatically create price plans. You'll need a price plan for each price book line. Price book lines correspond to the lines on the subscription plan. Later, you'll associate the price plans with the lines on the price book.

To work with a price plan on a creation record:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Edit a product.
3. Go to the **Record Creation** subtab.
4. Click **New CPQC Record Creation**.
5. In the **Record Type** field, select **Price Plan**.
Alternatively, select an existing price plan in the **Template Record** field to copy it.
6. Assign a sequence number.

 **Note:** Price plans must be created before the price book. Make sure sequence numbers for price plans are lower than the one for the price book.

7. In the **Answer for Record ID** field, select the answer where you want to store the internal ID for the new price plan.

8. Check the **Enable dynamic mode** box.

 **Note:** Price plans can only be created in dynamic mode. For more information about the dynamic mode, see the help topic [SuiteScript 2.x Standard and Dynamic Modes](#). Dynamic mode may be incompatible with the fixed order used to organize fields and their values under the **Record Fields** subtab.

9. Complete all required fields and any additional fields as needed.
10. Under the **Record Fields** subtab, specify the currency and type. These are body fields.
11. Click **Save**.
12. Under the **Sublists** subtab, create the price tiers sublist (ID pricetiers) with an RCR sublist record. For more information, see [Adding Sublists and Setting Field Values](#).
13. Under the **Sublist Lines** subtab, create at least one sublist line with an RCR sublist line record. For more information, see [Adding Sublists and Setting Field Values](#).

Working with a Price Book on a Creation Record

After working with price plans on creation records, use another creation record to automatically create a price book. The price book is limited to the number of lines specified on the subscription plan. Make sure you associate the price book with the corresponding subscription plan.

To work with a price book on a creation record:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Edit a product.
3. Go to the **Record Creation** subtab.
4. Click **New CPQC Record Creation**.
5. Complete all required fields and any additional fields as needed.
6. In the **Record Type** field, select **Price Book**.
Alternatively, select an existing price book as a template record.
7. The price book must be created after the price plans. Assign a sequence number that's greater than those associated with the price plans.
8. In the **Answer for Record ID** field, select the answer where you want to store the internal ID of the new price book.
9. Go to the **Record Fields** subtab.
10. In the **Field Name** column, select the **Subscription Plan** field.
11. To specify the subscription plan, two options are available:
 - If you use an existing subscription plan, select it in the **Select Value** column.
 - If you use a created subscription plan, specify the answer where you stored its internal ID. In the **Value** column, enter question and answer codes as a resolve expression {Q/A}. For more information, see [Obtaining Answer Data with Resolve Expressions](#).
12. Click **Save**.
13. Under the **Sublists** subtab, create a price interval sublist with an RCR sublist record. For more information, see [Adding Sublists and Setting Field Values](#).
14. On the RCR sublist record, select **Overwrite existing lines** in the **Line Add Method** field.

15. Under the **Sublist Lines** subtab, create sublist lines with RCR sublist line records. For more information, see [Adding Sublists and Setting Field Values](#).
16. For each sublist line, select the **Price Plan** field under the **Sublist Line Fields** subtab.
17. To specify the price plan value, two options are available:
 - If you want to use an existing price plan, select it in the **Select Value** column.
 - If you want to use a created price plan, specify the answer where you stored its internal ID. In the **Value** column, enter question and answer codes as a resolve expression {Q/A}.

Assigning the Configurable Subscription Plan to the Product or an Additional Item

After working with a configurable subscription plan on an item creation record, assign the item creation record to the product as the base item. Use this option if you want to add a single subscription plan to the transaction.

In NetSuite CPQ Configurator, edit a product record. In the **Base Item (Item Creation Record)** field, select the item creation record for the subscription plan. For more information about base items, see [Assigning Base Items to Products](#).

Alternatively, if you need to add multiple subscription plans to the transaction, create an additional item (formerly called a breakout item) for each subscription plan. On the additional item record, you can find the **Item Creation Record** field. Use this field to link the additional item to the item creation record of the subscription plan. For more information about additional items, see [Including Additional Items Related to the Configurable Item](#).

Creating a Configurable Subscription

After creating the subscription plan—either standard or configurable—you can work with the configurable subscription. Start by creating the answer to store its internal ID. The answer must be an input box text field.

The following table sums up the steps to work with configurable subscriptions.

Step Number	Name	Description	This Help Topic	NetSuite CPQ Help Topic	SuiteBilling Help Topic
1	Answer for subscription ID	Create the answer to store the internal ID of the newly created subscription.	—	Gathering Information with Questions and Answers Input Box Questions	—
2	Subscription	Create a creation record for the subscription.	Working with a Subscription on a Creation Record	Creating Records Required by the Configuration	■ Creating a New Stand-alone Subscription ■ Creating a Subscription from a Transaction
3	Additional line items	Include additional line items in the subscription.	Including Additional Line	—	—

Step Number	Name	Description	This Help Topic	NetSuite CPQ Help Topic	SuiteBilling Help Topic
			Items in the Subscription		
4	Lines and Pricing sublists	Create RCR sublist records for the sublists.	Working with the Lines and Pricing Sublists on RCR Sublist Records	Adding Sublists and Setting Field Values	■ Creating a New Stand-alone Subscription ■ Creating a Subscription from a Transaction
5	Price plans for sublist lines	Create creation records for the price plans associated with the Pricing sublist lines.	Creating Price Plans with Creation Records for Line Items	Creating Records Required by the Configuration	Creating Price Plans
6	Sublist lines	Create RCR sublist lines records for the sublist lines.	Working with Sublist Lines on RCR Sublist Line Records	Adding Sublists and Setting Field Values	■ Creating a New Stand-alone Subscription ■ Creating a Subscription from a Transaction

After setting up the configurable subscription, make sure the subscription will be associated with the corresponding lines on the transaction. For more information, see [Associating the Subscription with the Subscription Plan Line](#).

Working with a Subscription on a Creation Record

When working with a configurable subscription, use a creation record to automatically create the subscription with NetSuite CPQ Configurator.



Note: Make sure you associate the subscription with the corresponding subscription plan and price book during the setup.

To work with a subscription on a creation record:

1. Edit the product record.
2. Go to the **Record Creation** subtab.
3. Click **New CPQC Record Creation**.
4. In the **Record Type** field, select **Subscription**.
Alternatively, select an existing subscription in the **Template Record** field to copy it.
5. The subscription record must be created last. Assign it the greatest sequence number of all creation records.
6. In the **Answer for Record ID** field, select the answer where you want to store the internal ID of the configurable subscription.
7. If you want to include additional lines in the subscription, check the **Enable dynamic mode** box. For more information, see the help topic [SuiteScript 2.x Standard and Dynamic Modes](#).

In the **Lines** sublist you'll create later, lines will be appended to the inherited ones. This option requires dynamic mode to work properly.

 **Note:** Dynamic mode may be incompatible with the fixed order used to store fields and their values under the **Record Fields** subtab.

8. Complete all required fields and any additional fields as needed.
9. Go to the **Record Fields** subtab. During the setup, complete the required fields in this order:
 1. **Customer**
 2. **Subsidiary**
 3. **Currency**
 4. **Initial Term**
 5. **Subscription Plan**
 6. **Price Book**
 7. **Start Date**

If you follow this order, some required fields will automatically load their options.

10. For the **Customer** field:
 1. In the **Value** column, provide the customer ID. Two options are available:
 - Enter the predefined answer for the customer as a resolve expression {NS/ENTITYID}. This predefined answer stores the ID of the customer specified on the transaction. For more information, see [Including Additional Conditions in Rules and Actions with Predefined Answers](#).
 - Create an answer to store the customer ID and enter question and answer codes as a resolve expression {Q/A}. For more information, see [Obtaining Answer Data with Resolve Expressions](#).
 2. In the **Placeholder Value** column, select a placeholder customer. Make sure you choose a placeholder customer from a subsidiary that can work with subscriptions.
11. Some accounts may not include the **Subscription Plan** and the **Price Book** as required fields. If these fields don't appear on the sublist:
 1. Click **Add Field** to include them.
 2. In the **Field Name** column:
 - Select the **Subscription Plan** field.
 - Select the **Price Book** field.
12. To specify the subscription plan and the price book values, two options are available:
 - If you use an existing subscription plan and price book, select them in the **Select Value** column.
 - If you'll automatically create the subscription plan and price book, specify the answers where you'll store their internal IDs. In the **Value** column, enter question and answer codes as a resolve expression {Q/A}.
13. Complete any additional fields as needed.
14. Click **Save**.

Including Additional Line Items in the Subscription

Subscriptions automatically inherit the member items from the subscription plan. You may want to include additional line items in the subscription. To add line items to the subscription, use:

- RCR sublist records to create sublists. For more information, see [Working with the Lines and Pricing Sublists on RCR Sublist Records](#).
- Creation records if you need price plans for the sublist lines. For more information, see [Creating Price Plans with Creation Records for Line Items](#).
- RCR sublist lines records to create sublist lines. For more information, see [Working with Sublist Lines on RCR Sublist Line Records](#).

Working with the Lines and Pricing Sublists on RCR Sublist Records

When adding line items to the configurable subscription, use RCR sublist records to create these sublists:

- **Lines** (subscriptionline)
- **Pricing** (priceinterval)

The **Pricing** sublist is required because line items must be associated with pricing data. Then, create sublist lines with RCR sublist lines records.

 **Note:** If the subscription only needs to inherit the member items of the subscription plan, you don't need to create these sublists. You may use this option when automatically creating a subscription plan that already includes all required lines.

Working with the Lines Sublist on an RCR Sublist Record

Set up the **Lines** sublist first.

To work with the Lines sublist:

1. Edit the creation record for the configurable subscription.
2. Go to the **Sublists** subtab.
3. Click **New CPQC RCR Sublist**.
4. In the **Sublist ID** field, select **subscriptionline**.
5. Assign a sequence number.
6. In the **Line Add Method** field, select **Append new lines**.

This option adds new lines below the existing lines.

 **Note:** With this option, you must check the **Enable dynamic mode** box on the creation record for the configurable subscription.

7. Click **Save**.

Working with the Pricing Sublist on an RCR Sublist Record

After working with the **Lines** sublist, set up the **Pricing** sublist.

To work with the Pricing sublist:

1. Edit the creation record for the configurable subscription.
2. Go to the **Sublists** subtab.
3. Click **New CPQC RCR Sublist**.
4. In the **Sublist ID** field, select **priceinterval**.

5. Assign a sequence number. The **Pricing** sublist must have a greater sequence number than the **Lines** sublist.
6. In the **Line Add Method** field, select **Overwrite existing lines**.
This option writes the new lines over existing ones by starting from the first line.
7. Click **Save**.

Creating Price Plans with Creation Records for Line Items

Note: Before working with price plans on creation records, create the answers to store their internal IDs. These answers must be input box text fields. For more information, see [Input Box Questions](#).

Before working with sublist lines, take care of the price plans for the lines using creation records. Pricing data can be specified using price plans for any line item type. You'll need a price plan for every line where you want to apply pricing. For more information, see [Working with Lines for the Pricing Sublist](#).

Note: For recurring and usage line items, you can only use price plans to specify pricing data.

To work with a price plan on a creation record:

1. Edit a product.
2. Go to the **Record Creation** subtab.
3. Click **New CPQC Record Creation**.
4. In the **Record Type** field, select **Price Plan**.
Alternatively, select an existing price plan in the **Template Record** field to copy it.
5. Price plans must be created before the subscription. Assign them a lower sequence number than that of the subscription.
6. In the **Answer for Record ID** field, select the answer where you want to store the internal ID.
7. Check the **Enable dynamic mode** box. Price plans can only be created in dynamic mode. For more information, see the help topic [SuiteScript 2.x Standard and Dynamic Modes](#).

Note: Dynamic mode may be incompatible with the fixed order used to store fields and their values under the **Record Fields** subtab.

8. Complete all required fields and any additional fields as needed.
9. Under the **Record Fields** subtab, specify the currency and type. These are body fields.
10. Click **Save**.
11. Under the **Sublists** subtab, create the price tiers sublist (ID: pricetiers) with an RCR sublist record. For more information, see [Adding Sublists and Setting Field Values](#).
12. Under the **Sublist Lines** subtab, create at least one sublist line with an RCR sublist line record. For more information, see [Creating a Sublist Line Record](#).

Working with Sublist Lines on RCR Sublist Line Records

After working with the **Lines** and **Pricing** sublists, create their sublist lines with RCR sublist line records.

To access RCR sublist line records, edit the RCR sublist record. Then, go to the **Sublist Lines** subtab. Lastly, click **New CPQC RCR Sublist Line**.

Working with Lines for the Lines Sublist

When creating lines for the **Lines** sublist, add as many lines as needed. Under the **Sublist Line Fields** subtab, add the **Include** box and check it. This option ensures that the line will be added to the transaction. Then, complete all required fields and any additional fields as needed.

Working with Lines for the Pricing Sublist

When creating lines for the **Pricing** sublist, make sure you include a line for each inherited line item from the subscription plan. Because this sublist is created by overwriting existing lines, this setup ensures that the process starts from the first line.

If you don't want to edit inherited line items, remove all fields—including required ones—under the **Sublist Line Fields** subtab. If you do this, inherited line fields won't be overwritten during the creation process. If you want to edit inherited line items, you can maintain any existing fields you want to update and add any additional fields as needed.

You can specify pricing data for any line item type by creating price plans with creation records. The **Line Type** field indicates the line item type and can be one-time, recurring, usage, or commit plus overage. For more information about creating standard price plans for sublist lines, see [Creating Price Plans with Creation Records for Line Items](#).

Some line item type have specific requirements or additional ways to specify pricing data:

- **Recurring and usage line items** – These line items only work with price plans. You must create a price plan with a creation record for this line item type.
When creating the line, add the **Price Plan** field under the **Sublist Line Fields** subtab. Then, enter the answer that stores its internal ID as a resolve expression {Q/A}.
- **One-time line items** – As an alternative to the price plan, you can directly provide pricing data on the line. Under the **Sublist Line Fields** subtab, add the **Total Interval Value** field and provide a value.

After finishing this step and setting up the configurable subscription, make sure the subscription will be associated with the corresponding lines on the transaction. For more information, see [Associating the Subscription with the Subscription Plan Line](#).

Associating the Subscription with the Subscription Plan Line

As the last step for configurable subscriptions, you must ensure that the subscription will be associated with the subscription plan line on the transaction. To associate the subscription with the subscription plan line, you need to link the subscription to the base item (or the additional item) corresponding to the subscription plan using a mapping record. To link the two:

1. Create a line field mapping record (formerly called component field mapping record). For more information about mapping records, see [Setting Transaction Body and Line Fields Based on Configuration Data](#).
2. Link the mapping record to the base item (or the additional item) tied to the subscription plan.

On the transaction, the mapping record will set the **Subscription** line field for the subscription plan by adding the internal ID of the subscription.

To associate a subscription with the subscription plan line:

1. Edit a product record.
2. Go to the **Data Mapping** subtab and then to the **Line Field Mapping** subtab.
3. Click **New CPQC Line Field Mapping**.

4. Complete all required fields and any additional fields as needed.
5. Go to the **Transaction Line Fields** subtab.
6. In the **Field Name** column, add the **Subscription** field.
7. In the **Value** column, specify the internal ID of the created subscription record. Enter the answer where you saved the value as a resolve expression {Q/A}.
8. Click **Save**.
9. Associate the mapping record with the base item (or the additional item) linked to the subscription plan.
 1. Go to the product record for the base item (or the additional item record).
 2. In the **Line Field Mapping Records** field, select the mapping record you have created.
 3. Click **Save**.

By completing this step, you have finished the setup for configurable subscriptions.

Customizing Products with Scripts Using Action Records and Functions

Action records let you customize products using scripts that can automate repetitive tasks or perform advanced tasks, which would otherwise require multiple manual steps. For example, you can use actions to automatically set a series of answers on the product interface or create items required by the configuration.

In custom scripts stored in action records, you can use functions from the NetSuite CPQ Configurator library to provide advanced or custom interactions, and to read or modify data on the server. Some examples of these functions include `getValue()`, `showMessage()`, and `getData()`.

Actions become active if their rule matches the answers users select on the product interface and are processed according to their sequence number—those with a lower sequence number run before actions with a higher sequence number.

However, NetSuite CPQ Configurator distinguishes between before-event and after-event actions, depending on when they're verified and processed (if active) compared to the rest of building blocks (questions, answers, additional items, and so on). Every time a user makes a selection or enters information on the production interface, the system verifies the building block rules and performs the related tasks in this order:

1. Before-event actions according to their sequence number.
2. Other building blocks such as questions, answers, materials, and so on.
3. After-event actions according to their sequence number.

After all building blocks are verified and processed, the product interface refreshes to display any changes.

After-event actions are those with the **Run as after-event** box checked. All other actions are considered before-event. Before-event actions run before after-event actions, even if they have a higher sequence number.

Actions are either client-side or server-side. Each type supports different sets of functions depending on whether they run on the client or the server. For more information, see [Using Client-Side and Server-Side Actions](#).

The action record includes a code editor for writing and editing code. The code editor provides suggestions and documentation as you type. For more information, see [Supported Code Writing](#).

If you frequently use the same code in multiple action records, you can create common actions to share code across actions and products and keep it organized and consistent. For more information, see [Sharing Code with Common Actions](#).

To create an action:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Click **Edit** next to the product you want to modify.
3. On the **Actions** subtab, click **New CPQC Action**.
4. Enter a descriptive name for the action.
This name is displayed under Actions in the Audit menu of the product interface.
5. In the **Code** field, enter a unique identifier for this action.
You can use an abbreviated version of the name as the code to identify the action. The code must contain only uppercase letters, numbers, and underscores.
6. Provide a sequence number to determine the order in which the action is processed.
You can enter a negative or positive number, for example, -2 or 5.
7. If the action should be verified and processed after all other building blocks rules, check the **Run as after-event** box.
For example, you can use this option for actions that needs to process the updated material list.
8. To perform this action only the first time the rule condition is met, check the **Run only on first match** box. The action will only run again if the rule condition becomes false and then true again.
9. In the **Rule** field, provide the condition that determines whether the action will run.
Enter question and answer codes separated by a forward slash, for example, A/1. Use operators to combine multiple conditions, for example, A/1&B/2. For more information, see [Activating or Hiding Building Blocks with Rules](#).
10. (Optional) To include an additional rule in the action rule, select a rule category. The action will run only if both its rule and the rule category match answers selected by users on the product interface. For more information, see [Working with Rule Categories](#).
11. To define the tasks that the action will perform, three options are available:
 - In the code editor, enter the NetSuite CPQ Configurator functions.
 - In the **Add Function** field, select the NetSuite CPQ Configurator functions you want to add to the code editor and follow the onscreen instructions.
 - In the **Common Action** field, select a common action shared across all products and use its code.



Note: If you use a common action, any code entered in the editor will be disregarded. For more information, see [Sharing Code with Common Actions](#).

You can combine manually entered JavaScript functions with those selected in the **Add Function** field.

12. Click **Save**.

Sharing Code with Common Actions

If you frequently use the same code in multiple action records, you can create common actions that are shared across actions and products. This approach lets you easily maintain the code, ensure consistency, and promote code reuse.

Note: If you want to share common actions across multiple products, make sure question and answer codes for functions are the same.

After creating a common action, assign it to an action record using the **Common Action** field.

To create a common action:

1. Go to CPQ > Configurator > Common Actions.
2. Click **New CPQC Common Action**.
3. Provide a name for the common action.
4. To define the tasks that the action will perform, two options are available:
 - In the code editor, enter the NetSuite CPQ Configurator functions for the tasks you want to perform.
 - Use the code writing support:
 1. In the **Product ID** field, select the product from which you want to source question and answer codes for the NetSuite CPQ Configurator functions.
 2. In the **Add Function** field, select the functions you want to add to the code editor and follow the onscreen instructions.
5. Click **Save**.

Supported Code Writing

You can write and edit code using the integrated editor on the action record. As you type, the editor automatically displays a popup window with suggestions and documentation that guide you through the scripting process. Short annotations help you identify the suggestion type. For example, annotations can identify a suggestion as a function or a keyword.

You can scroll through the list of suggestions and select the one you prefer. When moving through the list, the suggestion popup displays documentation for each NetSuite CPQ Configurator function. If you select a function, the editor automatically adds a code template with parameter placeholders.

You can move through the placeholders by pressing **Tab** and replace them with business information. If you want to hide the suggestion popup, press **Esc**. To bring it back, press **Ctrl+Space**.

Using Client-Side and Server-Side Actions

Two types of actions are available—client-side and server-side. Client-side actions contain scripts that run in the web browser, whereas server-side actions run on the server and can read or modify data on the server. Depending on their type, actions can work with different sets of functions.

The functions can be categorized as follows:

- **Universal functions** – Work with questions and answers. For example, `getValue()` and `setValue()`.
- **User interface functions** – Work with and change the look and feel of the product interface. For example, `showMessage()`, `customizeUI()`, and `getProgress()`.
- **Window interaction functions** – Work with the parent window. For example, `queryParent()` and `sendMessage()`.
- **Status functions** – Work with the NetSuite CPQ Configurator statuses (startup, running, final). For example, `submitConfig()`.

- **Database functions** – Read and write data to the database on the server. For example, `getData()`, `getItemPrice()`, and `createRecord()`.

Only server-side actions can work with functions that make requests to the database. Client-side actions can't access the database directly, but they can request information by calling server-side actions. Both server-side and client-side actions work with the functions that interact with questions and answers. Client-side actions can also work with functions that interact with the user interface, the parent window, and statuses.

The following table sums up the available function types for client-side and server-side actions.

Client-Side Actions	Server-Side Actions
Universal functions	Universal functions
User interface functions	Database functions
Window interaction functions	—
Status Functions	—

User interface functions, window interaction functions, and status functions are considered client-side functions because they only work in client-side actions. Database functions are considered server-side functions because they work only in server-side actions. For more information about the function types, see [List of Functions](#).

To create a server-side function, check the **Run on server** box on the action record. Actions without this option enabled are considered client-side.

Both server-side and client-side actions can be written in JavaScript ES5 or ES6. When activated by their rule, server-side and client-side actions run synchronously. Server-side actions can also run asynchronously when called by the `runAction()` function from a client-side action. Asynchronous functions must always be stored in client-side actions.

Extensive server-side actions must be called asynchronously using the `runAction()` function so that the upcoming actions don't freeze the browser window by waiting for its response. In the rule of a server-side action called using `runAction()`, you must enter `X(*/*)` —the always false operator—because the action isn't activated by answers selected by users on the product interface.

The following examples show how to call a server-side action asynchronously from a client-side action:

```
1 | runAction('ACTION_CODE').done(callback);
```

```
1 | (async function() {
2 |   //code
3 |   await runAction('ACTION_CODE');
4 |   //code
5 | })();
```

In the first example, the callback function runs after the server-side action has completed its task. For more information about using the `runAction()` function, see [Performing Server Requests in Batches](#).

Establishing Client-Server Communication

Server-side actions can't access the window object to obtain information from the client, and client-side actions can't access the server. Client and server-side actions can share information through the following resources:

- [Questions and Answers](#)

- [The Scratchpad Variable](#)
- [Function-Generated Tables](#)
- [QTables](#)

Questions and Answers

Client-side and server-side actions can store and obtain information from questions and answers through universal functions, such as `setValue()` and `getValue()`. After submitting the configuration, this information is saved to the configuration record. With this method, you can share information from client to server and the other way around. Use this method to share a small amount of data that will be displayed on the product interface or used in rules.

For more information about implementing questions and answers in client-server communication, see [Transferring Data Using Questions and Answers](#).

The Scratchpad Variable

The scratchpad variable is a temporary object available for both client- and server-side actions. In the scratchpad variable, you can store any data type and transfer it from client to server and the other way around. After submitting the configuration, the scratchpad variable is cleared and isn't saved to the configuration record. Use this option for large amounts of temporary data that don't need to be stored in the configuration record.

The following example shows how to save data obtained from the server to the scratchpad variable.

```
1 | scratchpad.companyname = customer.companyname;
```

The following example shows how to save data obtained from the client to the scratchpad variable.

```
1 | scratchpad.sku = prompt('Enter the item SKU');
```

For more information about implementing the scratchpad variable in client-server communication, see [Transferring Data Using the Scratchpad Variable](#).

Function-Generated Tables

Client-side and server-side actions can store and obtain data through function-generated tables (formerly called CE-generated tables) because they act like variables. However, data must have a table-like structure in the form of an array of objects where each object is a row, and each key of the object is a column. Use this option to share data from the server to the client when you work with input box popup table answers (formerly called picklists). To share data from the client to the server, use the scratchpad variable.

For more information about transferring data with function-generated tables, see [Transferring Data from Server to Client with Function-Generated Tables](#).

QTables

Client-side and server-side actions can store and obtain information from qTables because they act like variables. Data must have a table-like structure. Use this option to share data from the server to the client when you need to display the information on the product interface as a table. To share data from the client to the server, use the scratchpad variable.

For more information about transferring data with qTables, see [Transferring Data from Server to Client with QTables](#).

Implementing Client-Side and Server-Side Actions

Note: If you don't update actions, they will no longer work with future versions of NetSuite CPQ Configurator.

Complete the following steps to implement client-side and server-side actions:

1. In your sandbox account, verify that products and actions work as expected with the latest version of NetSuite CPQ Configurator. For more information, see [Verify the Compatibility with Client-Side and Server-Side Actions](#).
2. Separate non-working actions into client-side and server-side. For more information, see [Separating Actions into Client-Side and Server-Side](#).
3. If you use common actions, separate client-side and server-side functions into different common actions. For more information, see [Separating Client-Side and Server-Side Functions in Common Actions](#).
4. Update any custom event handlers attached to user interface elements or any CPQ modules that directly call server-side functions. For more information, see [Calling Actions from Event Handlers Attached to User Interface Elements](#) and [Calling Actions from CPQ Modules](#).
5. If products work as expected after implementing client-side and server-side actions, apply the changes to your production account with the latest version of NetSuite CPQ Configurator. For more information, see [Applying Changes to Production Accounts](#).

Verify the Compatibility with Client-Side and Server-Side Actions

In your sandbox account, verify that your implementations are compatible with client-side and server-side actions.

To verify the compatibility with the new action types:

1. In your sandbox account, make a copy of the existing products:
 1. Go to the options record at CPQ > Configurator > Settings.
 2. Export the products to your computer.
 3. Import the products back into your sandbox account.

For more information about exporting and importing products, see [Exporting Products](#) and [Importing Products](#).

2. On each product record, check the **Block database functions in client-side actions** box.
This option prevents functions that read and write data to the database from working on the client.
3. Make sure products and actions continue to work as expected. You should test the products using typical end-to-end scenarios in your business. These scenarios may include:
 1. Launching the product from a transaction or a website.
 2. Configuring all aspects of a product.
 3. Saving the configured product to a transaction or the shopping cart.

Separating Actions into Client-Side and Server-Side

If products and actions don't work as expected with the latest version of NetSuite CPQ Configurator, separate actions into client-side and server-side. For more information about specific use cases, see [Code Examples](#).

To separate actions into client-side and server-side:

1. In NetSuite, go to CPQ > Configurator > Product Maintenance.
2. Edit a product record.
3. Go to the **Actions** subtab:
 - Click **Edit** to modify actions.
 - Click **New CPQC Action** to create actions.
4. Create or edit actions as required to obtain client-side or server side actions. For more information see [Customizing Products with Scripts Using Action Records and Functions](#) and [Using Client-Side and Server-Side Actions](#).
5. In client-side actions, store:
 - Client-side functions.
For more information, see [List of Functions](#).
 - Asynchronous functions in JavaScript ES6.
6. In server-side actions:
 - Store server-side functions.
 - Check the **Run on server** box.
7. Decide how to establish the communication between client and server. For more information, see [Establishing Client-Server Communication](#).
8. Rearrange sequence numbers so actions run in the correct order.
9. Make sure products continue to work as expected.
10. If products continue to work as expected, export the new product versions from the options record.
11. On the options record, import the new production versions back into the old product versions using the **Target Product** field.
This option preserves the product ID.
For more information, see [Exporting Products](#) and [Importing Products](#).

Separating Client-Side and Server-Side Functions in Common Actions

Common actions inherit their type from the action to which they are linked. Common actions linked to server-side actions can only include server-side functions, while common actions linked to client-side actions can only include client-side functions. For more information about common actions, see [Sharing Code with Common Actions](#).

To work with client-side and server side functions in common actions:

1. Go to CPQ > Configurator > Common Actions.
 - Click **Edit** next to a common action to modify it.
 - Click **New CPQC Common Action** to create a common action.
2. Separate client-side and server-side functions into separate common actions. Create new common actions when necessary. For more information, see [Code Examples](#).
3. Decide how to establish the communication between client and server. For more information, see [Establishing Client-Server Communication](#).
4. Modify the actions linked to common actions to be a server-side or a client-side action. For more information, see [Separating Actions into Client-Side and Server-Side](#).

5. Create new client-side or server-side actions to link to the new common actions.

Common actions are automatically included in the product export.

Applying Changes to Production Accounts

If products work as expected after implementing client-side and server-side actions in your sandbox account, apply the changes to your production account using the latest version of NetSuite CPQ Configurator.

To apply the changes to your production account:

1. In your sandbox account, go to the options record at CPQ > Configurator > Settings.
2. Export the products with client-side and server-side actions to your computer.
3. In your production account, go to the options record.
4. Import new products into the original ones using the **Target Product** field. This option preserves the product ID.

For more information about exporting and importing products, see [Exporting Products](#) and [Importing Products](#).

Code Examples

If actions combine database functions with user interface, window interaction, and status functions, you must separate them into client-side and server-side actions. See the following example:

Original Action	New Actions
<pre> 1 // A. Loads the file 2 loadFile('MyFolder/myFile.txt').done(function(data) { 3 // B. Shows the message 4 showMessage(data); 5 }); </pre>	Server-Side Action <pre> 1 // A. Server-side action (Check the Run on server box) 2 // Loads the file 3 loadFile('MyFolder/myFile.txt').done(function(data) { 4 // Saves file data to the scratchpad variable 5 scratchpad.fileContent = data; 6 }); </pre> Client-Side Action <pre> 1 // B. Client-side action 2 // Shows the message stored in the scratchpad variable 3 showMessage(scratchpad.fileContent); </pre>

The original action loads a file from the File Cabinet and shows its content in a popup window on the user interface. The function `loadFile()` interacts with the database and must be stored in a server-side action, while `showMessage()` works with the user interface and requires a client-side action.

After checking the **Run on server** box on the action record, edit the action by adding the scratchpad variable (`scratchpad.fileContent`) where you save the file data to be sent to the client. Then, remove the `showMessage()` function and create a client-side action to store it. The client-side action obtains data from the scratchpad variable and shows the file content in a popup window on the user interface.

The previous example shows a simple use case. The following code examples help you implement client-side and server-side actions for other common scenarios:

- [Transferring Data Using Questions and Answers](#)

- Transferring Data Using the Scratchpad Variable
- Transferring Data from Server to Client with QTables
- Transferring Data from Server to Client with Function-Generated Tables
- Performing Server Requests in Batches
- Calling Actions from Event Handlers Attached to User Interface Elements
- Calling Actions from CPQ Modules

Transferring Data Using Questions and Answers

Client-side and server-side actions can share information using questions and answers. With questions and answers, you can share information from the server to the client and the other way around. During implementation, you may need to create the questions and answers that will store the data to be transferred. For more information about questions and answers, see [Gathering Information with Questions and Answers](#).

Transferring Data from Server to Client Using Questions and Answers

The following example shows how to transfer data from the server to the client with questions and answers.

Original Action	New Actions
<pre> 1 // A. Obtains customer data 2 var currentCustomer = getValue("NS", "ENTITYID"); 3 getData({ 4 type: 'customer', 5 filter: ['internalid', 'is', currentCustomer], 6 fields: ['companyname'] 7 }).done(function(data) { 8 var customer = data[0]; 9 //B. Displays customer data on the user interface 10 showMessage(customer.companyname); 11 }); </pre>	Server-Side Action <pre> 1 // A. Server-side action (Check Run on server box) 2 // Obtains customer data 3 var currentCustomer = getValue("NS", "ENTITYID"); 4 getData({ 5 type: 'customer', 6 filter: ['internalid', 'is', currentCustomer], 7 fields: ['companyname'] 8 }).done(function(data) { 9 var customer = data[0]; 10 // Saves customer data in an answer 11 setValue('DATA', 'CUSTOMER', customer.companyname); 12 }); </pre> Client-Side Action <pre> 1 // B. Client-side action 2 // Obtains customer data from the answer 3 var companyname = getValue('DATA', 'CUSTOMER'); 4 // Displays customer data on the user interface 5 showMessage(companyname); </pre>

The original action obtains customer information using the `getData()` function—a server-side function. Then, the customer information is displayed on the user interface in a popup window using the `showMessage()` function—a client-side function.

Because the original action combines a server-side function and a client-side function, you must separate it into a server-side and a client-side action. After obtaining the customer data with the `getData()` function, the new server-side action A saves the information to the DATA/CUSTOMER answer using the universal function `setValue()`. Storing the customer information in an answer makes it available for the client-side action. The new client-side action B retrieves the customer information from the DATA/CUSTOMER answer using the universal function `getValue()` and displays it in a popup window.

Transferring Data from Client to Server Using Questions and Answers

The following example shows how to transfer data from client to server with questions and answers.

Original Action	New Actions
<pre> 1 // A. Asks users to provide the item SKU 2 var sku = prompt('Enter item SKU'); 3 // B. Searches for items with the provided item SKU 4 getData({ 5 type: 'item', 6 filter: ['itemid', 'is', sku], 7 fields: ['itemid', 'displayname'] 8 }).done(function(data) { 9 setValue('DATA', 'ITEMS', data); 10 }); </pre>	Client-Side Action <pre> 1 // A. Client-side action 2 // Asks users to provide the item SKU 3 var sku = prompt('Enter item SKU'); 4 // Saves the item SKU in an answer 5 setValue('ITEM', 'SKU', sku); </pre> Server-Side Action <pre> 1 // B. Server-side action (Check Run on server box) 2 // Obtains the item SKU from the answer 3 var sku = getValue('ITEM', 'SKU', sku); 4 // Searches for items with the provided item SKU 5 getData({ 6 type: 'item', 7 filter: ['itemid', 'is', sku], 8 fields: ['itemid', 'displayname'] 9 }).done(function(data) { 10 setValue('DATA', 'ITEMS', data); 11 }); </pre>

The original action asks users to provide the item SKU using the `prompt()` function—a JavaScript client-side function. Then, the original action searches for the items with the provided SKU using `getData()`—a server-side function. Finally, the item SKU is saved to the answer DATA/ITEMS.

Because the original action combines a client-side function and a server-side function, you must separate it into a server-side action and a client-side action. After prompting the user to provide the item SKU, the new client-side action A saves the information to the answer ITEM/SKU using the universal function `setValue()`. Storing the item SKU in an answer makes it available for the server-side action. The new server-side action B retrieves the item SKU from the answer using the universal function `getValue()`, searches for the item SKU, and saves it to an answer.

Transferring Data Using the Scratchpad Variable

Client-side and server-side actions can share information using the scratchpad variable. With the scratchpad variable, you can share information from the server to the client and the other way around.

Transferring Data from Server to Client Using the Scratchpad Variable

The following example shows how to transfer data from the server to the client using the scratchpad variable.

Original Action	New Actions
<pre> 1 // A. Obtains customer data 2 var currentCustomer = getValue("NS", "ENTITYID"); 3 getData({ 4 type: 'customer', 5 filter: ['internalid', 'is', currentCustomer], 6 fields: ['companyname'] 7 }).done(function(data) { 8 var customer = data[0]; 9 // B. Displays customer data on the user interface 10 showMessage(customer.companyname); 11 }); </pre>	Server-Side Action <pre> 1 // A. Server-side action (Check Run on server box) 2 // Obtains customer data 3 var currentCustomer = getValue("NS", "ENTITYID"); 4 getData({ 5 type: 'customer', 6 filter: ['internalid', 'is', currentCustomer], 7 fields: ['companyname'] 8 }).done(function(data) { 9 var customer = data[0]; 10 //Saves customer data to the scratchpad variable 11 scratchpad.companyname = customer.companyname; 12 }); </pre> Client-Side Action

Original Action	New Actions
	<pre> 1 // B. Client-side action 2 // Obtains data from the scratchpad variable and displays it on the user interface 3 showMessage(scratchpad.companyname); </pre>

The original action obtains customer information using `getData()`—a server-side function. Then, customer information is displayed on the user interface in a popup window using `showMessage()`—a client-side function.

Because the original action combines a server-side function and a client-side function, you must separate it into a server-side and a client-side action. After obtaining the customer data, the new server-side action A saves the information to the scratchpad variable (`scratchpad.companyname`). The customer information becomes available for the new client-side action B. The client-side action B retrieves the customer information from the scratchpad variable and displays it in a popup window on the user interface.

Transferring Data from Client to Server Using the Scratchpad Variable

The following example shows how to transfer data from client to server using the scratchpad variable.

Original Action	New Actions
<pre> 1 // A. Asks users to provide the item SKU 2 var sku = prompt('Enter item SKU'); 3 // B. Searches for items with the provided item SKU 4 getData({ 5 type: 'item', 6 filter: ['itemid', 'is', sku], 7 fields: ['companyname'] 8 }).done(function(data) { 9 setValue('DATA', 'ITEMS', data); 10 }); </pre>	Client-Side Action <pre> 1 // A. Client-side action 2 // Asks users to provide the item SKU and saves data to the scratch pad variable 3 scratchpad.sku = prompt('Enter item SKU'); </pre> Server-Side Action <pre> 1 // B. Server-side action (Check Run on server box) 2 // Searches for items by obtaining the item SKU from the scratchpad variable 3 getData({ 4 type: 'item', 5 filter: ['itemid', 'is', scratchpad.sku], 6 fields: ['companyname'] 7 }).done(function(data) { 8 setValue('DATA', 'ITEMS', data); 9 }); </pre>

The original action asks users for the item SKU using the `prompt()` function—a JavaScript client-side function. Then, the original action searches for the items with the provided SKU using `getData()`—a server-side function. Finally, the item SKU is saved to the answer DATA/ITEMS.

Because the original action combines a client-side function and a server-side function, you must separate it into a server-side action and a client-side action. After prompting users for the item SKU, the new client-side action A saves the information to the scratchpad variable (`scratchpad.sku`). The item SKU becomes available for the server-side action B. The new server-side action B retrieves the item SKU from the scratchpad variable and searches for the item SKU.

Transferring Data from Server to Client with QTables

You can transfer data from server to client using `qTables`. See the following example:

Original Action	New Actions
<pre> 1 // A. Asks users to provide the beginning of the item SKU </pre>	Client-Side Action

Original Action	New Actions
<pre> 2 var sku = prompt('Enter the beginning of the item SKU'); 3 // B. Searches for items by the beginning of the item SKU 4 getData({ 5 type: 'item', 6 filter: ['itemid', 'startswith', sku], 7 fields: ['itemid'] 8 }).done(function(data) { 9 //B1. Stores the found items in a qTable and displays it on the user interface 10 var t = getQTable('ITEMS'); 11 t.setRowCount(data.length); 12 for (var i = 0; i < data.length; i++) { 13 t.set('ITEM', i + 1, data[i].itemid); 14 } 15 }); </pre>	<pre> 1 // A. Client-side action 2 //Asks users to provide the beginning of the item SKU and stores the user input in the scratchpad variable 3 scratchpad.sku = prompt('Enter the beginning of the item SKU'); </pre> Server-Side Action <pre> 1 // B. Server-side action (Check Run on server box) 2 // Searches for items by the beginning of the item SKU 3 getData({ 4 type: 'item', 5 filter: ['itemid', 'startswith', scratchpad.sku], 6 fields: ['itemid'] 7 }).done(function(data) { 8 //B1. Stores the found items in a qTable 9 var t = getQTable('ITEMS'); 10 t.setRowCount(data.length); 11 for (var i = 0; i < data.length; i++) { 12 t.set('ITEM', i + 1, data[i].itemid); 13 } 14 }); </pre> Client-Side Action <pre> 1 // C. Client-side action 2 //Refreshes the user interface to display the found items in the qTable 3 renderView(); </pre>

For more information about qTables, see [QTable Questions](#).

The original action asks users to provide the beginning of the item SKU using the `prompt()` function—a JavaScript client-side function. Then, the original action searches for items that match those characters using `getData()`—a server-side function. Finally, the found item SKUs are saved to a qTable and displayed on the user interface.

Because the original action combines a client-side function, a server-side function, and updates the qTable on the user interface, you must separate it into a server-side action and multiple client-side actions. After prompting the user for the beginning of item SKU, the client-side action A saves the information to the scratchpad variable (`scratchpad.sku`).

The information saved to the scratchpad variable becomes available for the server-side action. The new server-side action B obtains the information from the scratchpad variable and searches for item SKUs that match the user input. The found items are saved to a qTable. To update the qTable, the client-side action C refreshes the user interface using the `renderView()` function and displays the found item SKUs in the qTable.

Transferring Data from Server to Client with Function-Generated Tables

You can transfer data from the server to the client using function-generated tables automatically filled in with data from your account. See the following example:

Original Action	New Actions
<pre> 1 // A. Asks users to provide the beginning of the item SKU 2 var sku = prompt('Enter the beginning of the item SKU'); 3 // B. Searches for items by the beginning of the item SKU 4 getData({ 5 type: 'item', 6 filter: ['itemid', 'startswith', sku], 7 fields: ['itemid'] 8 }).done(function(data) { </pre>	Client-Side Action <pre> 1 // A. Client-side action 2 // Asks users to provide the beginning of the item SKU and stores the user input in the scratchpad variable 3 scratchpad.sku = prompt('Enter the beginning of the item SKU'); </pre>

Original Action	New Actions
<pre> 9 //B1. Stores the found items in a function-generated table 10 var t = getTable('ITEMS'); 11 t.set(data); 12 t.setStatus('ready'); 13 };</pre>	Server-Side Action <pre> 1 // B. Server-side action (Check Run on server box) 2 // Searches for items by the beginning of the item SKU 3 getData({ 4 type: 'item', 5 filter: ['itemid', 'startswith', scratchpad.sku], 6 fields: ['itemid'] 7 }).done(function(data) { 8 var t = getTable('ITEMS'); 9 t.set(data); 10 t.setStatus('ready'); 11 });</pre>

For more information about function-generated tables, see [Creating Tables Using Functions](#).

The original action asks users to provide the beginning of the item SKU using the `prompt()` function—a JavaScript client-side function. Then, the original action searches for items that start with the provided characters using `getData()`—a server-side function. Finally, the found item SKUs are saved to a function-generated table and can be used, for example, in an input box popup table answer.

Because the original action combines a client-side function and a server-side function, you must separate it into a server-side action and a client-side action. After prompting the user to provide the beginning of the item SKU, the new client-side action A saves the information to the scratchpad variable (`scratchpad.sku`). This information becomes available for the server-side action. The new server-side action B obtains the information from the scratchpad variable and searches for item SKUs that match the user input. The found items are saved in a function-generated table, which is available for use in client-side actions. For example, you can use it to provide data for input box popup table answers.

Performing Server Requests in Batches

Your current implementation may include actions where database functions perform multiple operations of the same type and a separate server request is sent for each operation. For example, if an action creates 20 records, the server receives 20 requests—one for each record. However, the new architecture requires database functions to be stored in server-side actions. Server-side actions have limits on the number of server requests and operations they can process. To overcome this limitation, you can batch server requests so that a single server request processes multiple operations.

To implement this solution, you can create loops in both the client-side and the server-side actions. The client-side action keeps track of the operations to be processed. Its loop calls the server-side action until all operations are completed.

The server-side action keeps track of the governance usage units. Its loop processes multiple operations in a single server request. If it runs out of usage units, the server-side action exits the loop. The client-side action relaunches the server-side action if there are still operations to be processed. See the following example:

Original Action	New Actions
<pre> 1 // A. Creates multiple records 2 (async function() { 3 var records = getValue('DATA', 'RECORDS'), 4 created = []; 5 for (var i = 0; i < records.length; i++) { 6 var newRecord = await createRecord(records[i]); 7 created.push(newRecord.id); 8 } 9 setValue('DATA', 'CREATED', created); 10 })();</pre>	Client-Side Action <pre> 1 // A. Client-side action 2 (async function() { 3 scratchpad.output = []; 4 scratchpad.records = getValue('DATA', 'RECORDS'); // Array of 5 records to be created 6 scratchpad.index = 0; 7 let count = scratchpad.records.length; 8 do {</pre>

Original Action	New Actions
	<pre> 8 await runAction('CREATE_RECORDS'); 9 } while (scratchpad.index < count); 10 console.log('FINISH', scratchpad.output); 11 }()); </pre> Server-Side Action <pre> 1 // B. Server-side action (Check Run on server box) 2 let records = scratchpad.records, 3 count = records.length, 4 weight = 0; 5 do { 6 let remaining = getRemainingUnits(); 7 if (remaining < weight) { 8 let unprocessed = count-scratchpad.index; // records to be 9 created on the following requests 10 console.log('Operation completed. Records left:', 11 unprocessed); 12 restartAction(); 13 } 14 let record = records[scratchpad.index]; 15 createRecord(record).done((data) => { 16 scratchpad.output.push(data); 17 }); 18 if (!weight) { 19 weight = remaining - getRemainingUnits(); // how many units 20 are consumed by a single iteration 21 } 22 scratchpad.index++; 23 } while (scratchpad.index < count); </pre>

The original action stores the list of records to be created in the answer DATA/RECORDS. Then, the action loops through the list and creates the records one by one. The `createRecord()` function sends a server request for every record to be created.

There is no information about how many records the answer stores. If you implement a `for` loop in a server-side action, the action may fail because if the required operations exceed the usage unit limit.

In the new implementation, the client-side function A stores the list of records to be created in the `scratchpad` variable. Then, it starts a loop that calls the server-side action using the `runAction()` function until all records are created. As you can see from the example, client-side actions can use asynchronous functions, which are supported in JavaScript ES6.

The server-side action B retrieves the list of records to be created from the `scratchpad` variable. Then, it dynamically calculates the usage units at the beginning and the end of each iteration through the `remainingUsageUnits()` function.

If there are enough usage units, the server-side action runs the loop to create the records. If there aren't enough usage units, `restartAction()` stops the loop and sends information to the client-side action about whether to restart the server-side action. Then, the server-side action sends information about the created records to the client-side action using the `scratchpad` variable. The client-side action verifies the number of created records against the total number of records required. If there are still records left, the client-side action relaunches the server-side action until all records are created.

Calling Actions from Event Handlers Attached to User Interface Elements

Custom event handlers attached to user interface elements can't directly call server-side functions. Instead, you must:

- Store the server-side function in a server-side action.
- Call the server-side action using the `runAction()` function from the custom script.

- Use questions and answers to share information between the client and server because the scratchpad variable isn't available in custom scripts.
- In the rule of the server-side action, enter X(*/*)—the always false operator—because the action isn't activated by answers users select on the product interface.

The following example shows how to implement `runAction()` in an event handler attached to a user interface element.

Original Custom Script	New Implementation
<pre> 1 // Custom script 2 // A. Adds an onclick event handler to a user interface button 3 jQuery('#my-button').click(function() { 4 var currentCustomer = getValue('NS', 'ENTITYID'); 5 // B. Obtains the customer name 6 getData({ 7 type: 'customer', 8 filter: ['internalid', 'is', currentCustomer], 9 fields: ['companyname'] 10 }).done(function(data) { 11 var customer = data[0]; 12 //C. Displays the customer name on the user interface (if 13 //users click the button) 14 showMessage(customer.companyname); 15 }); 16 }); </pre>	Updated Custom Script <pre> 1 // Custom script 2 // A. Adds an onclick event handler to a user interface button 3 jQuery('#my-button').click(function() { 4 // Calls the server-side action 5 runAction('LOAD_DATA').done(function() { 6 var companyname = getValue('DATA', 'CUSTOMER'); 7 // C. Displays the customer name on the user interface (if 8 //users click the button) 9 showMessage(companyname); 10 }); 11 }); </pre> Server-Side Action <pre> 1 // Server-side action 2 // Check the Run on server box 3 // B. Obtains customer data 4 var currentCustomer = getValue('NS', 'ENTITYID'); 5 getData({ 6 type: 'customer', 7 filter: ['internalid', 'is', currentCustomer], 8 fields: ['companyname'] 9 }).done(function(data) { 10 var customer = data[0]; 11 // Stores customer data in an answer 12 setValue('DATA', 'CUSTOMER', customer.companyname); 13 }); </pre>

The original custom script adds an event handler to a user interface button. The custom script retrieves customer data using `getData()`—a server-side function. When users click the button, the customer name is displayed in a popup window using `showMessage()`—a client-side function.

In the new implementation, the custom script still adds the event handler and uses `showMessage()` to display the information. However, instead of calling `getData()` directly, the new version retrieves the customer information by calling a server-side action using `runAction()`. The server-side action B loads the customer data and stores it in an answer to make it available on client.

Calling Actions from CPQ Modules

CPQ modules are client scripts and can't directly call server-side functions. Instead, you must:

- Store server-side functions in a server-side action.
- Call the server-side action from the module using the `runAction()` function.
- Use questions and answers to share information between modules and server-side actions, because modules can't access the scratchpad variable.
- In the rule of the server-side action, enter X(*/*)—the always false operator—because the action isn't activated by answers users select on the product interface.

The following example shows how to implement `runAction()` in a module.

Original Module	New Implementation
<pre> 1 <script script-type="module" type="text/javascript"> 2 var app = angular.module('formApp'); 3 app.modController('modctrl', [function () { 4 this.getCost = function() { 5 var sku = getValue('ITEMS', 'SKU'); 6 if (sku) { 7 // Loads the item cost 8 getData({ 9 type: 'item', 10 filter: ['itemid', 'is', sku], 11 fields: ['cost'] 12 }).done(function(data) { 13 var cost = data[0].cost; 14 showMessage(sku+' cost is '+cost); 15 }); 16 } else { 17 alert('Please enter the item SKU'); 18 } 19 }); 20 }); 21 </script> 22 <div ng-controller="modctrl as mc"> 23 <div style="margin:10px;"> 24 <div>Load item cost</div> 25 <button style="margin:10px;padding:5px 15px;" type="button" ng- 26 click="mc.getCost();">Show cost</button> 27 </div> </pre>	Updated Module <pre> 1 <script script-type="module" type="text/javascript"> 2 var app = angular.module('formApp'); 3 app.modController('modctrl', [function () { 4 this.getCost = function() { 5 var sku = getValue('ITEMS', 'SKU'); 6 if (sku) { 7 runAction('ITEM_COST').done(function(data) { 8 var cost = getValue('MISC', 'COST'); 9 showMessage(sku+' cost is '+cost); 10 }); 11 } else { 12 alert('Please enter the item SKU'); 13 } 14 }); 15 }); 16 </script> 17 <div ng-controller="modctrl as mc"> 18 <div style="margin:10px;"> 19 <div>Load item cost</div> 20 <button style="margin:10px;padding:5px 15px;" type="but 21 ton" ng-click="mc.getCost();">Show cost</button> 22 </div> </pre> Server-Side Action <pre> 1 // Server-side action 2 // Check the Run on server box 3 var sku = getValue('ITEMS', 'SKU'); 4 getData({ 5 type: 'item', 6 filter: ['itemid', 'is', sku], 7 fields: ['cost'] 8 }).done(function(data) { 9 setValue('MISC', 'COST', data[0].cost); 10 }); </pre>

When users click the button on the user interface, the original module retrieves the item SKU from the answer ITEMS/SKU. Then, the module searches the item cost for that item by directly calling the `getData()` function—a server-side function. Finally, the item cost is displayed in a popup window on the user interface using the `showMessage()` function—a client-side function.

In the new implementation, a server-side action has been created to store the `getData()` function. The updated module calls the server-side action using the `runAction()` function and shares the item SKU through the answer ITEMS/SKU. The server-side action uses the `getData()` function to obtain the item SKU from the answer ITEMS/SKU, searches for the item cost, and saves it to the answer MISC/COST to make it available for the module. Then, the module retrieves the item cost from the answer and displays it.

List of Functions

The following table classifies functions as client-side, server-side, or both. Any function marked with **Yes** in both columns is a universal function.

The table also contains information about governance usage units for server-side functions. Because NetSuite CPQ Configurator is a Suitelet, each server-side action can use up to 1,000 usage units. However, some units are used to launch the action itself, so the maximum usage units allowed for each server-side action is 927. Depending on the record type, the governance units used vary:

- Transaction records – Use 10 units each.
- Custom records – Use 2 units each.

- All other records – Use 5 units each.

In the table, these units are referred to as **record type units**.

For more information about governance usage units in NetSuite, see the help topic [SuiteScript Governance and Limits](#).

Function	Client-Side	Server-Side	Governance Usage Units
addBreakoutItem()	Yes	—	—
addLink()	Yes	—	—
addMaterial()	Yes	—	—
addMessageListener()	Yes	—	—
addRuleset()	Yes	—	—
ajax()	Yes	—	—
changeRuleset()	Yes	—	—
closeConfig()	Yes	—	—
copyRecord()	—	Yes	Record type units × 3
createEvent()	Yes	—	—
createRecord()	—	Yes	Record type units × 3
createRecords()	—	Yes	Record type units × 3 × Number of records to be created
createTable()	Yes	Yes	None
customizeUI()	Yes	—	—
deleteEvent()	Yes	—	—
deleteFile()	—	Yes	■ Deleting by id: 20 ■ Deleting by name: 30
deleteRecord()	—	Yes	Record type units
evalRule()	Yes	Yes	None
findList()	Yes	—	—
getAnswerCode()	Yes	Yes	None
getAnswerDesc()	Yes	—	—
getAnswerLabel()	Yes	—	—
getAnswerName()	Yes	—	—
getAnswers()	Yes	Yes	None
getAnswersCodes()	Yes	Yes	None
getAnswerVariable()	Yes	—	—
getContextVariable()	Yes	—	—

Function	Client-Side	Server-Side	Governance Usage Units
getData()	—	Yes	- Using the id parameter: Number of required record types × Record type units Process records with the same record type units - Without the id parameter: (Number of required record types ÷ 10) + 5
getEvent()	Yes	—	—
getEvents()	Yes	—	—
getFieldOptions()	—	Yes	Record type units + (Number of required options ÷ 10) + (Number of fields × 5)
getFromExternal()	Yes	—	—
getItemPrice()	—	Yes	(Number of price levels ÷ 10) + 5
getLineNum()	Yes	—	—
getMovDate()	—	Yes	None
getOrderData()	Yes	Yes	30
getOrderField()	Yes	—	—
getOrderItems()	Yes	—	—
getOrderSublist()	Yes	—	—
getParsedValue()	Yes	Yes	None
getProgress()	Yes	—	—
getQTable()	Yes	Yes	None
getRemainingUnits()	—	Yes	None
getRuleset()	Yes	—	—
getTable()	Yes	Yes	None
getTable().loadContent()	Yes	Yes	- Dynamic tables: (Number of required tables ÷ 10) + 10 - Static tables: 10 - CSV tables: 20 - Function-generated tables: none
getValue()	Yes	Yes	None
hasAnswer()	Yes	Yes	None
hideMessage()	Yes	—	—
hideModal()	Yes	—	—
isAnswered()	Yes	Yes	None
loadConfig()	—	Yes	Loading from the File Cabinet: 10

Function	Client-Side	Server-Side	Governance Usage Units
			■ All other cases: 5
loadFavorite()	—	Yes	—
loadFile()	—	Yes	10
loadListKeyValue()	Yes	—	—
loadListWithArray()	Yes	—	—
nresolve()	Yes	Yes	None
priceMatrix()	Yes	Yes	See getItemPrice()
processItemCreations()	Yes	—	—
processRecordCreations()	Yes	—	—
queryParent()	Yes	—	—
readFiles()	Yes	—	—
recalcRulesets()	Yes	—	—
removeLink()	Yes	—	—
renderView()	Yes	—	—
reset()	Yes	Yes	None
resetTab()	Yes	—	—
resolve()	Yes	Yes	None
restartAction()	—	Yes	None
runAction()	Yes	—	—
runSavedSearch()	—	Yes	5
saveFile()	—	Yes	10
scopeApply()	Yes	—	—
scopeRefresh()	Yes	—	—
sendMessage()	Yes	—	—
setAnswer()	Yes	Yes	None
setDebugMode()	Yes	—	—
setFormattedValue()	Yes	Yes	None
setPicklistColumn()	Yes	—	—
setTab()	Yes	—	—
setValue()	Yes	Yes	None
showAudit()	Yes	—	—
showMessage()	Yes	—	—

Function	Client-Side	Server-Side	Governance Usage Units
showModal()	Yes	—	—
showPicklistModal()	Yes	—	—
sresolve()	Yes	Yes	None
startConfig()	Yes	—	—
submitConfig()	Yes	—	—
updateRecordFields()	—	Yes	Record type units × 2
updateTable()	—	Yes	12
validateAddress()	Yes	—	—

The `getOrderData()` function is available on the server only if the **Submit to transaction page** box is checked on the options record. This option sends configuration data to the transaction page after submitting the configuration. For more information, see [Sending Data to the Transaction Page](#).

The methods of the `table` and `qTable` objects are available in both client- and server-side actions. You can obtain these objects using the `getTable()` and `getQTable()` functions, respectively.

Optimizing Server-Side Actions

You can achieve optimal product performance by consolidating server-side actions. If you combine multiple tasks into a single, larger server-side action, server requests are reduced and performance improved. An optimized product typically launches a minimum number of server-side action requests—ideally one—for each user action, such as a click or data entry.

This approach helps prevent server response delays caused by multiple requests for a single action. For examples of optimized server-side actions, see [Code Examples](#). For more information about verifying server-side actions performance, see [Verifying Server-Side Actions Performance](#).

Another way to optimize server-side actions is to apply the lightweight mode, which only loads product data when required. This mode enhances performance for non-optimized server-side actions that don't interact with product data. Examples of functions that require product data are `setAnswer()`, `getTable()`, and `getQTable()`.

When applying the lightweight mode, use the scratchpad variable to transfer data from the server to the client. Avoid using questions and answers, `qTables`, and function-generated tables for this purpose. Additionally, excluding functions that load product data can further improve server-side action performance. Whenever possible, it's best to use these functions in client-side actions to optimize performance.

The list of functions that load product data—even in lightweight mode—includes:

- `createTable()`
- `getAnswerDesc()`
- `getAnswerLabel()`
- `getAnswerName()`
- `getAnswerVariable()`
- `getQTable()`
- `getTable()`
- `reset()`

- `setAnswer()`
- `setValue()`
- `updateTable()`

To enable the lightweight mode:

1. Go to CPQ > Configurator > Settings.
2. Check the **Enable lightweight mode** box.
3. Click **Save**.

Verifying Server-Side Actions Performance

You can verify the performance of server-side actions by launching a product and appending the "&debug=T" string to the product URL. Then, open the console and analyze the detailed logs for each server-side action. The following image shows an example of logs that can help you identify potential performance issues.

```

SSAPerf : Create SERIAL ▶ [{-}]
:: SSA : "CREATE_SERIAL" : perflog ▼ {tasks: Array(6), stats: {-}, weight: {-}} ⓘ
  ▼ stats:
    auth: "0.2%"
    http_request: "36.4%"
    load_product: "22.2%"
    other: "1.4%"
    run_ssa: "39.8%"
    ▶ [[Prototype]]: Object
  ▼ tasks: Array(6)
    ▶ 0: {name: 'http-data-send', time: 0.407}
    ▶ 1: {name: 'auth', time: 0.004}
    ▶ 2: {name: 'ssa-load-product', time: 0.554}
    ▶ 3: {name: 'ssa-run', time: 0.994}
    ▶ 4: {name: 'other', time: 0.035}
    ▶ 5: {name: 'http-data-receive', time: 0.501}
    length: 6
    ▶ [[Prototype]]: Array(0)
  ▼ weight:
    request: 806
    response: 173033
    ▶ [[Prototype]]: Object
    ▶ [[Prototype]]: Object
Server action callback CREATE_SERIAL
FINISH ▶ [Array(788)]
  
```

In the **tasks** section of the logs, you can view performance metrics (in seconds) for each server-side action:

- **auth** – Shows the time spent on authentication when a server-side action is initiated. The server must verify a user's authorization before allowing access to server-side functions.
A high value may indicate that a large amount of data (megabytes) is being sent. Consolidate server-side actions for large data transfers into a single server-side action to ensure a faster authentication and improve performance.
- **http-data-send** and **http-data-receive** – Show the time required to send data to and from the server. High values usually indicate that temporary data isn't being cleared when you send requests to the server. To optimize performance, remove large amounts of data from the scratchpad variable and answers after use.
- **ssa-load-product** – Indicates the time required for loading a product. If this key is present, the product has been loaded. If the product shouldn't be loaded, remove functions that load it from the server-side action.

- **other** – Corresponds to the time required to perform internal tasks related to the request and prepare the response for the client. A high value may indicate performance issues.

Alternatively, you can view the same metrics as percentages in the **stats** section. The **http-data-send** and **http-data-receive** keys are combined under the **http_request** key.

Configuring Items From Transactions

Users can access configurable items from eligible transactions. For more information about eligible transactions, see [Selecting Eligible Transactions](#).

The basic workflow to configure items is the following:

1. Open or create an eligible transaction.
2. Click the **Configure** button.
3. Select a configurable item on the Configurable Items page.
This page provides an overview of all products that represent configurable items.
4. Alternatively, click the arrow on the **Configure** button and select a specific configurable item or a category.
5. Configure the item on the user interface.
6. Click **Submit & Close** to add the configured item to the transaction. This button also closes the product interface.
7. Save the transaction.

To modify a configured item, edit the transaction and click the **Edit Config** button on the configured line item. If you convert a transaction, the converted transaction will contain a copy of the original configurations. You'll be able to edit the configured items in one transaction without affecting the other.